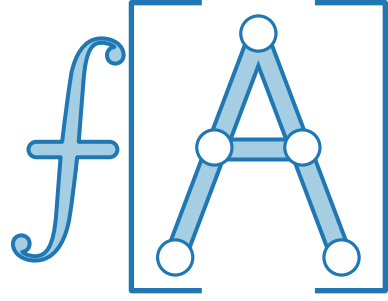
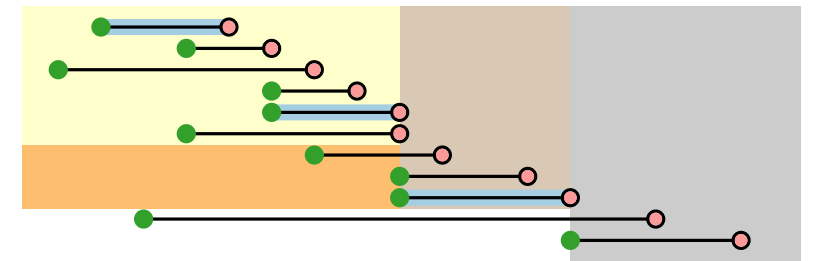
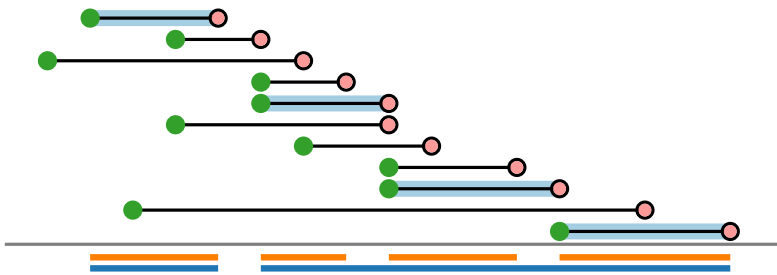




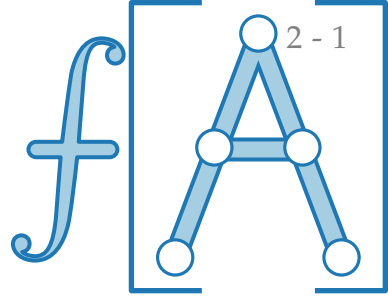
Fortgeschrittene Algorithmen

1. Vorlesung Greedy Algorithmen

Teil I: Ablaufplanung

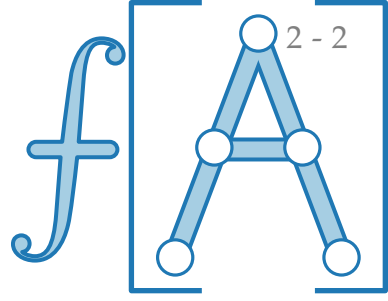


Entwurfstechniken



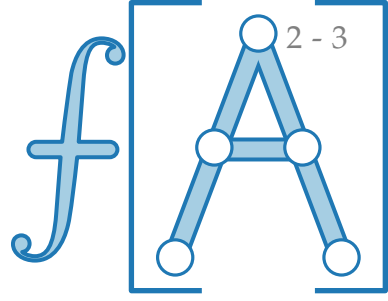
Entwurfstechniken

- Inkrementell



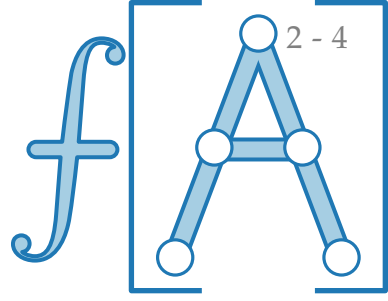
Entwurfstechniken

- Inkrementell
- Rekursiv



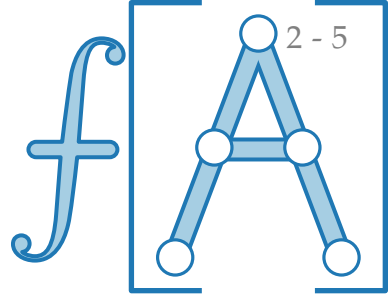
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche



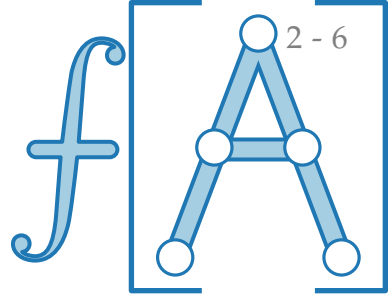
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy



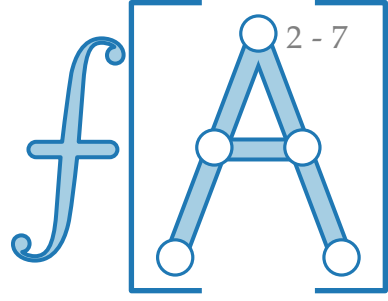
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren



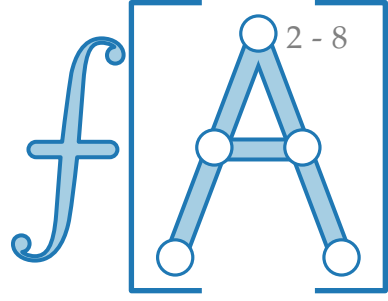
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren
- Randomisiert



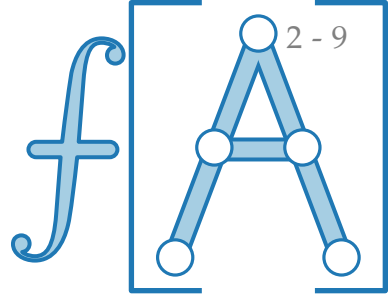
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren
- Randomisiert
- Parallel



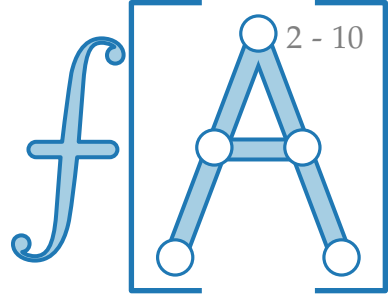
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren
- Randomisiert
- Parallel
- [Online]



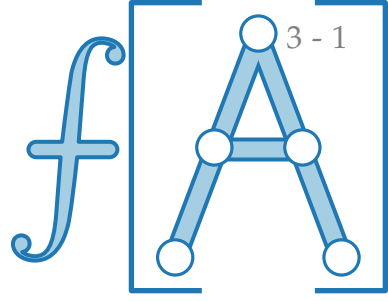
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren
- Randomisiert
- Parallel
- [Online]



Operations Research

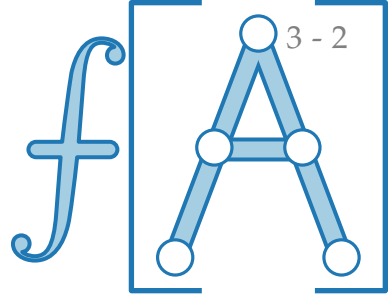
Optimierung für Wirtschaftsabläufe.



Operations Research

Optimierung für Wirtschaftsabläufe.

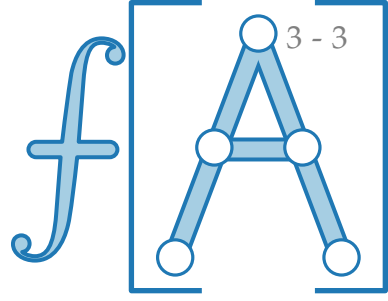
- Standortplanung



Operations Research

Optimierung für Wirtschaftsabläufe.

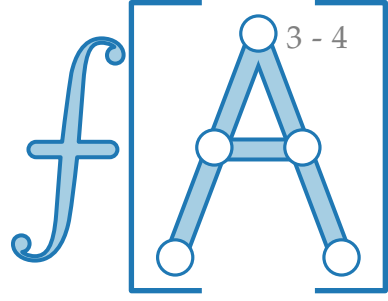
- Standortplanung
- Ablaufplanung



Operations Research

Optimierung für Wirtschaftsabläufe.

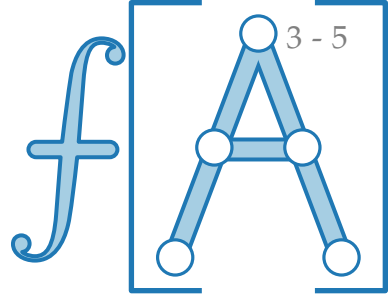
- Standortplanung
- Ablaufplanung
- Flottenmanagement



Operations Research

Optimierung für Wirtschaftsabläufe.

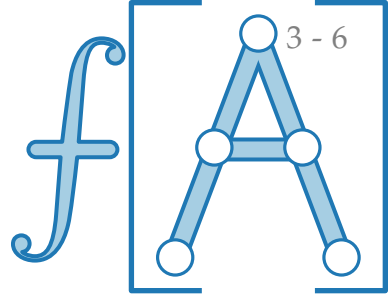
- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme



Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

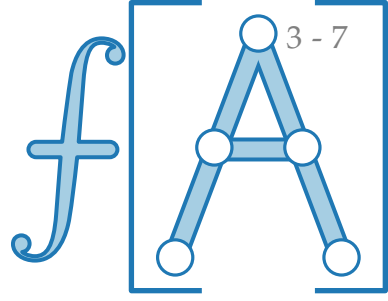


Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.



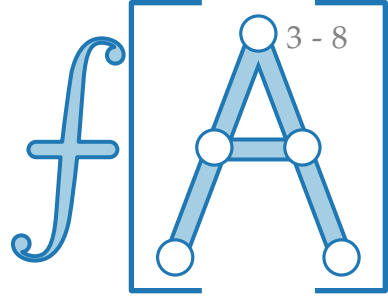
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik



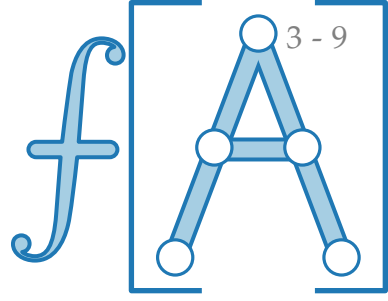
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen



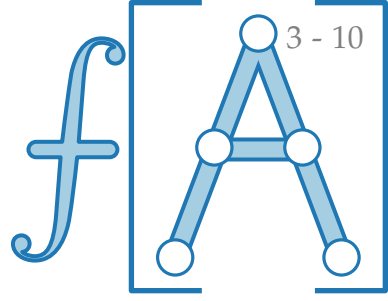
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen, Wahrscheinlichkeitstheorie



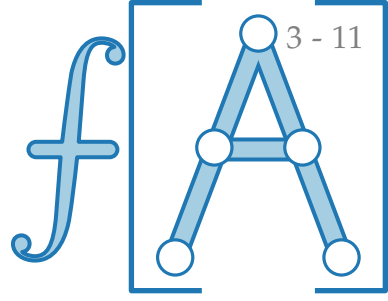
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen, Wahrscheinlichkeitstheorie, Spieltheorie



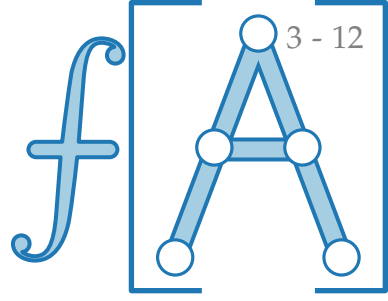
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen, Wahrscheinlichkeitstheorie, Spieltheorie,
Graphentheorie



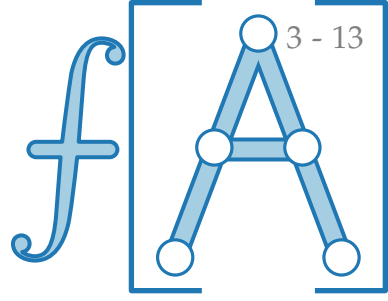
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen, Wahrscheinlichkeitstheorie, Spieltheorie, Graphentheorie, mathematische Programmierung



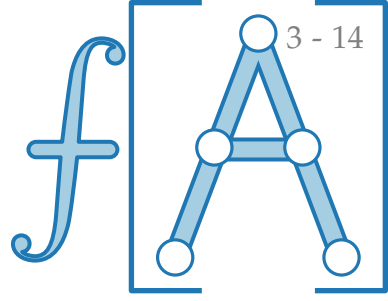
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen, Wahrscheinlichkeitstheorie, Spieltheorie, Graphentheorie, mathematische Programmierung, Simulation...



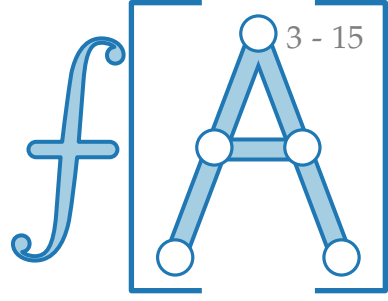
Operations Research

Optimierung für Wirtschaftsabläufe.

- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen, Wahrscheinlichkeitstheorie, Spieltheorie, Graphentheorie, mathematische Programmierung, Simulation...



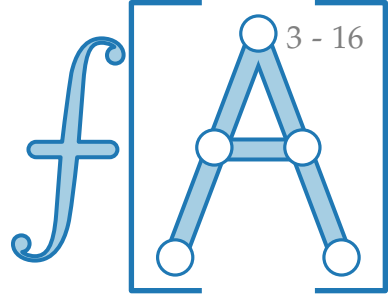
Operations Research

Optimierung für Wirtschaftsabläufe.

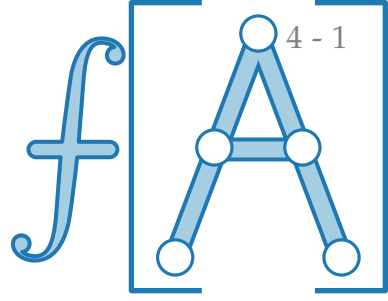
- Standortplanung
- Ablaufplanung
- Flottenmanagement
- Pack- und Zuschnittprobleme
- ...

Werkzeuge.

Statistik, Algorithmen, Wahrscheinlichkeitstheorie, Spieltheorie, Graphentheorie, mathematische Programmierung, Simulation...

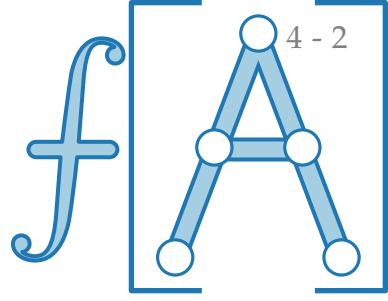


Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

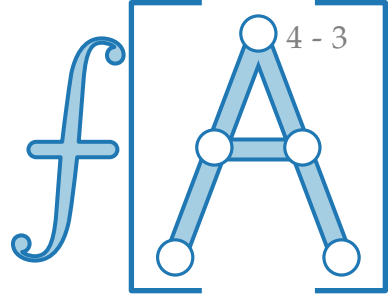
Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

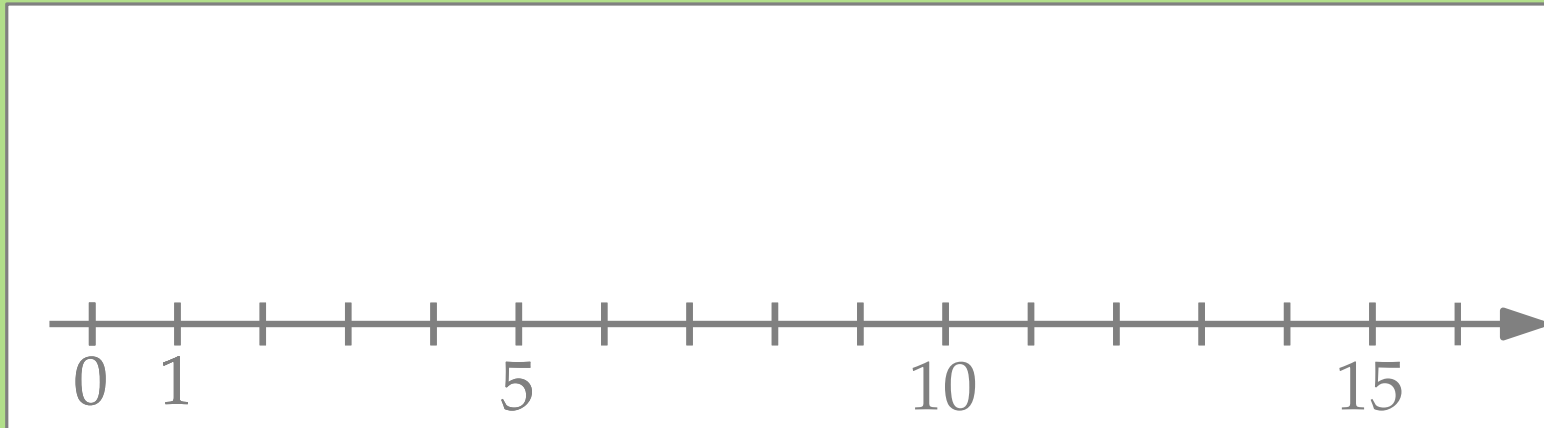
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

Ein einfaches Problem der Ablaufplanung

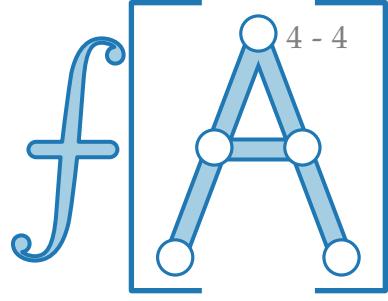


Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

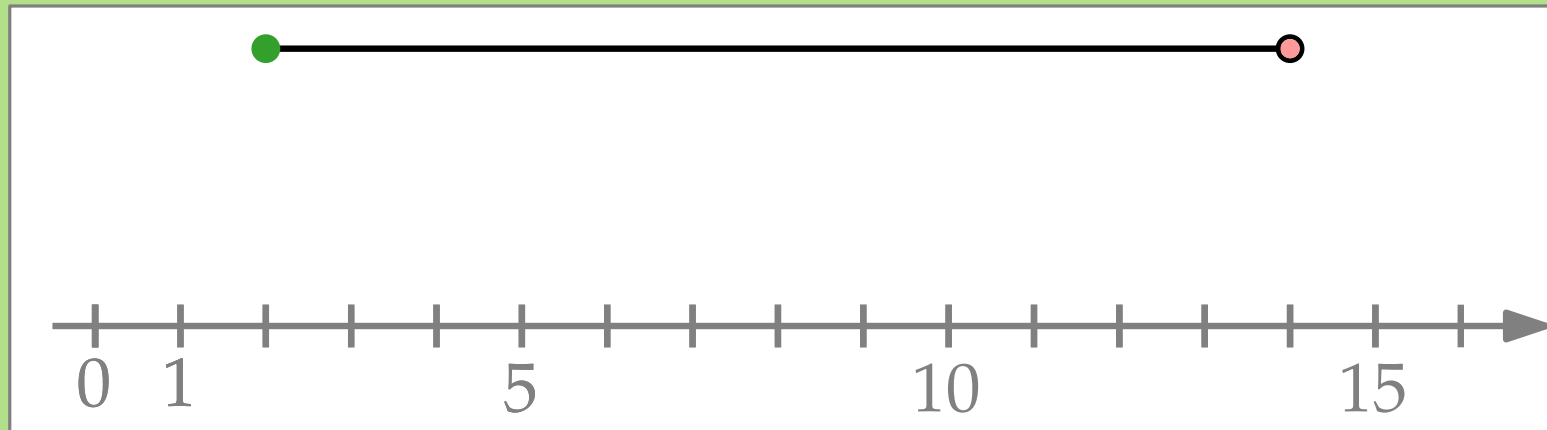


Ein einfaches Problem der Ablaufplanung

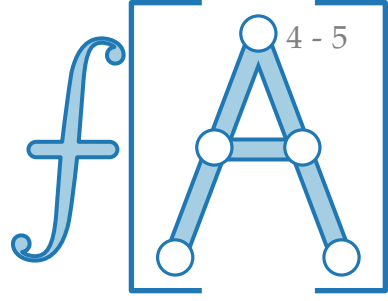


Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

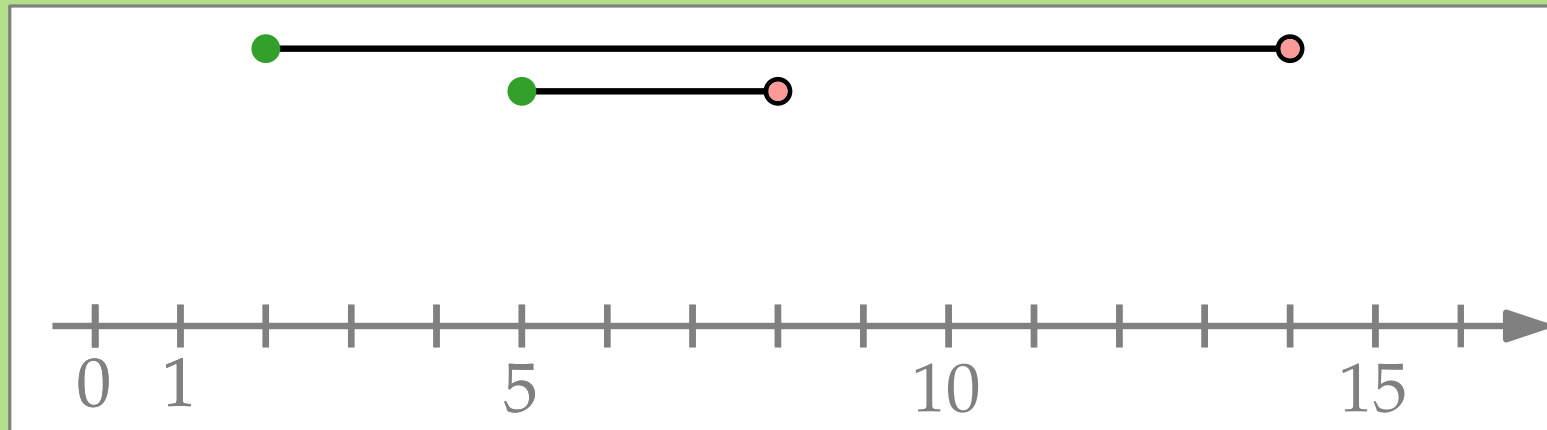


Ein einfaches Problem der Ablaufplanung

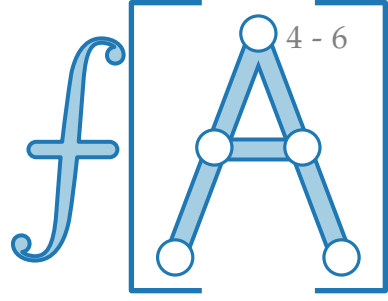


Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

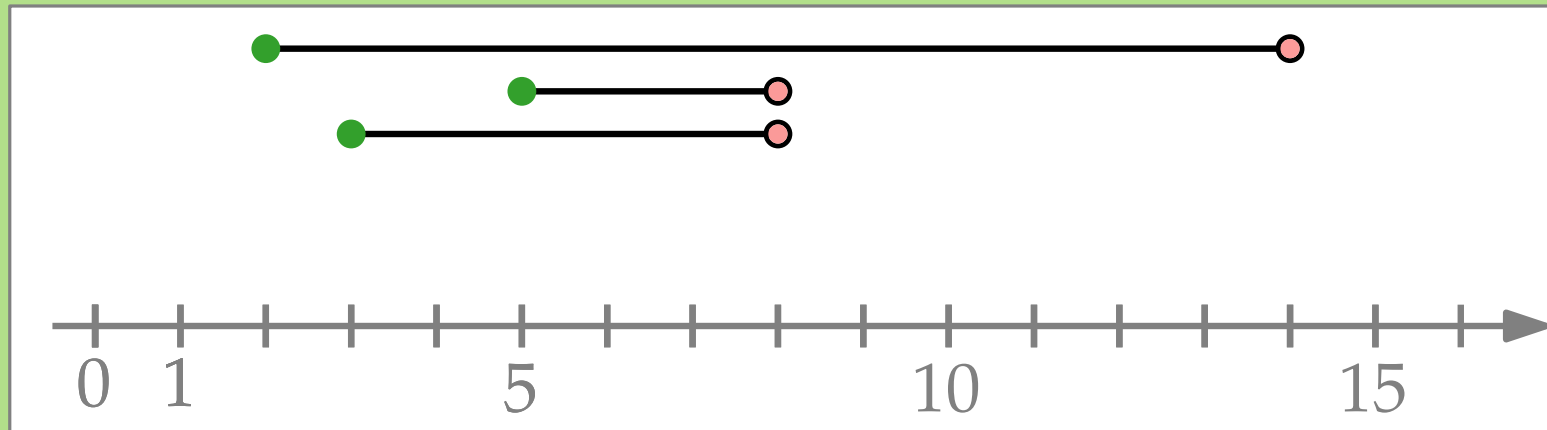


Ein einfaches Problem der Ablaufplanung

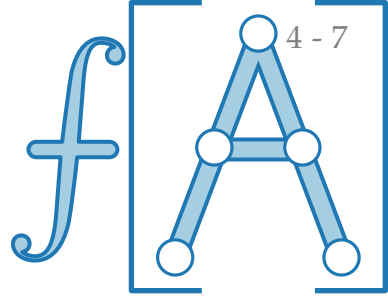


Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

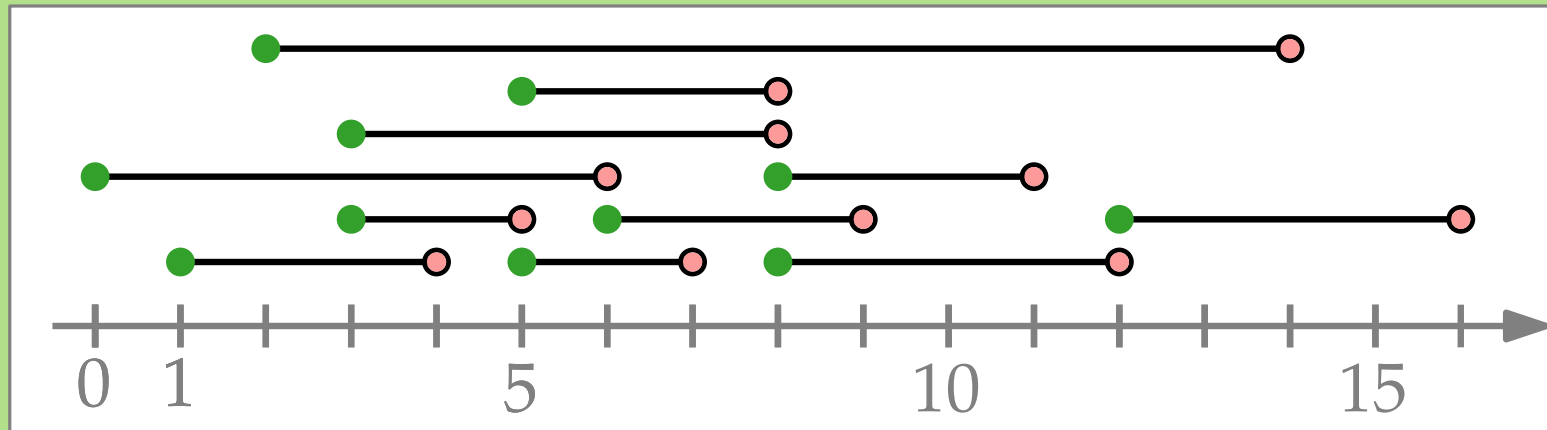


Ein einfaches Problem der Ablaufplanung

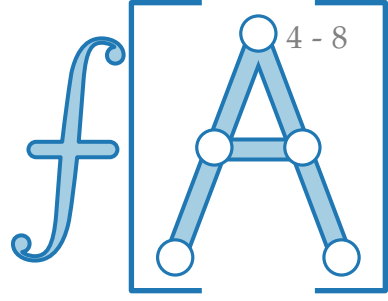


Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

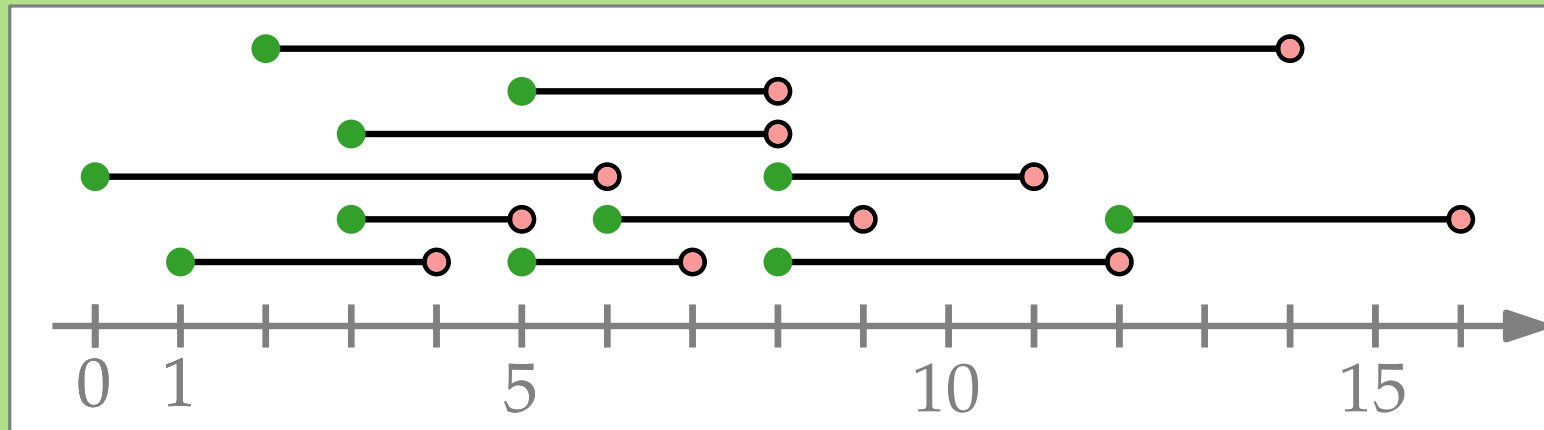


Ein einfaches Problem der Ablaufplanung



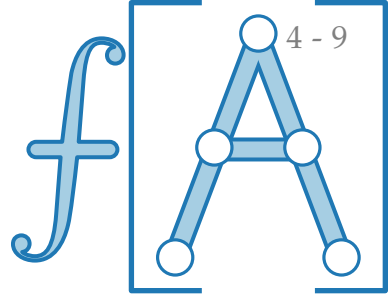
Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



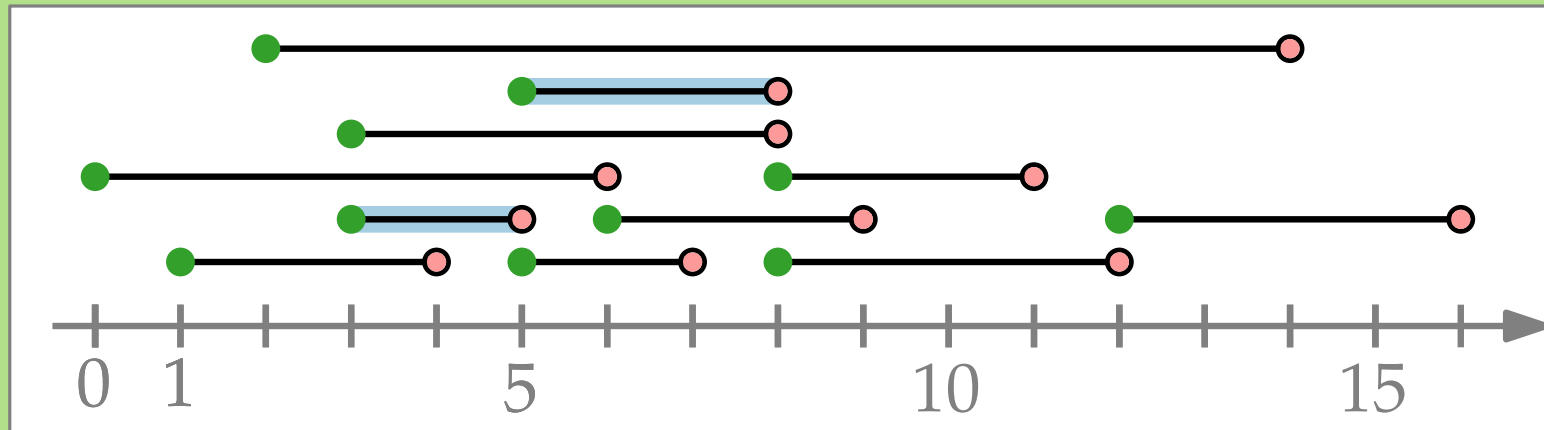
a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

Ein einfaches Problem der Ablaufplanung



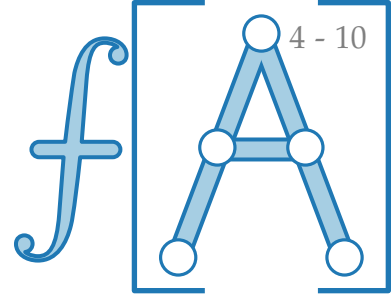
Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



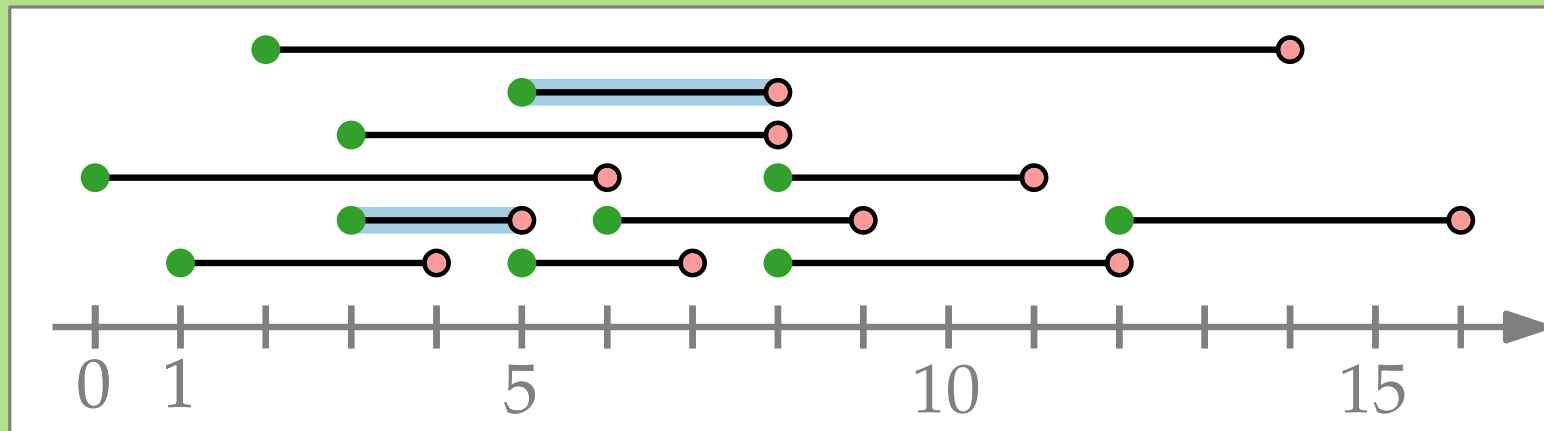
a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

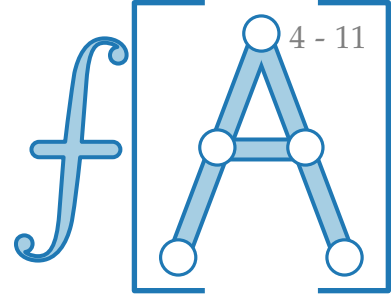
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

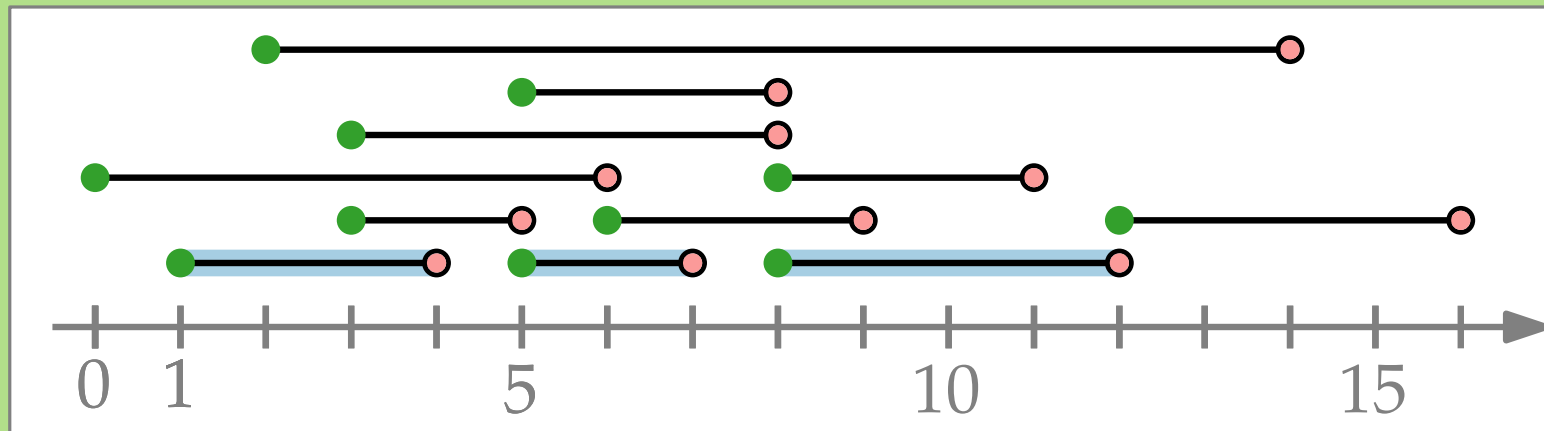
Die Aktivitäten in $A' \subset A$ sind **paarweise kompatibel**, wenn für jedes Paar $a_i, a_j \in A'$ gilt, dass a_i und a_j kompatibel sind.

Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

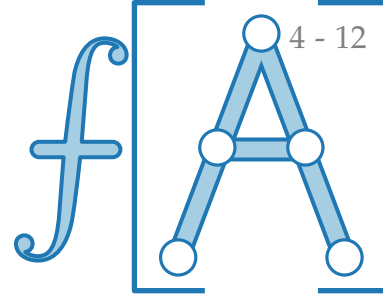
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

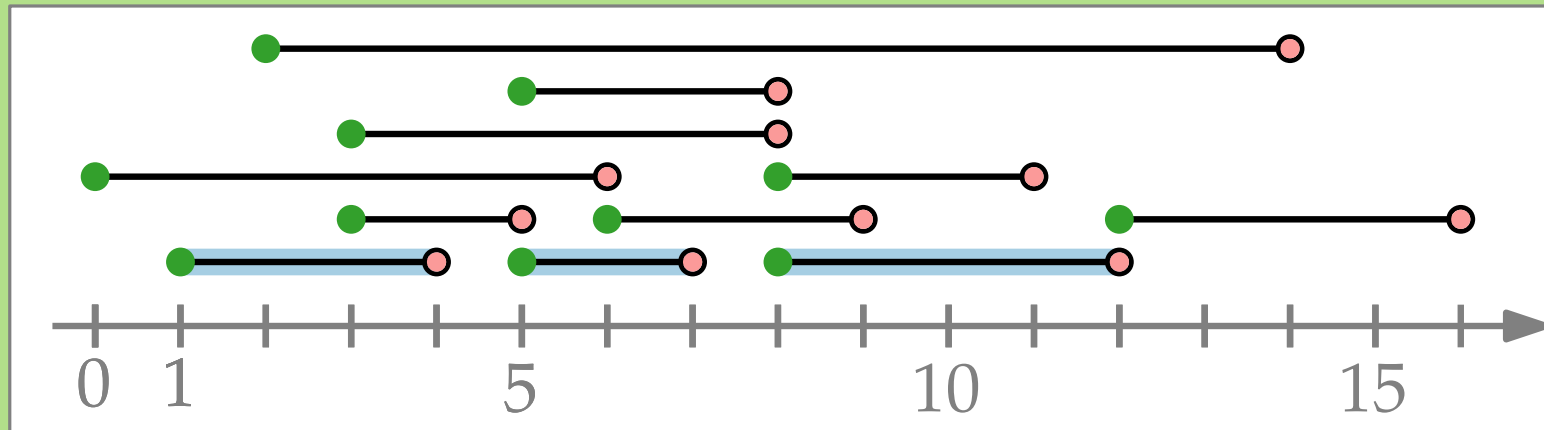
Die Aktivitäten in $A' \subset A$ sind **paarweise kompatibel**, wenn für jedes Paar $a_i, a_j \in A'$ gilt, dass a_i und a_j kompatibel sind.

Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

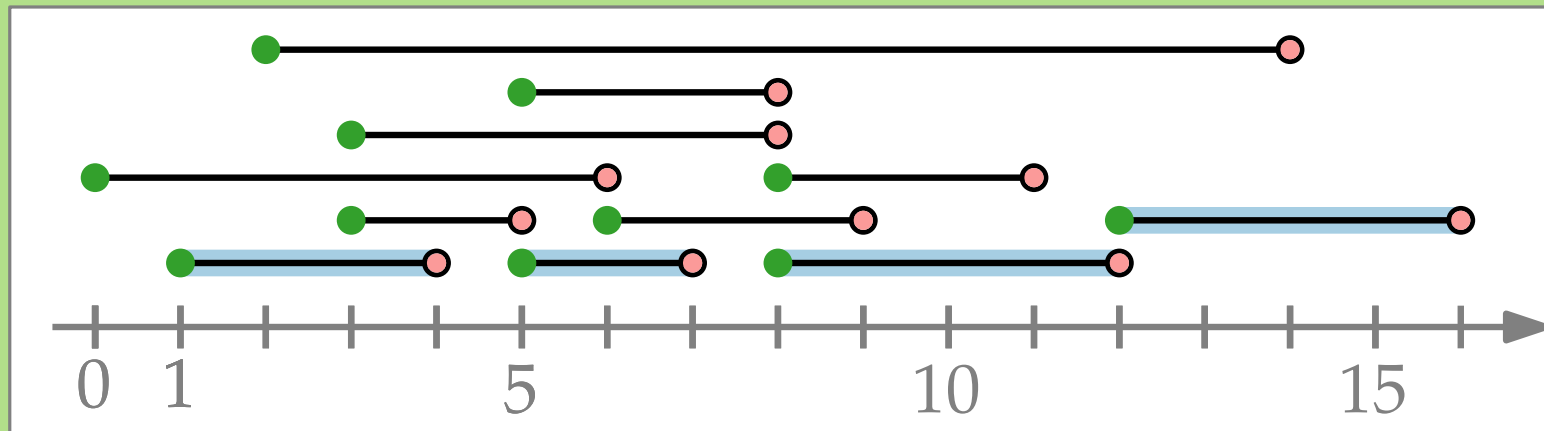
Die Aktivitäten in $A' \subset A$ sind **paarweise kompatibel**, wenn für jedes Paar $a_i, a_j \in A'$ gilt, dass a_i und a_j kompatibel sind.

Ges. eine größtmögliche Menge paarweise kompatibler Aktivitäten.

Ein einfaches Problem der Ablaufplanung

Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

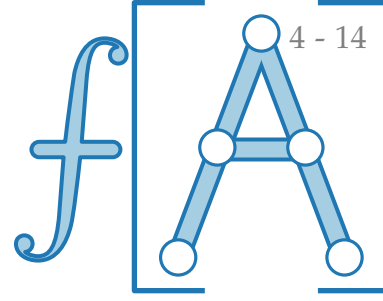


a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

Die Aktivitäten in $A' \subset A$ sind **paarweise kompatibel**, wenn für jedes Paar $a_i, a_j \in A'$ gilt, dass a_i und a_j kompatibel sind.

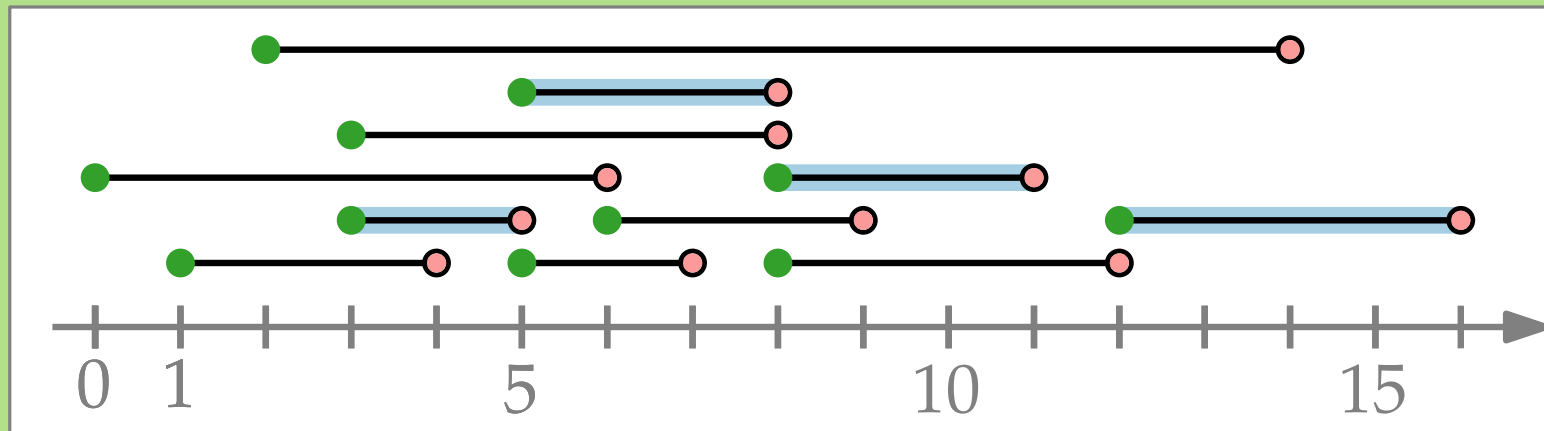
Ges. eine größtmögliche Menge paarweise kompatibler Aktivitäten.

Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

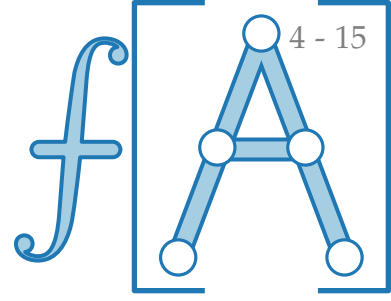


a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

Die Aktivitäten in $A' \subset A$ sind **paarweise kompatibel**, wenn für jedes Paar $a_i, a_j \in A'$ gilt, dass a_i und a_j kompatibel sind.

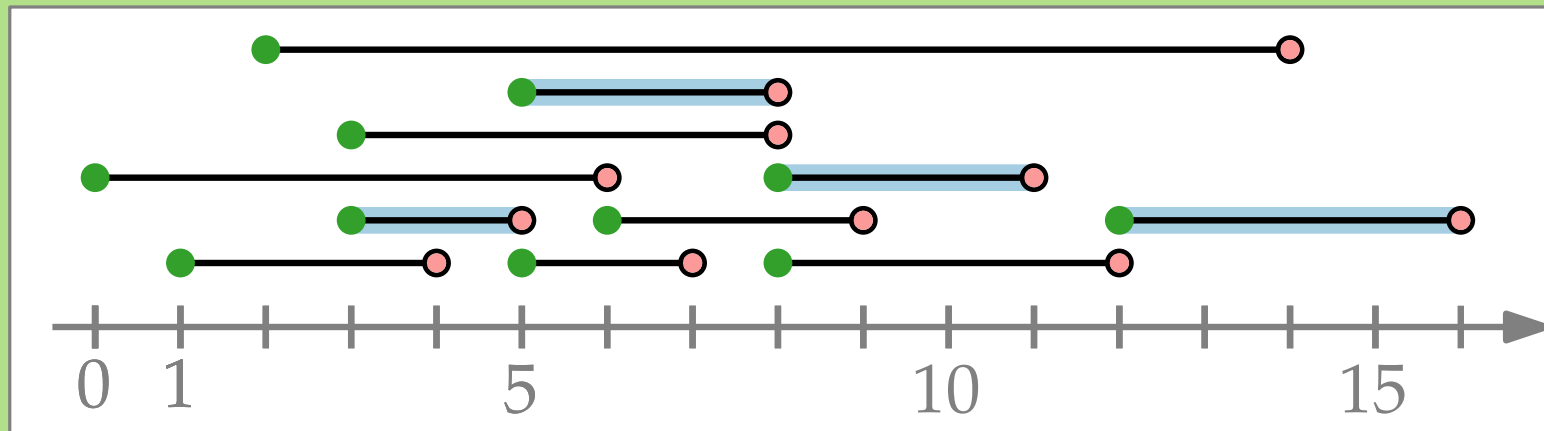
Ges. eine größtmögliche Menge paarweise kompatibler Aktivitäten.

Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



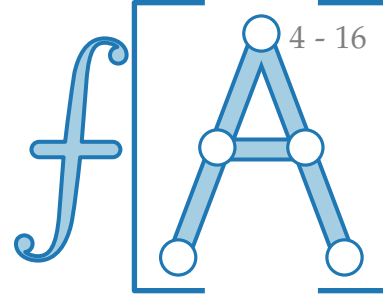
a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

Die Aktivitäten in $A' \subset A$ sind **paarweise kompatibel**, wenn für jedes Paar $a_i, a_j \in A'$ gilt, dass a_i und a_j kompatibel sind.

Ges. eine größtmögliche Menge paarweise kompatibler Aktivitäten.

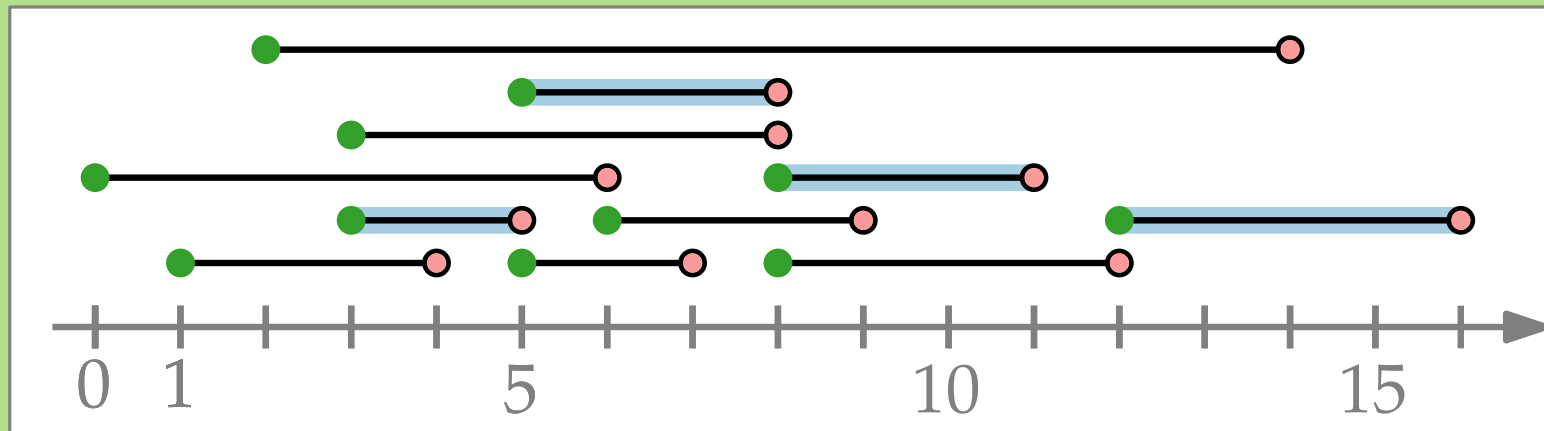
Grund.

Ein einfaches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **Aktivitäten**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



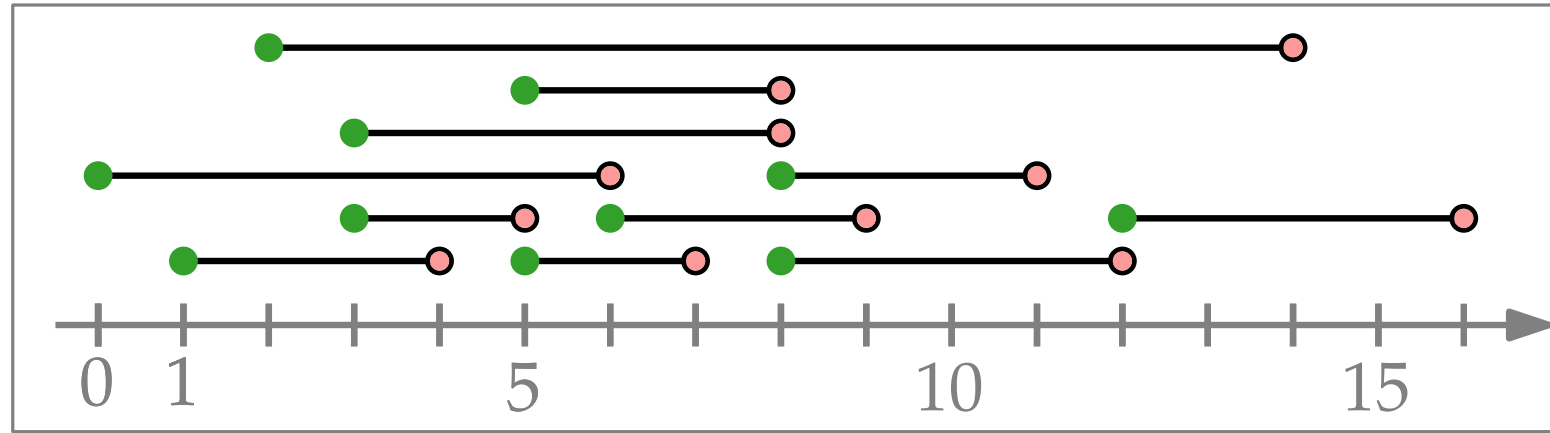
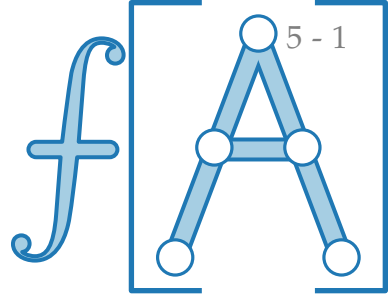
a_i und a_j sind **kompatibel**, wenn $a_i \cap a_j = \emptyset$.

Die Aktivitäten in $A' \subset A$ sind **paarweise kompatibel**, wenn für jedes Paar $a_i, a_j \in A'$ gilt, dass a_i und a_j kompatibel sind.

Ges. eine größtmögliche Menge paarweise kompatibler Aktivitäten.

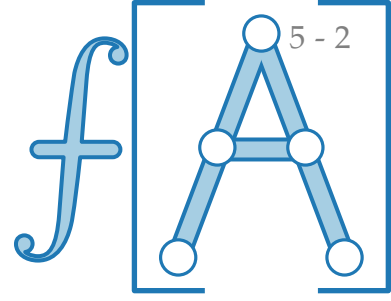
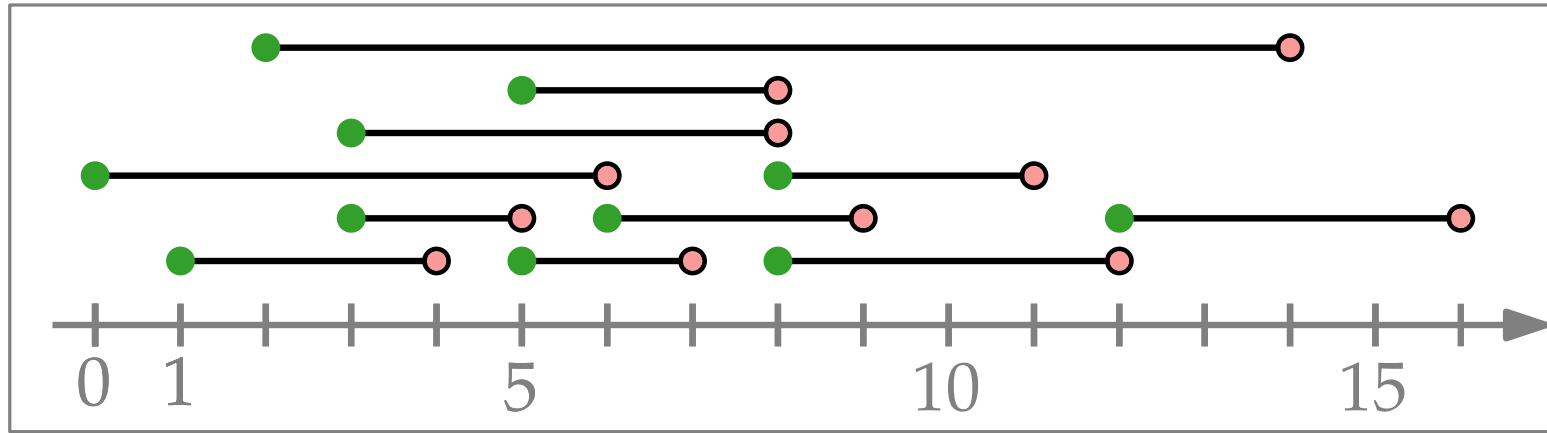
Grund. Aktivitäten (je 1€), die gleiche Ressource benutzen

Ein kleiner technischer Trick



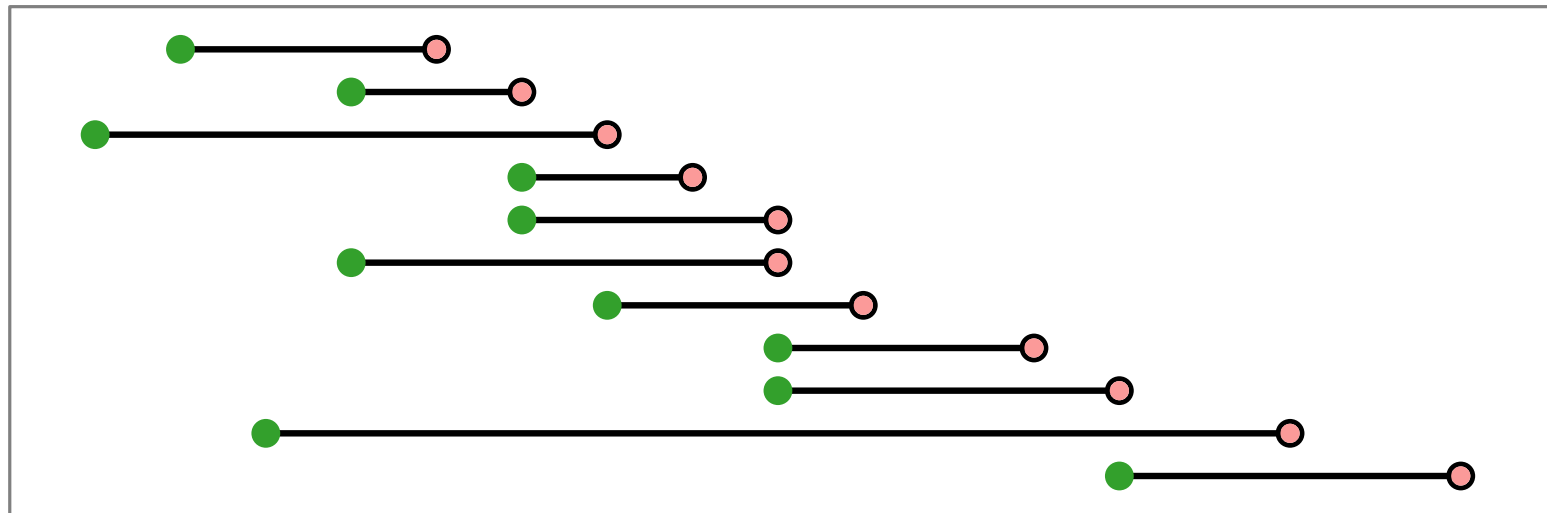
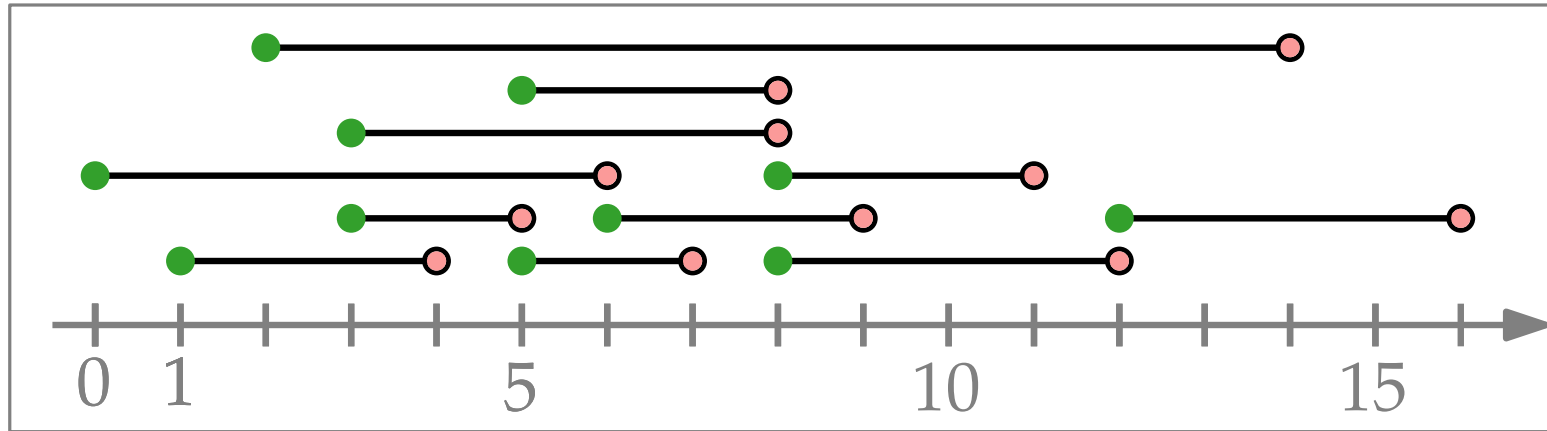
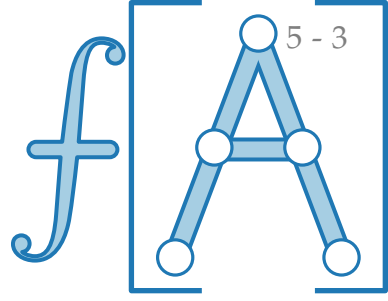
Ein kleiner technischer Trick

Wir nummerieren (für den Rest der Vorlesung) die Aktivitäten so, dass für die Endtermine gilt $e_1 \leq e_2 \leq \dots \leq e_n$.



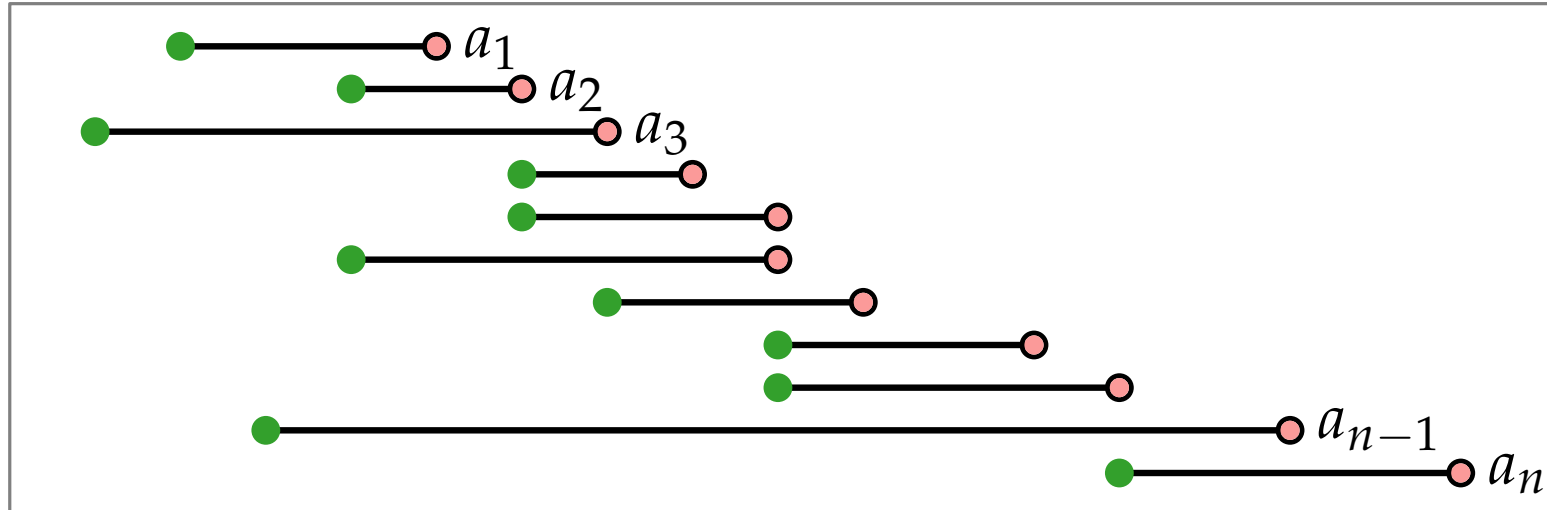
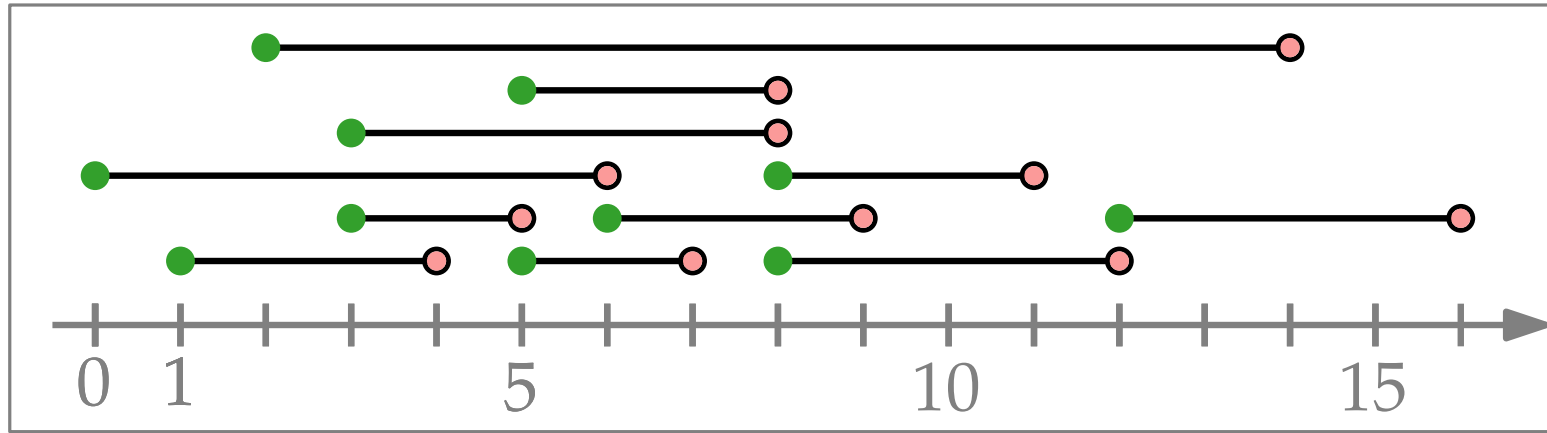
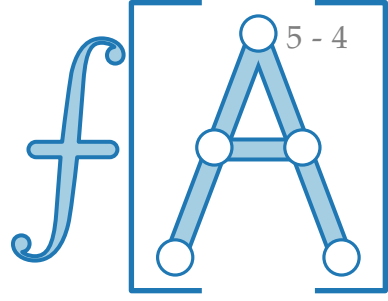
Ein kleiner technischer Trick

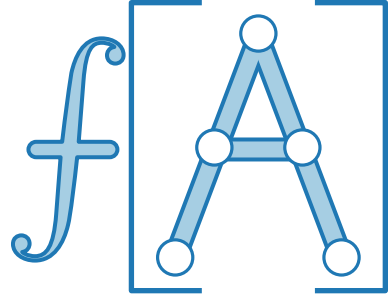
Wir nummerieren (für den Rest der Vorlesung) die Aktivitäten so, dass für die Endtermine gilt $e_1 \leq e_2 \leq \dots \leq e_n$.



Ein kleiner technischer Trick

Wir nummerieren (für den Rest der Vorlesung) die Aktivitäten so, dass für die Endtermine gilt $e_1 \leq e_2 \leq \dots \leq e_n$.

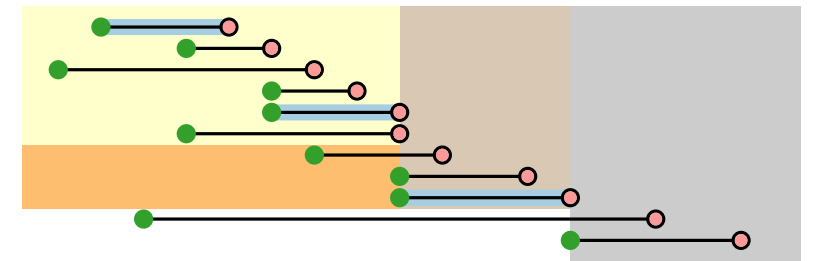
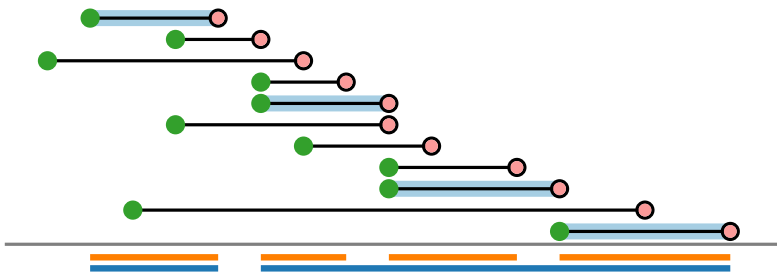




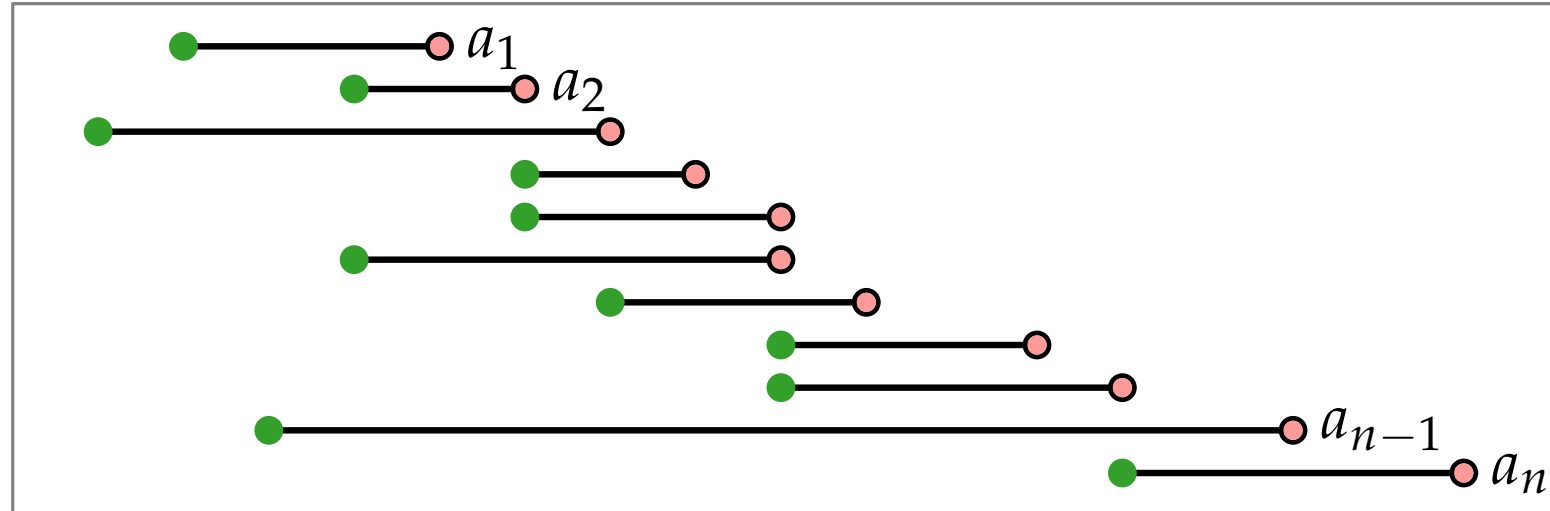
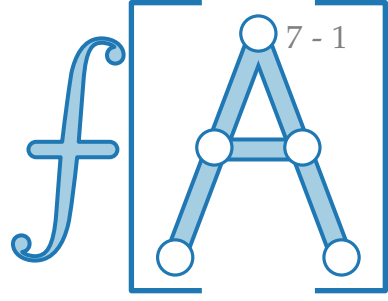
Fortgeschrittene Algorithmen

1. Vorlesung Greedy Algorithmen

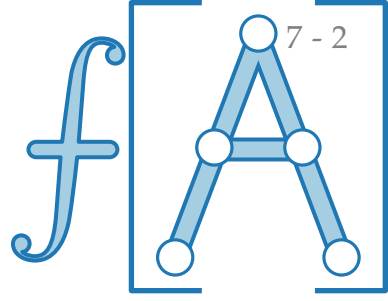
Teil II: Greedy Algorithmus



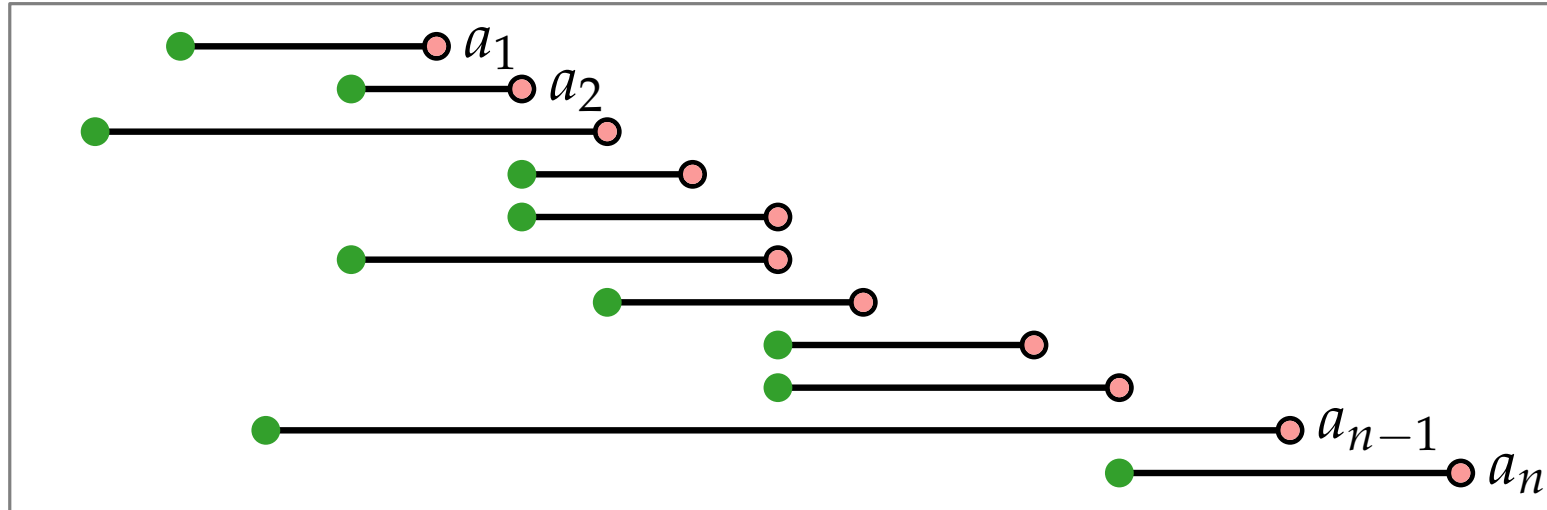
Charakterisierung optimaler Lösungen



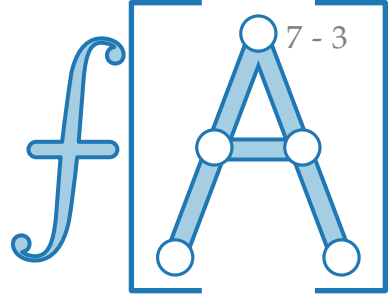
Charakterisierung optimaler Lösungen



Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

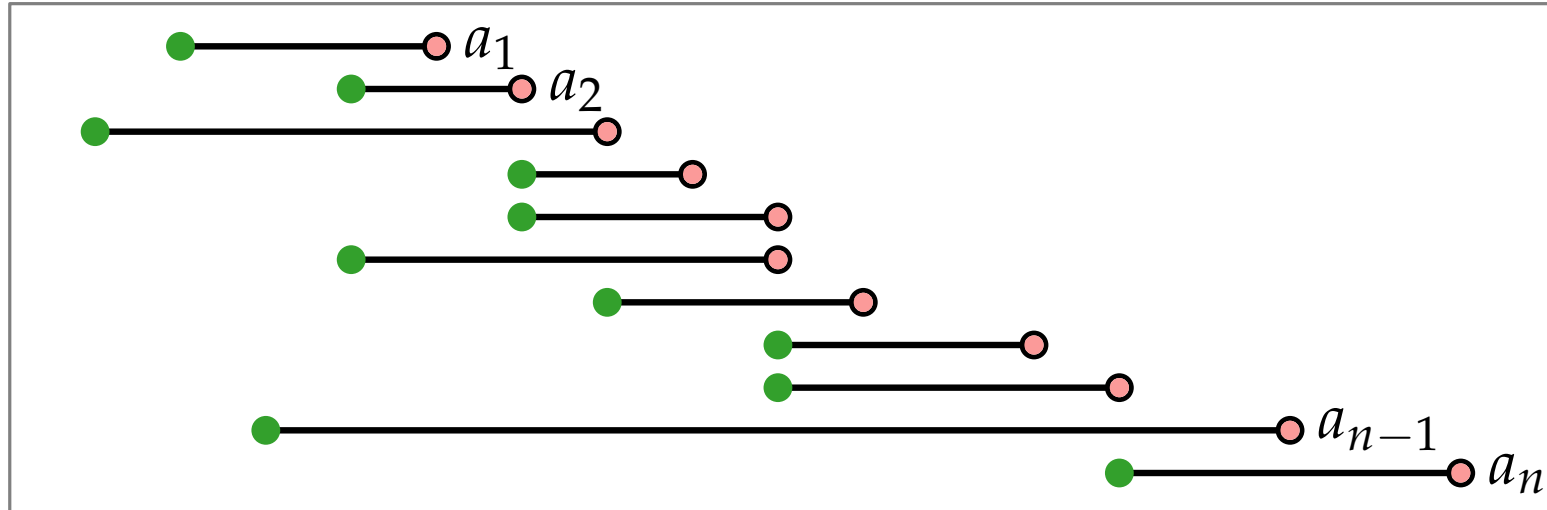


Charakterisierung optimaler Lösungen

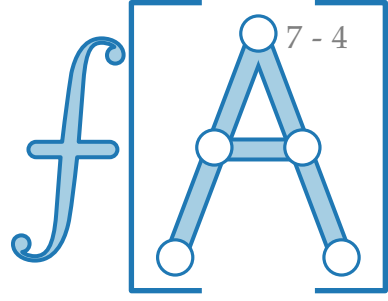


Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit

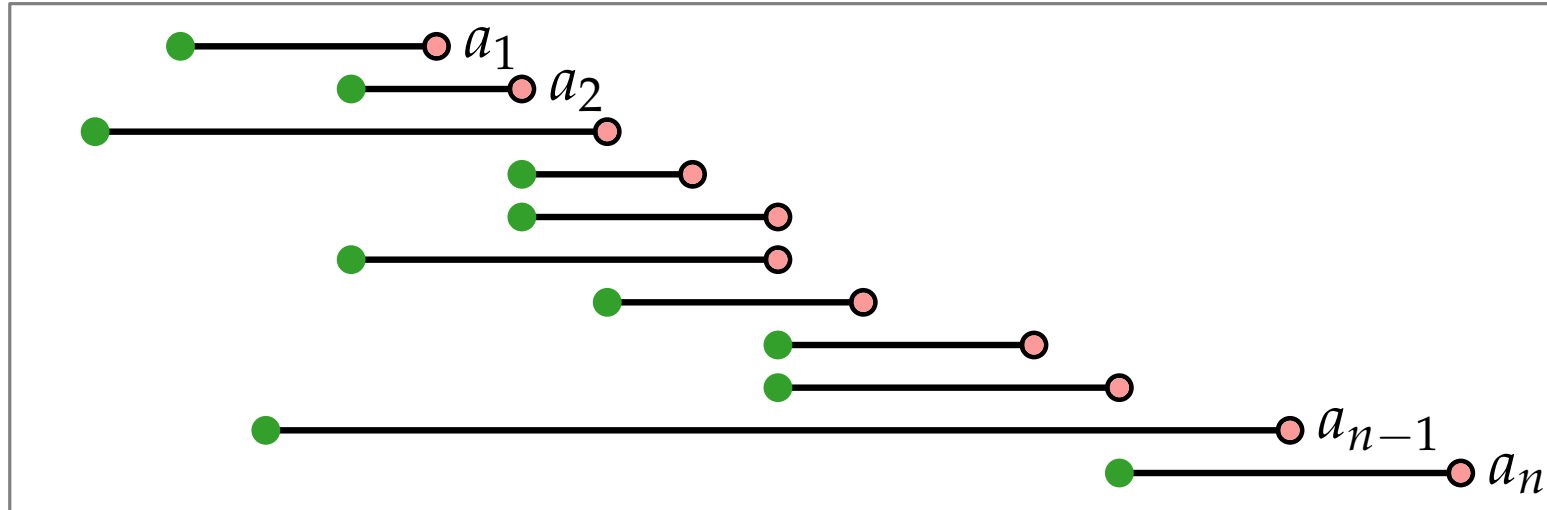


Charakterisierung optimaler Lösungen

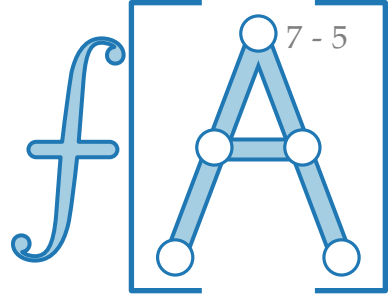


Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.



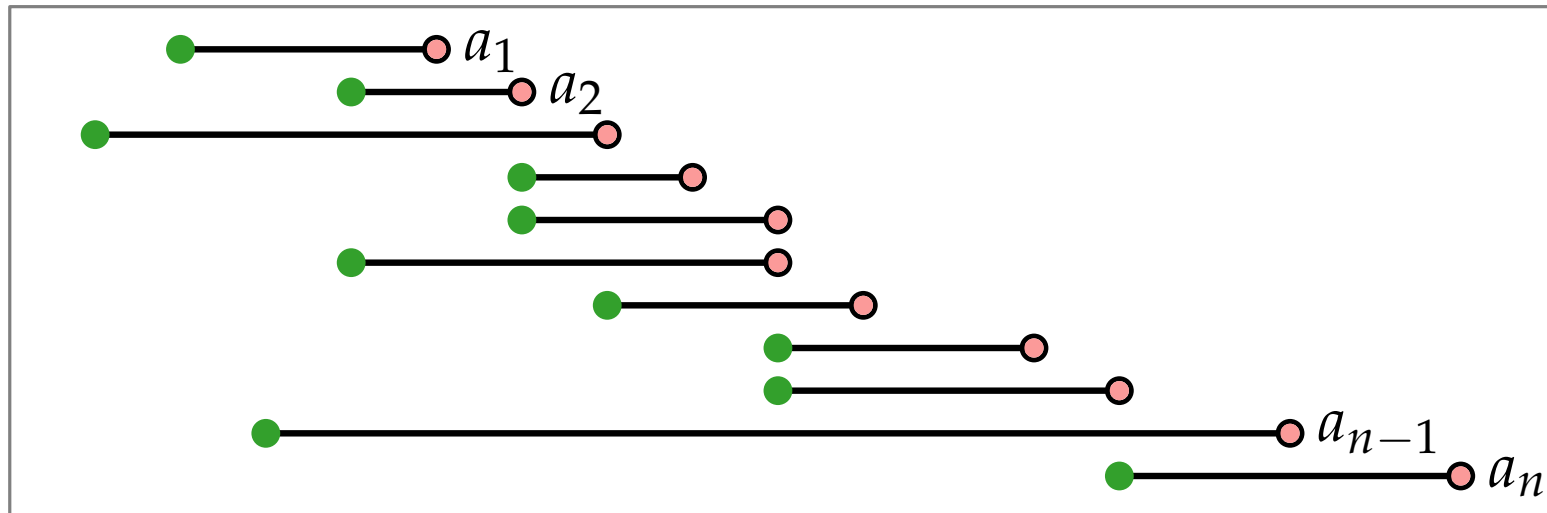
Charakterisierung optimaler Lösungen



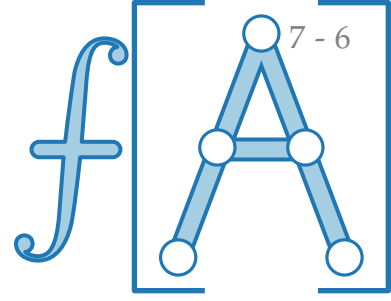
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.



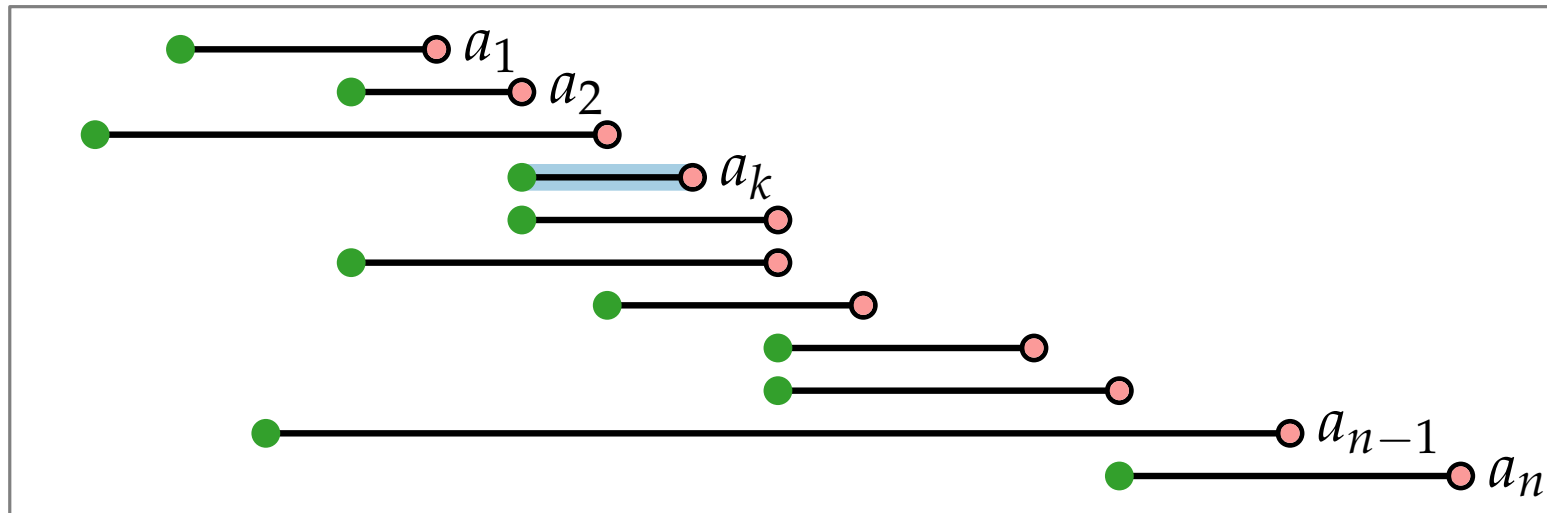
Charakterisierung optimaler Lösungen



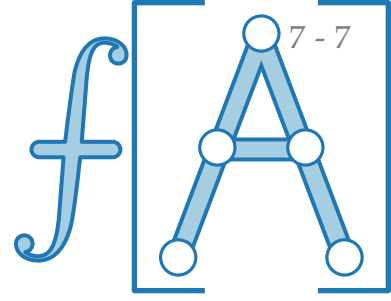
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.



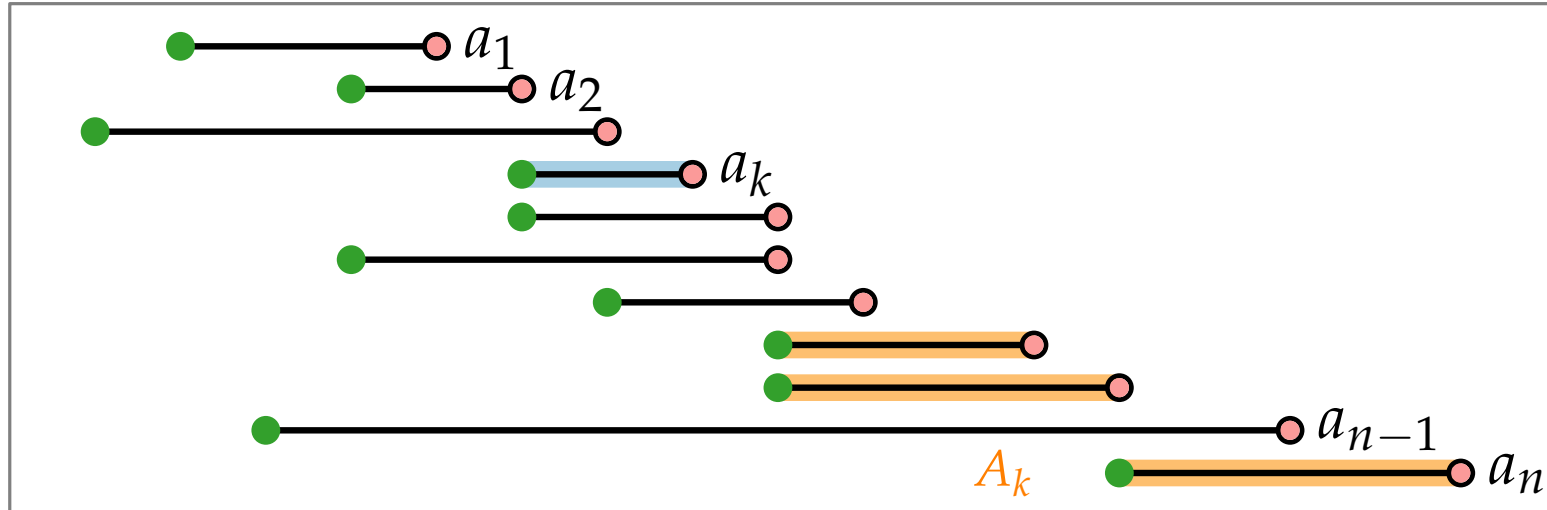
Charakterisierung optimaler Lösungen



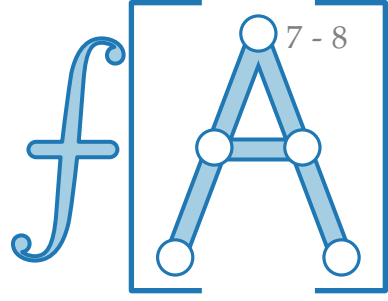
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.



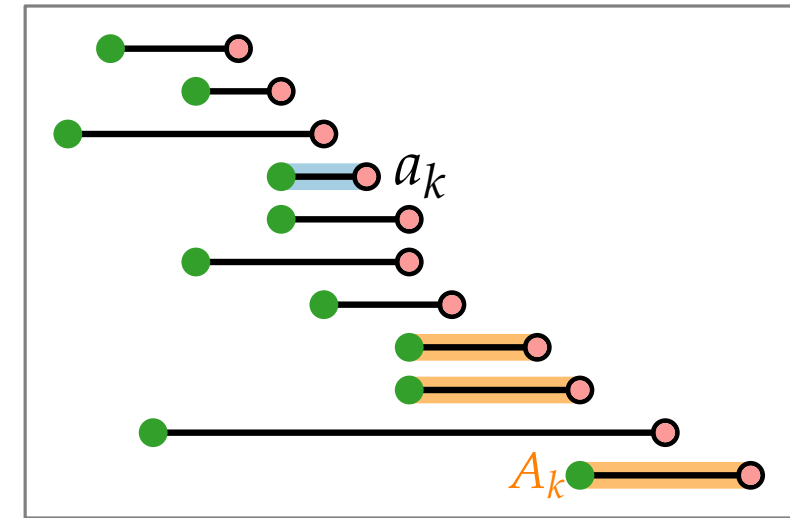
Charakterisierung optimaler Lösungen



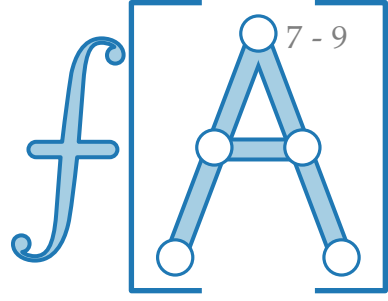
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.



Charakterisierung optimaler Lösungen

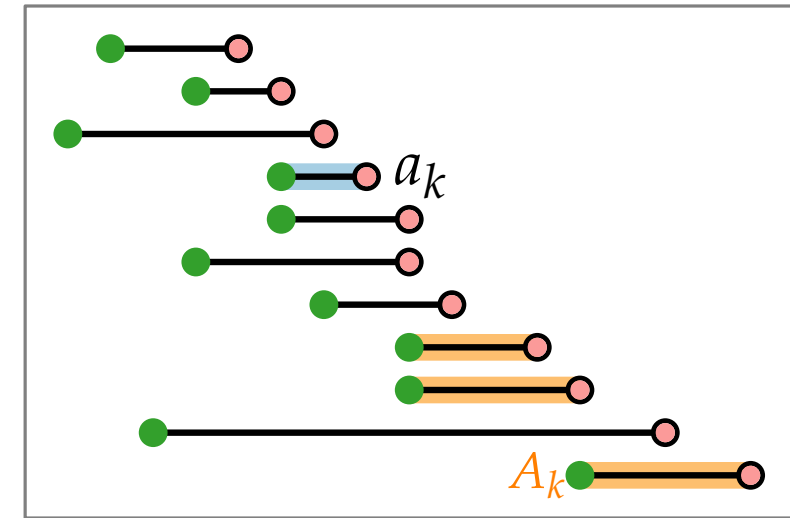


Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

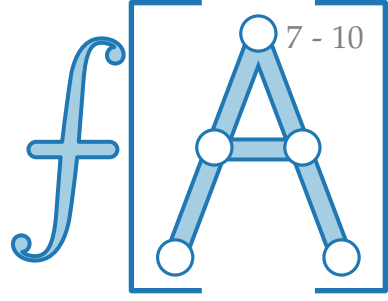
Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

Sei L_k eine optimale Lösung von A_k .



Charakterisierung optimaler Lösungen



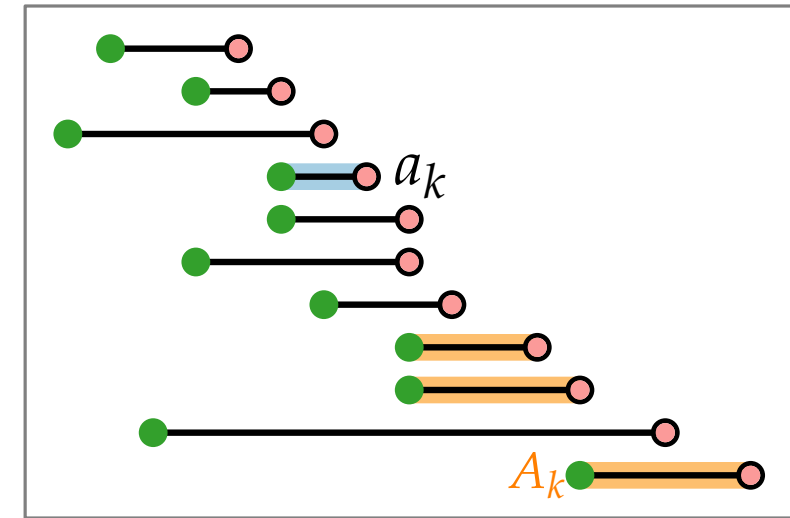
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

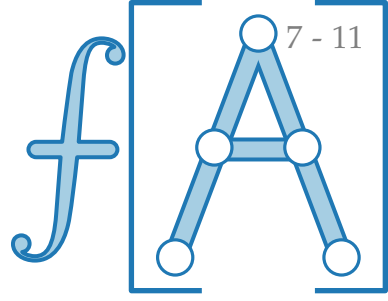
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

Sei L_k eine optimale Lösung von A_k .

Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Charakterisierung optimaler Lösungen



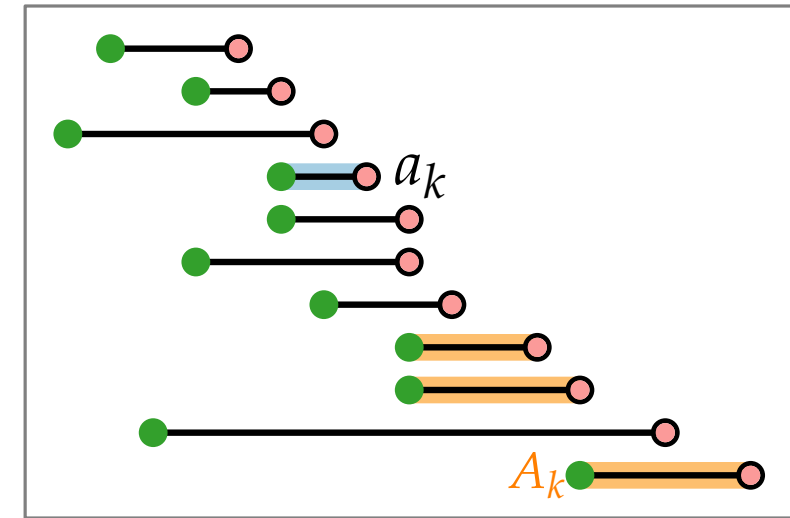
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

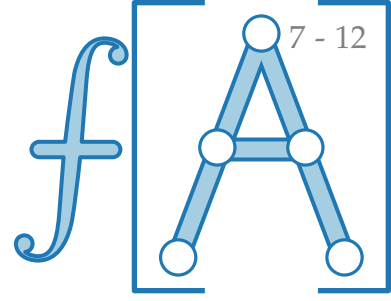
Sei L_k eine optimale Lösung von A_k .

Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Satz. Sei $A_k \neq \emptyset$.

Charakterisierung optimaler Lösungen



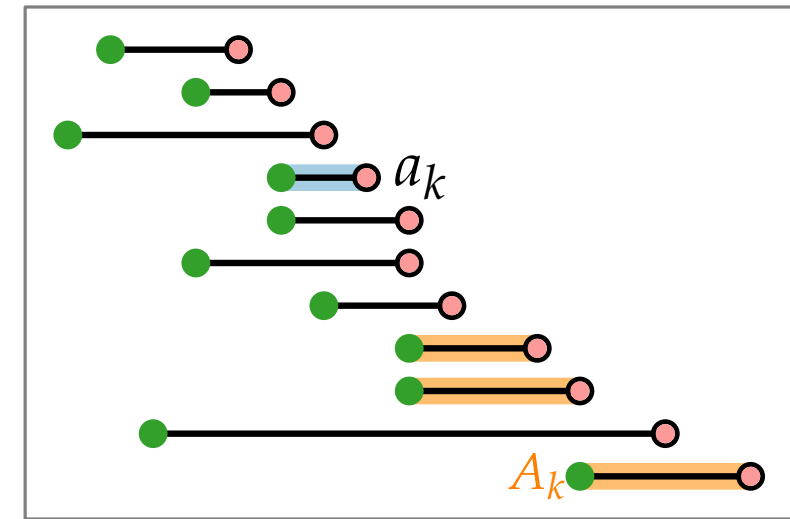
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

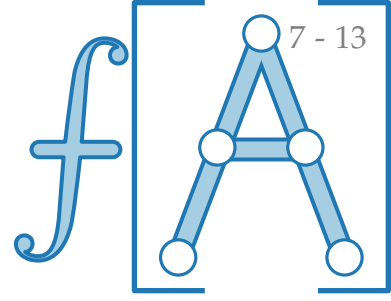
Sei L_k eine optimale Lösung von A_k .

Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Satz. Sei $A_k \neq \emptyset$.
Sei a_m Aktivität mit frühester Endzeit in A_k .

Charakterisierung optimaler Lösungen



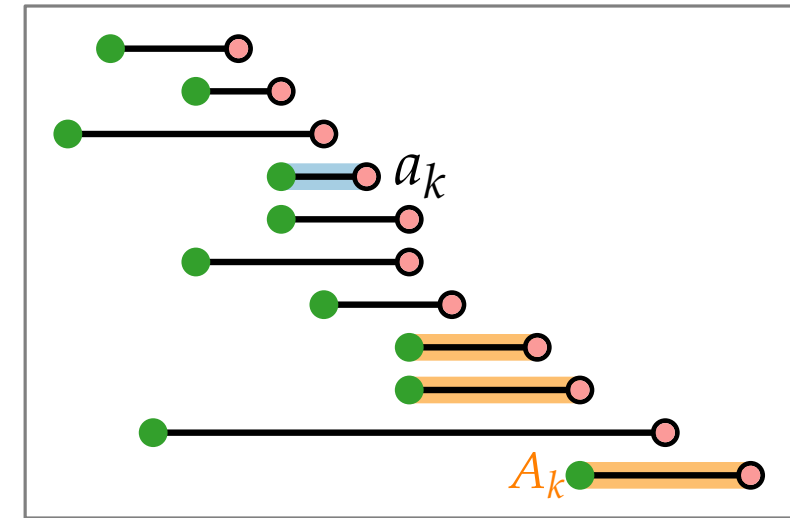
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

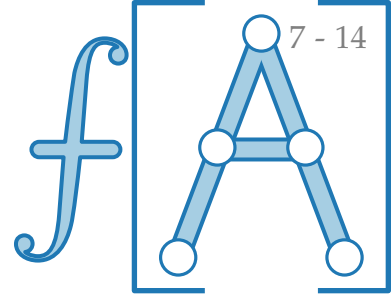
Sei L_k eine optimale Lösung von A_k .

Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Satz. Sei $A_k \neq \emptyset$.
Sei a_m Aktivität mit frühester Endzeit in A_k .
 $\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Charakterisierung optimaler Lösungen



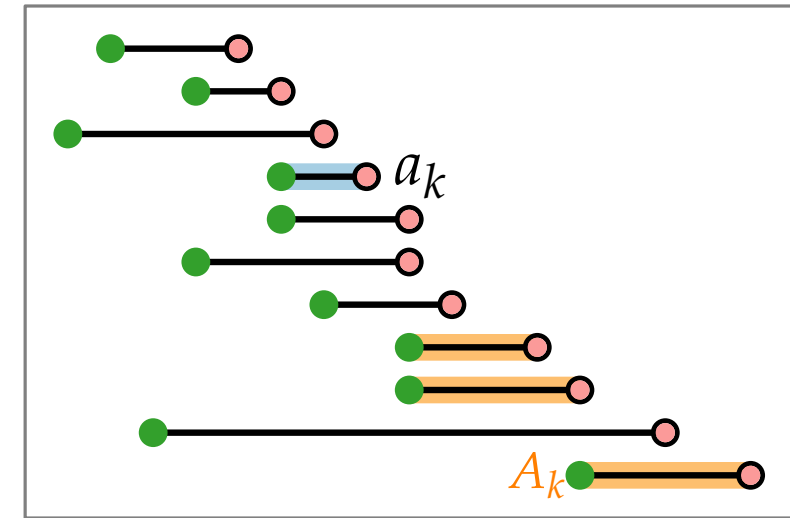
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

Sei L_k eine optimale Lösung von A_k .

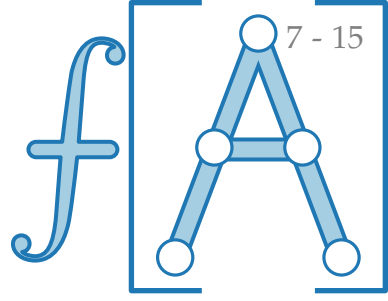
Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Satz. Sei $A_k \neq \emptyset$.
Sei a_m Aktivität mit frühester Endzeit in A_k .
 $\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Beweis.

Charakterisierung optimaler Lösungen



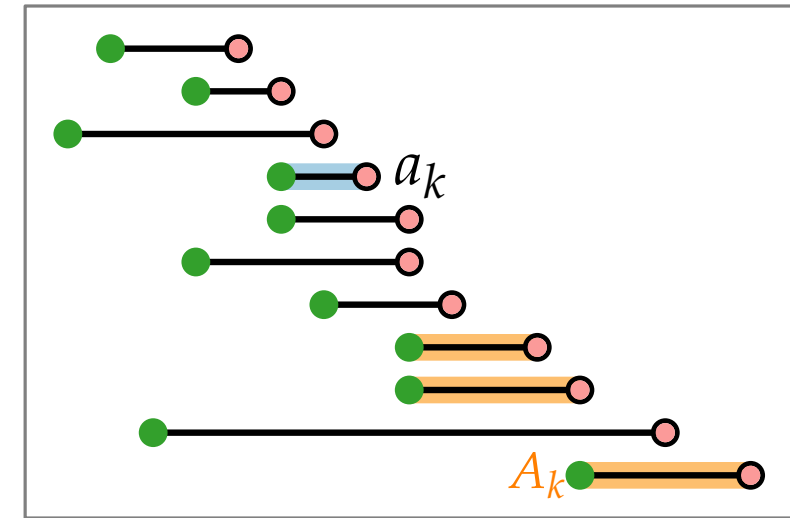
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

Sei L_k eine optimale Lösung von A_k .

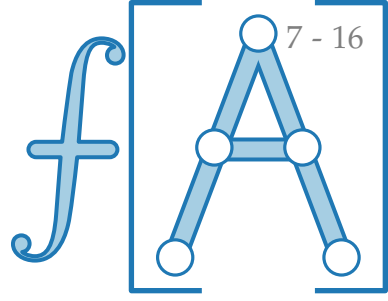
Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Satz. Sei $A_k \neq \emptyset$.
Sei a_m Aktivität mit frühester Endzeit in A_k .
 $\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Beweis. Austauschargument!

Charakterisierung optimaler Lösungen



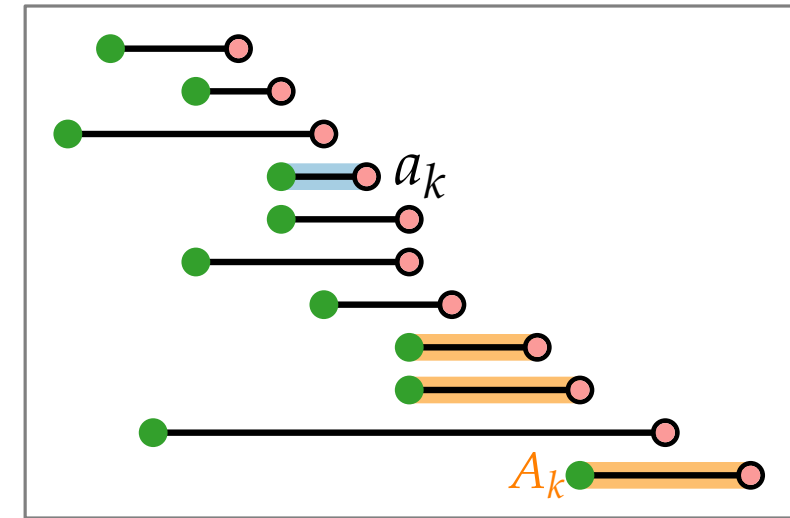
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

Sei L_k eine optimale Lösung von A_k .

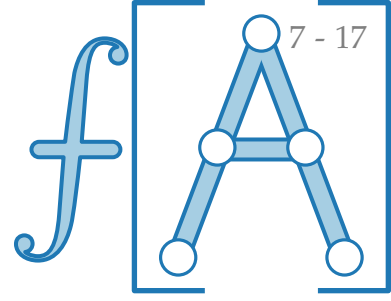
Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Satz. Sei $A_k \neq \emptyset$.
Sei a_m Aktivität mit frühester Endzeit in A_k .
 $\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Beweis. Austauschargument! Tausche Akt. von L_k mit frühester Endzeit mit a_m .

Charakterisierung optimaler Lösungen



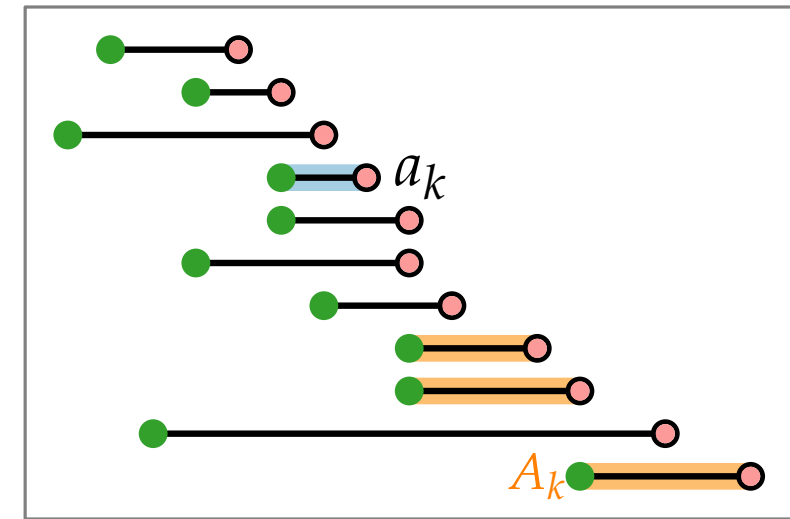
Idee. Sei L optimale Lösung für A . – Welche Aktivität hat gute Chancen, die erste („linkeste“) in L zu sein?

Intuition. Die Aktivität a_1 mit frühester Endzeit – weil a_1 die gemeinsame Ressource am wenigsten einschränkt.

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen.

Sei L_k eine optimale Lösung von A_k .

Falls Intuition korrekt, dann ist $\{a_1\} \cup L_1$ optimal.



Satz. Sei $A_k \neq \emptyset$.

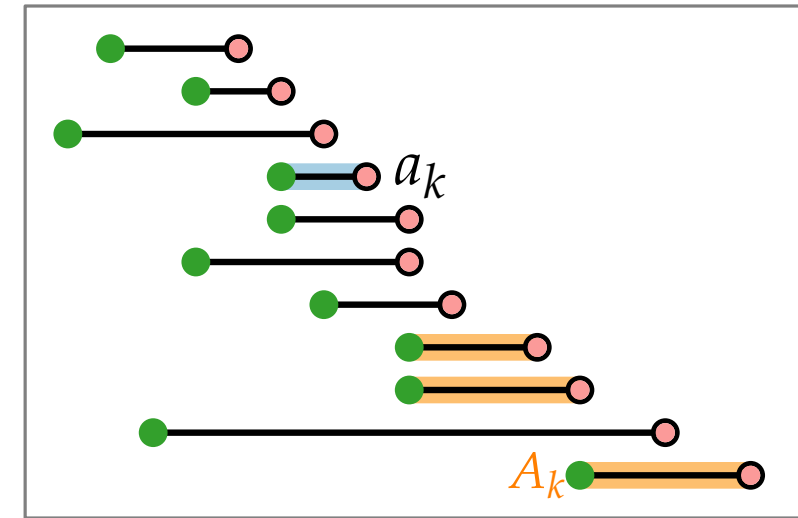
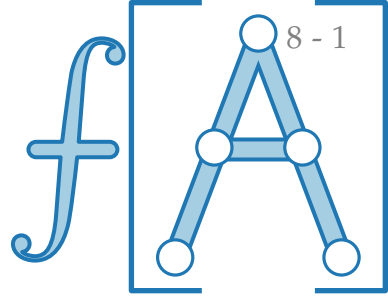
optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Beweis. Austauschargument! Tausche Akt. von L_k mit frühester Endzeit mit a_m .

Greedy – rekursiv



Satz.

Sei $A_k \neq \emptyset$.

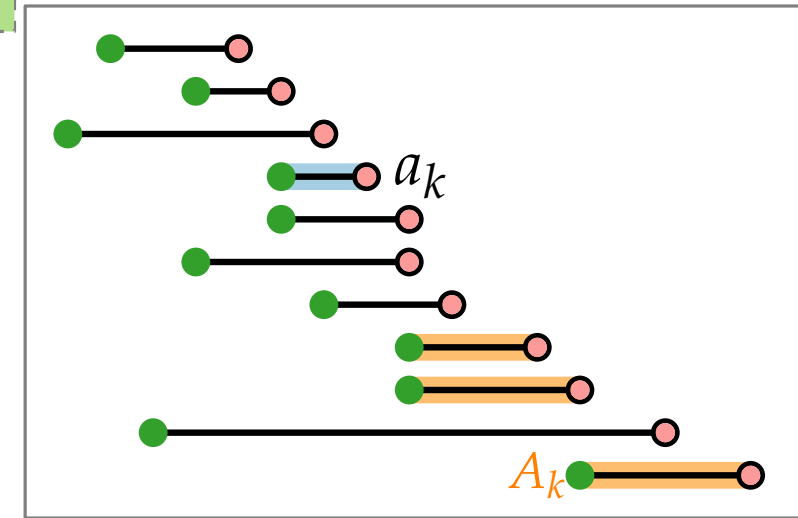
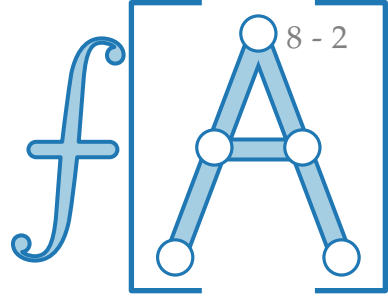
optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv

GREEDYRECURSIVE(int[] s , int[] e)



Satz.

Sei $A_k \neq \emptyset$.

optimale
Teilstruktur!

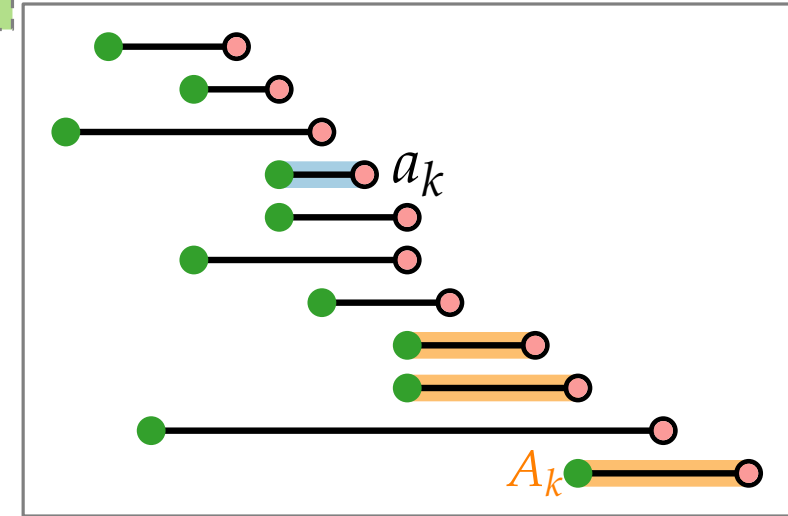
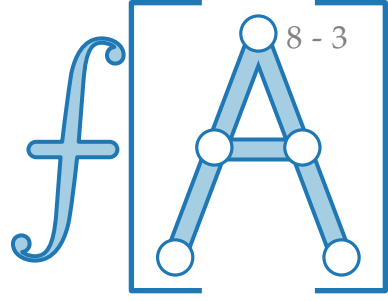
Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv

GREEDYRECURSIVE(int[] s , int[] e)

$$e_0 = -\infty$$



Satz.

Sei $A_k \neq \emptyset$.

optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

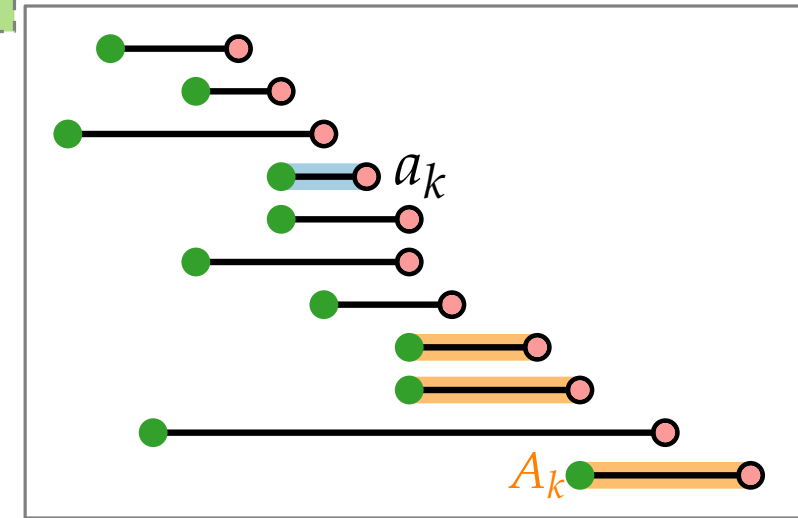
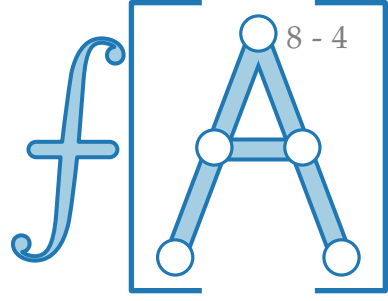
$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv

GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

// $\Rightarrow A_0 = A$



Satz.

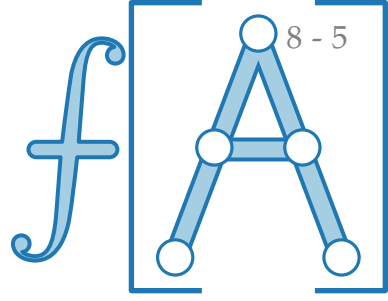
Sei $A_k \neq \emptyset$.

optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv

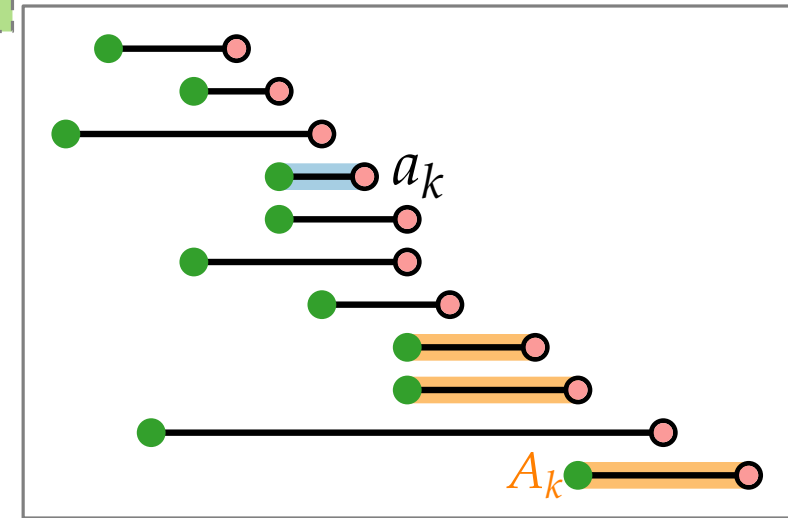


GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig



Satz.

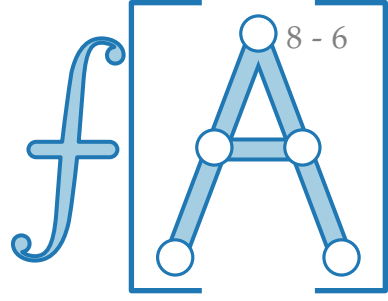
Sei $A_k \neq \emptyset$.

optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv



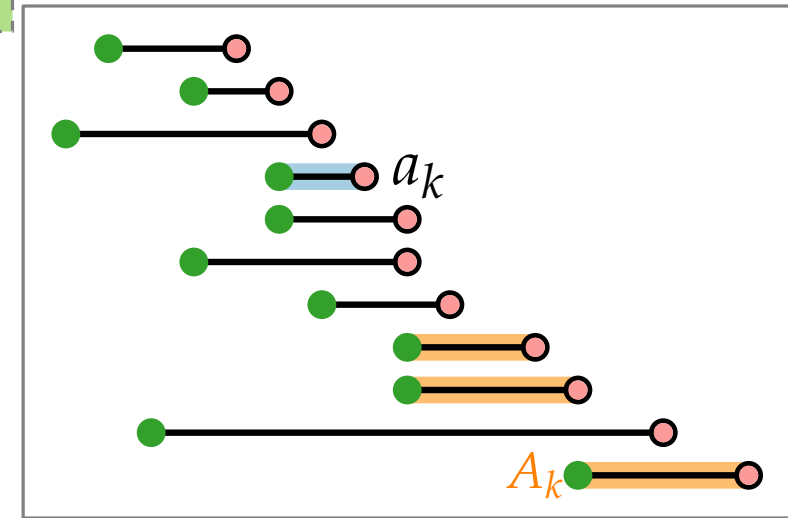
GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

return GREEDYRECURSIVEMAIN($s, e, 0$)



Satz.

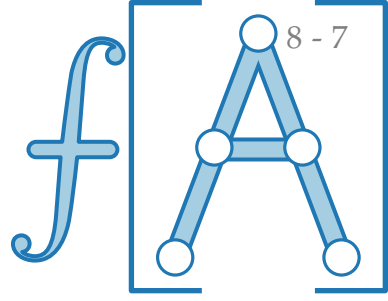
Sei $A_k \neq \emptyset$.

optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv



GREEDYRECURSIVE(int[] s , int[] e)

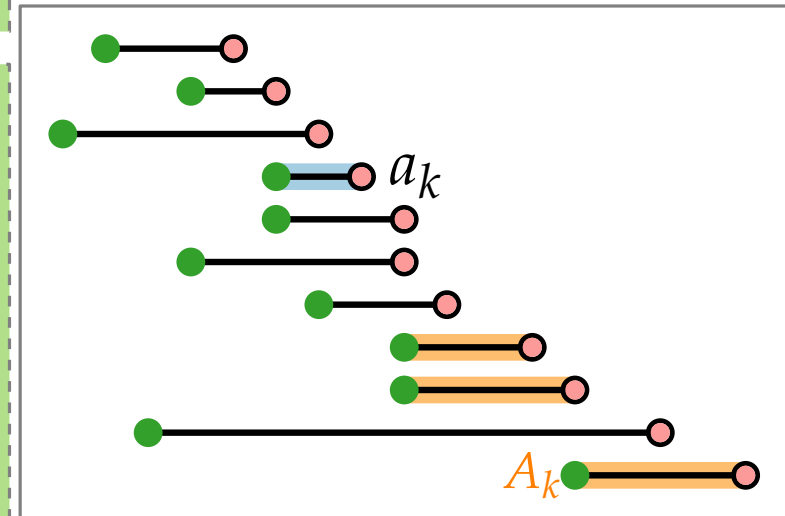
$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

return GREEDYRECURSIVEMAIN($s, e, 0$)

GREEDYRECURSIVEMAIN(int[] s , int[] e , int k) // opt. Lsg. für A_k



Satz.

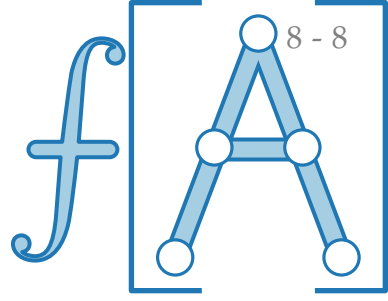
Sei $A_k \neq \emptyset$.

optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv



GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

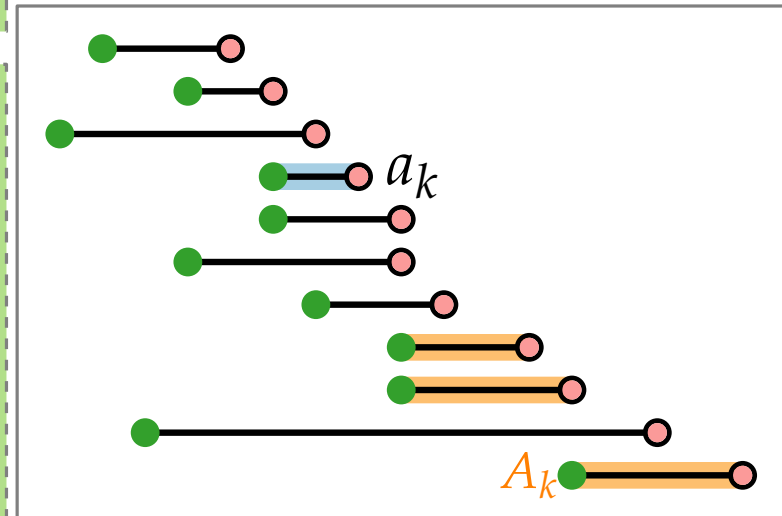
// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

return GREEDYRECURSIVEMAIN($s, e, 0$)

GREEDYRECURSIVEMAIN(int[] s , int[] e , int k) // opt. Lsg. für A_k

return $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$



Satz.

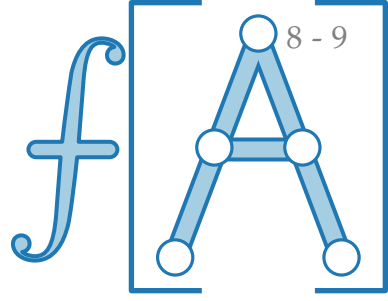
Sei $A_k \neq \emptyset$.

optimale
Teilstruktur!

Sei a_m Aktivität mit frühester Endzeit in A_k .

$\Rightarrow$ es gibt eine opt. Lösung von A_k , die a_m enthält.

Greedy – rekursiv



GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

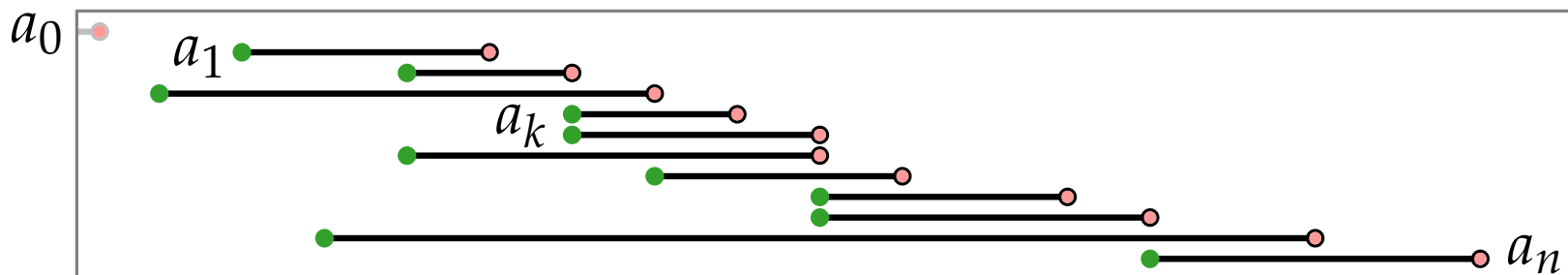
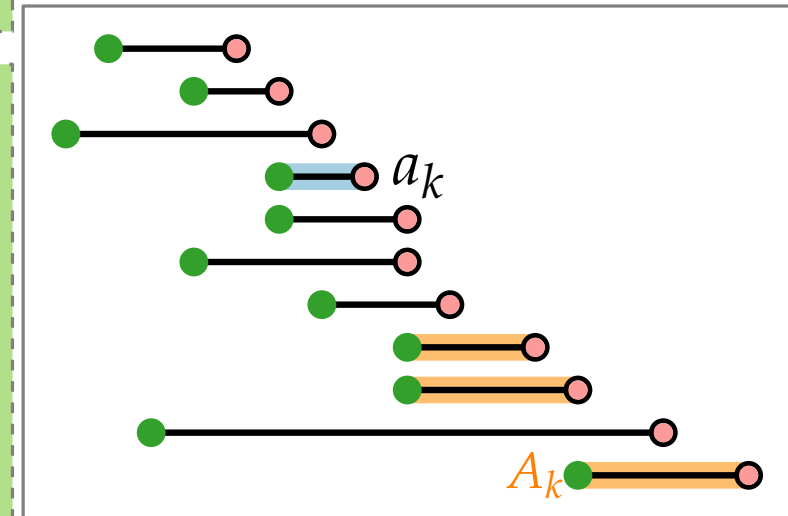
// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

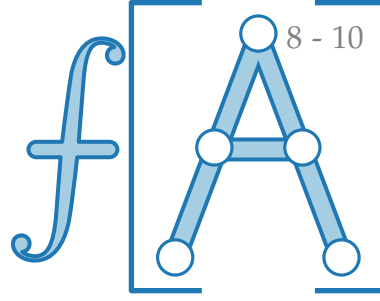
return GREEDYRECURSIVEMAIN($s, e, 0$)

GREEDYRECURSIVEMAIN(int[] s , int[] e , int k) // opt. Lsg. für A_k

return $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

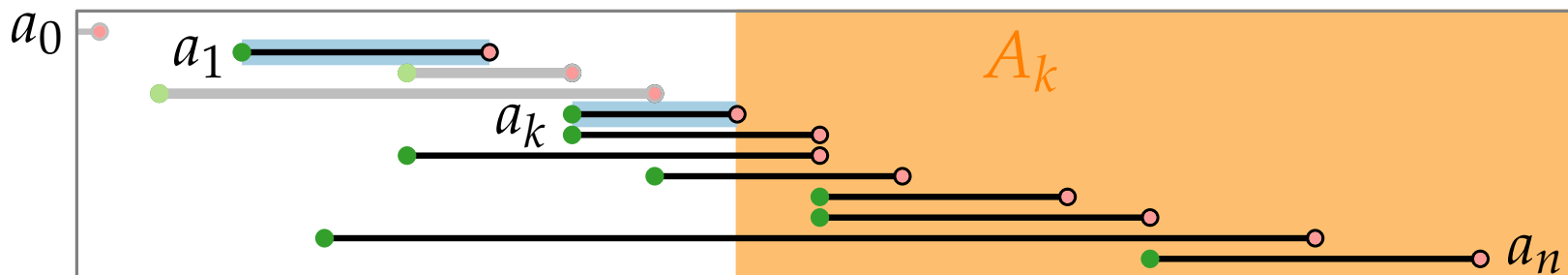
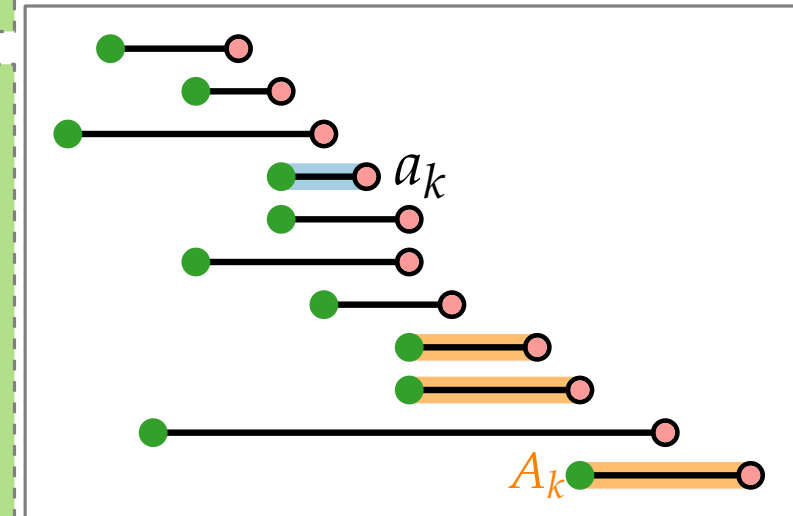
// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

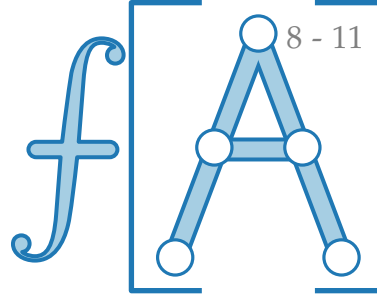
```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

```
return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(\mathbf{s}, \mathbf{e}, m)$ 
```



Greedy – rekursiv



GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

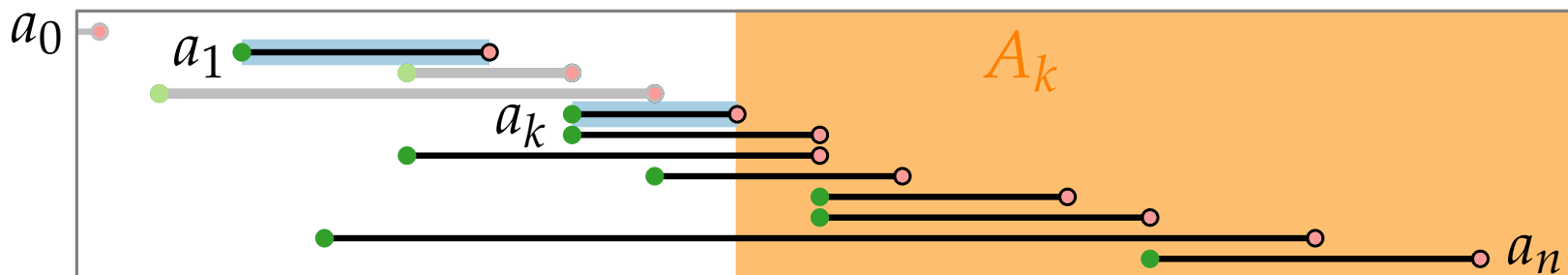
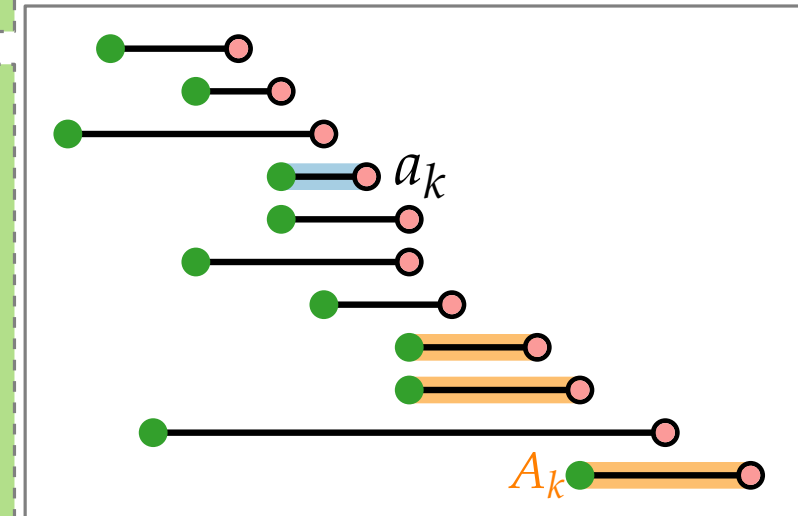
Aktivitäten nach Endzeiten sortieren, falls nötig

return GREEDYRECURSIVEMAIN($s, e, 0$)

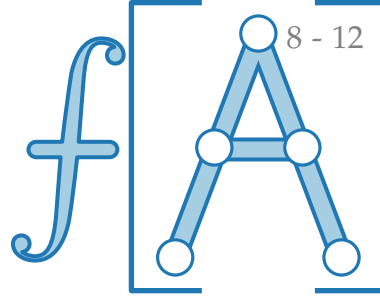
GREEDYRECURSIVEMAIN(int[] s , int[] e , int k) // opt. Lsg. für A_k

$m = k + 1$; $n = s.length$

return $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

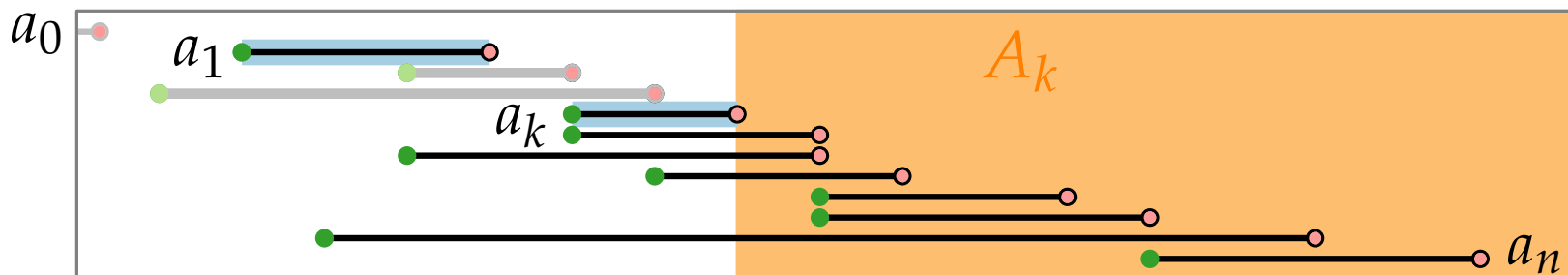
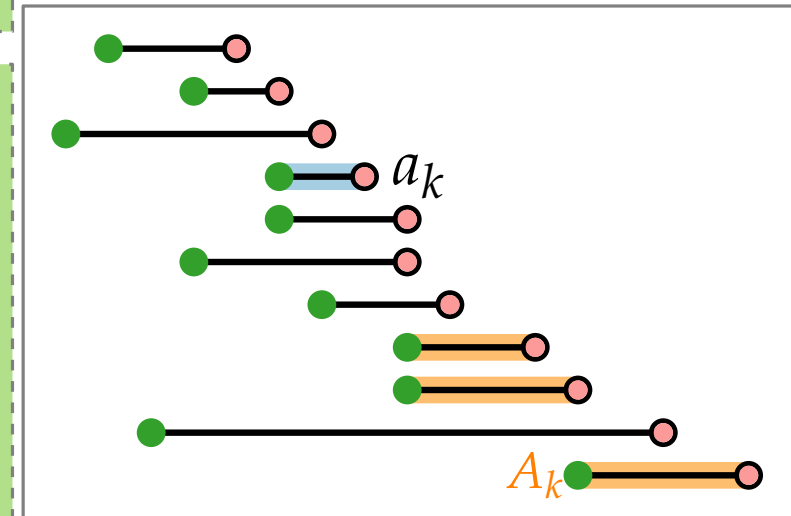
```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

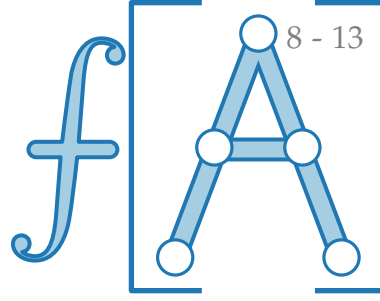
$m = k + 1; n = s.length$

// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

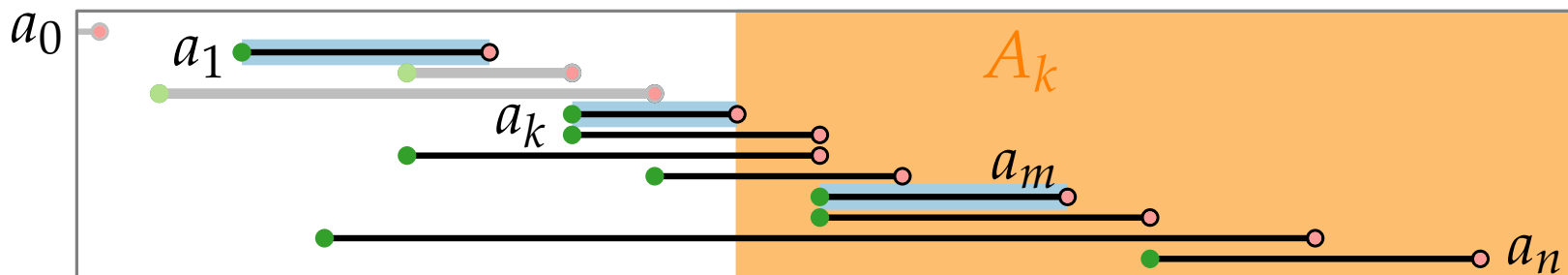
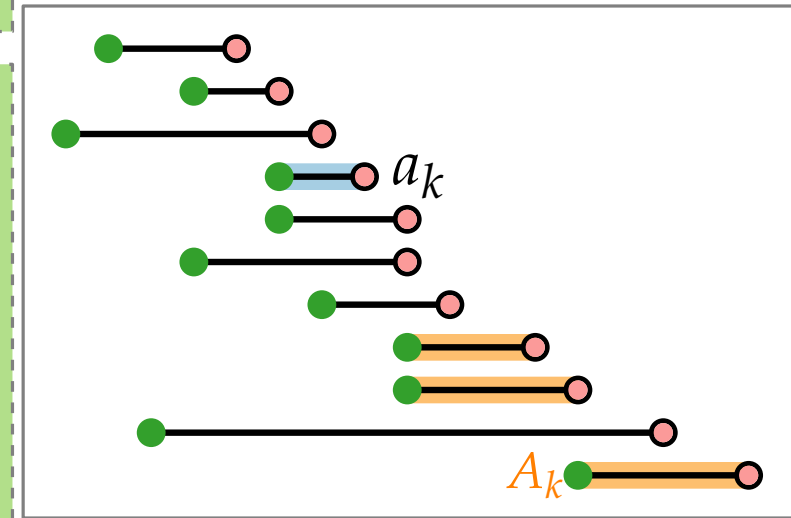
```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

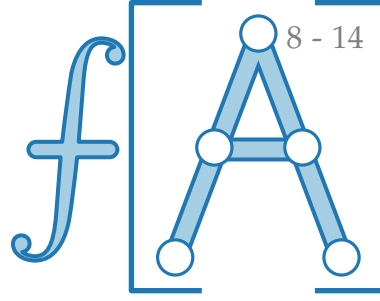
$m = k + 1; n = s.length$

// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$ // $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1$; $n = s.length$

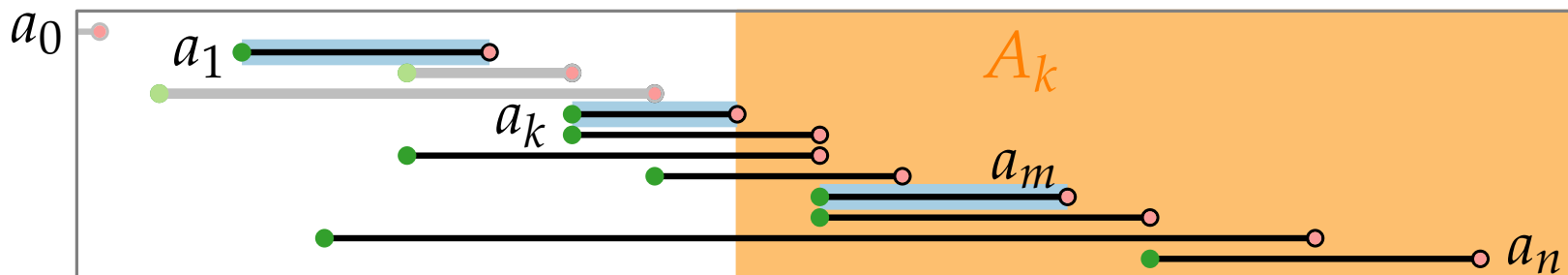
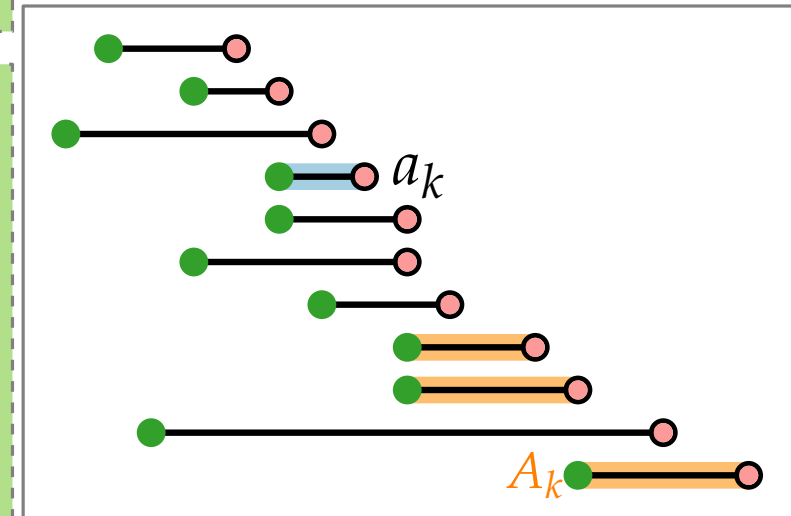
// Finde Aktivität a_m mit kleinster Endzeit in A_k

while

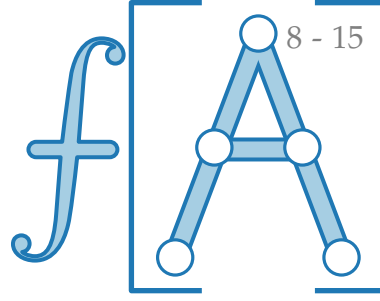
do

└

```
return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

return GREEDYRECURSIVEMAIN($s, e, 0$)

GREEDYRECURSIVEMAIN(int[] s , int[] e , int k) // opt. Lsg. für A_k

$m = k + 1; n = s.length$

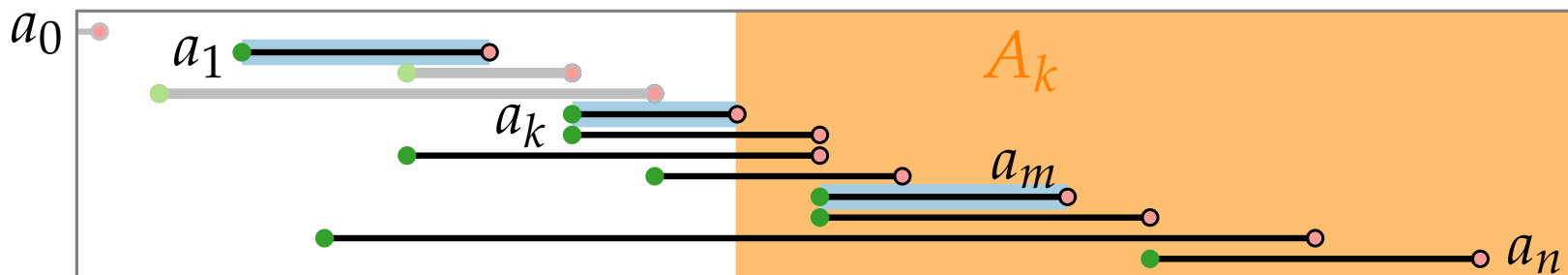
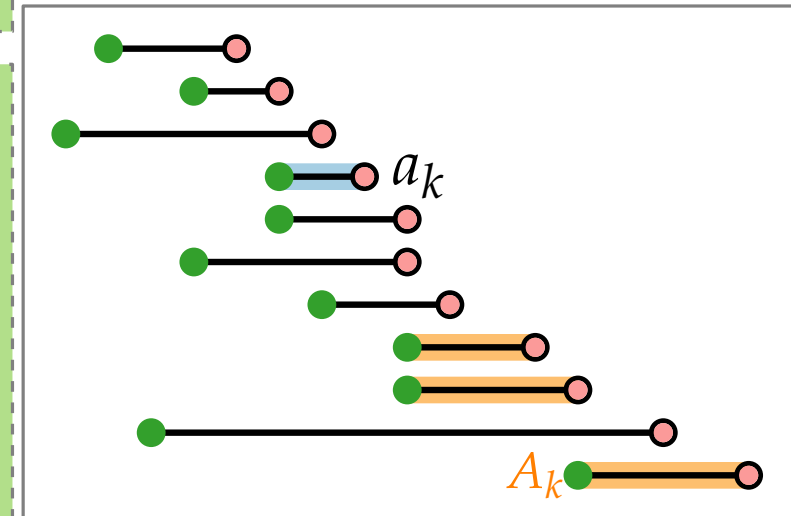
// Finde Aktivität a_m mit kleinster Endzeit in A_k

while

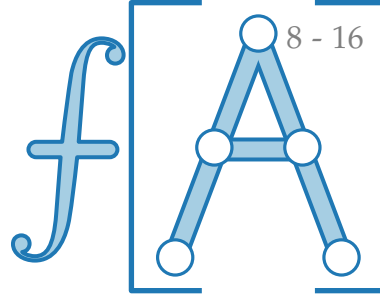
do

$\perp m = m + 1$

return $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

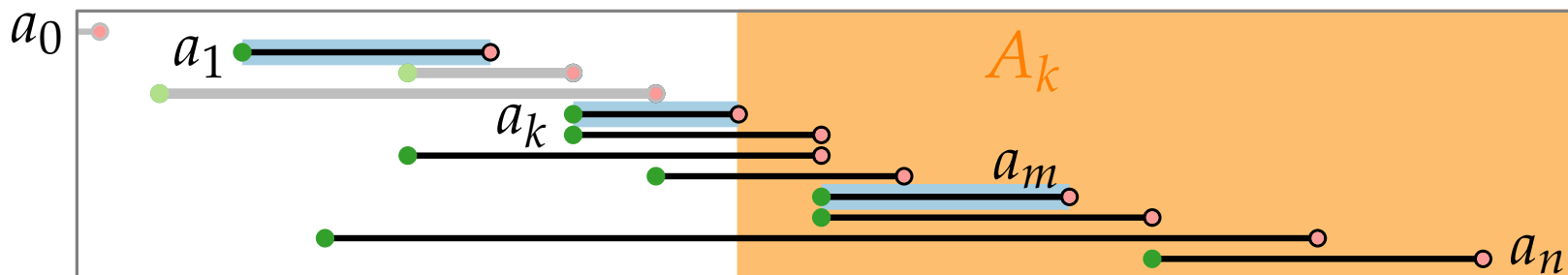
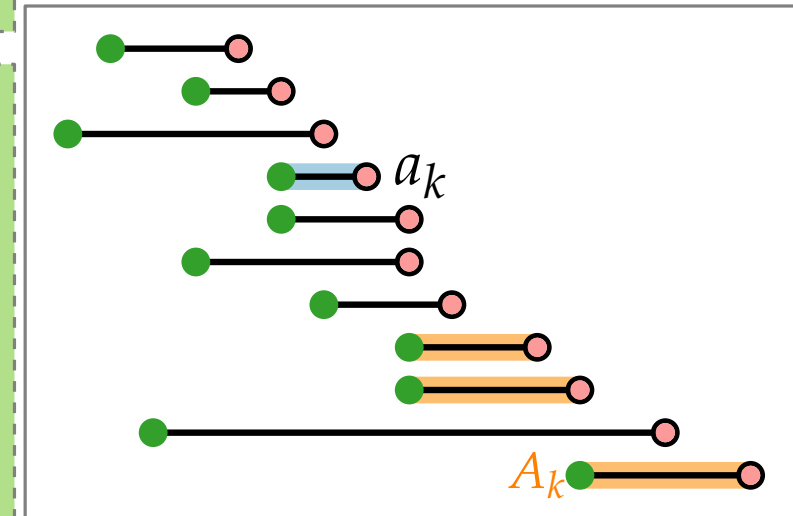
$m = k + 1; n = s.length$

// Finde Aktivität a_m mit kleinster Endzeit in A_k

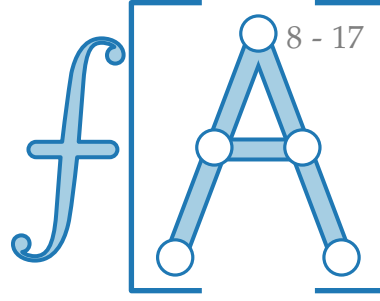
```
while  $s[m] < e[k]$  do
```

```
   $m = m + 1$ 
```

```
return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

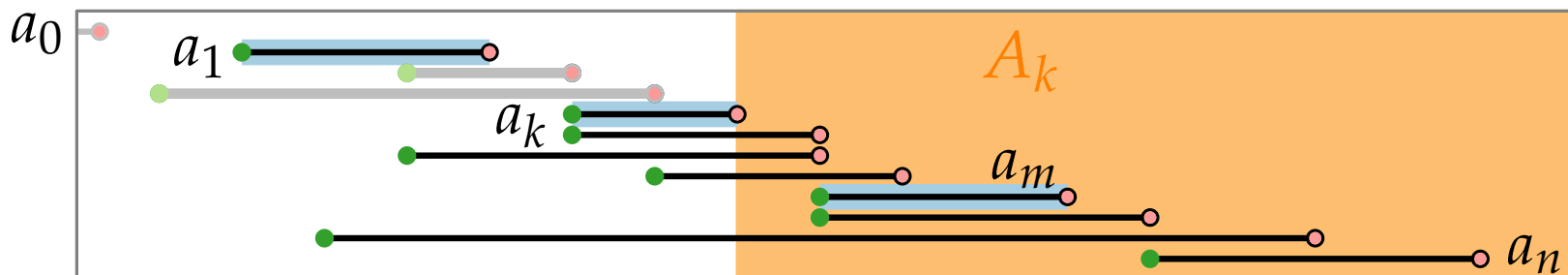
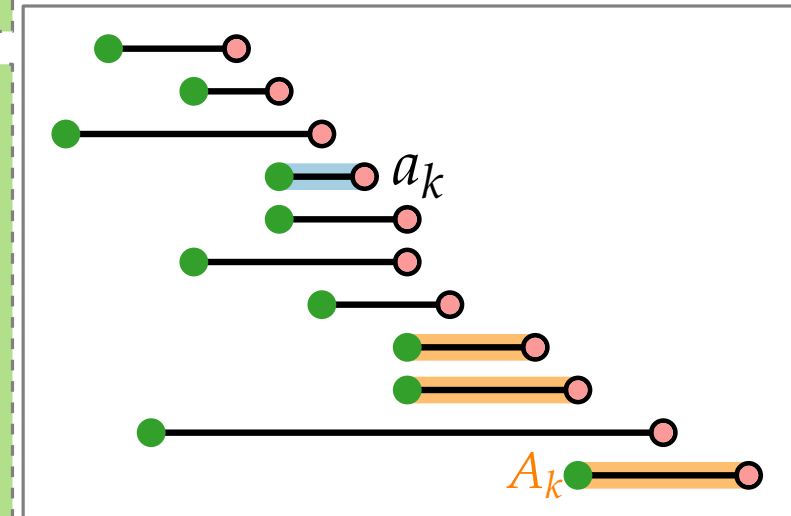
$m = k + 1; n = s.length$

// Finde Aktivität a_m mit kleinster Endzeit in A_k

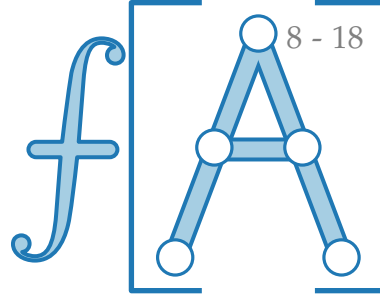
```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

```
     $m = m + 1$ 
```

```
return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

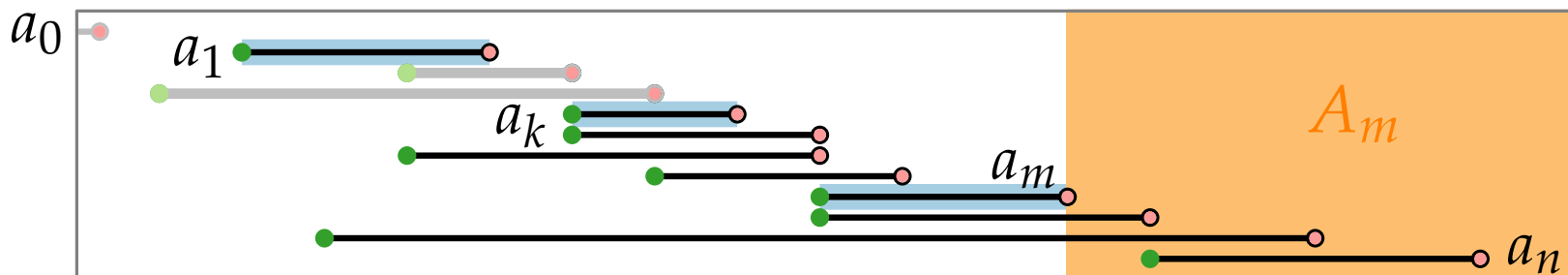
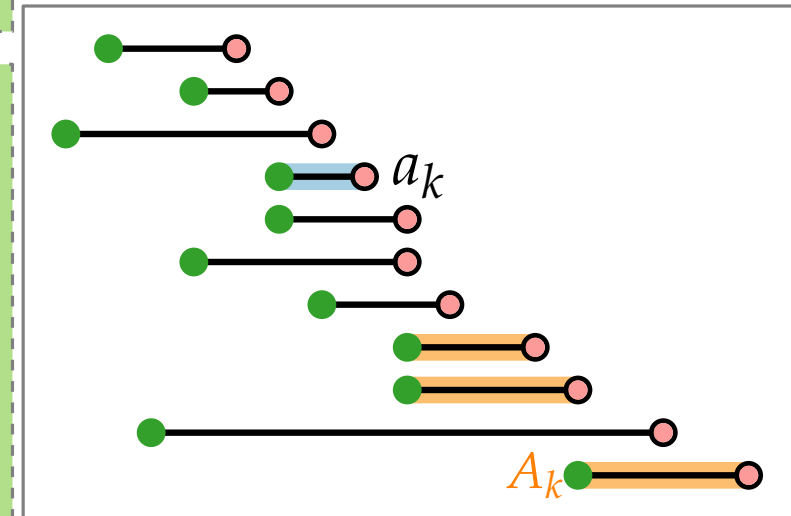
$m = k + 1; n = s.length$

// Finde Aktivität a_m mit kleinster Endzeit in A_k

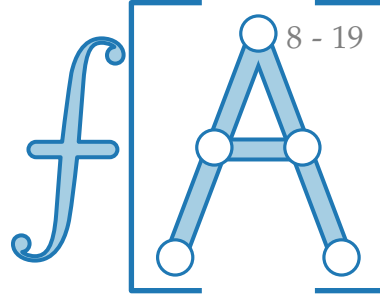
```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

```
     $m = m + 1$ 
```

```
return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



GREEDYRECURSIVE(int[] s , int[] e)

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

return GREEDYRECURSIVEMAIN($s, e, 0$)

GREEDYRECURSIVEMAIN(int[] s , int[] e , int k) // opt. Lsg. für A_k

$m = k + 1; n = s.length$

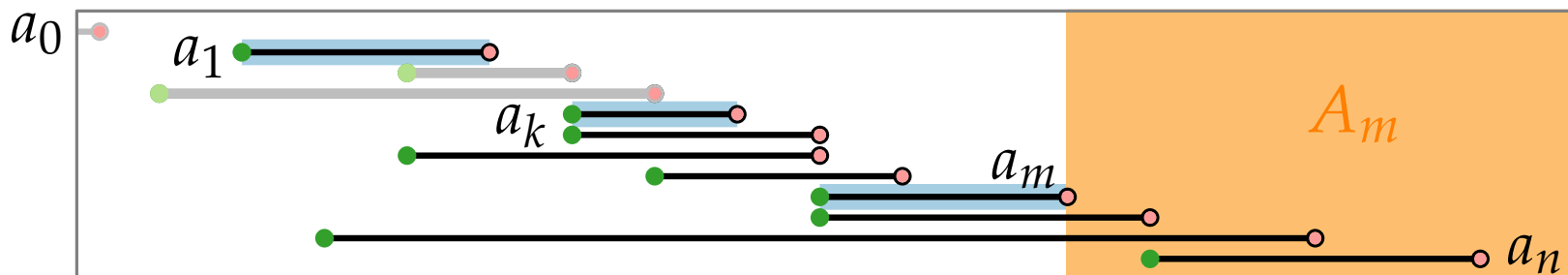
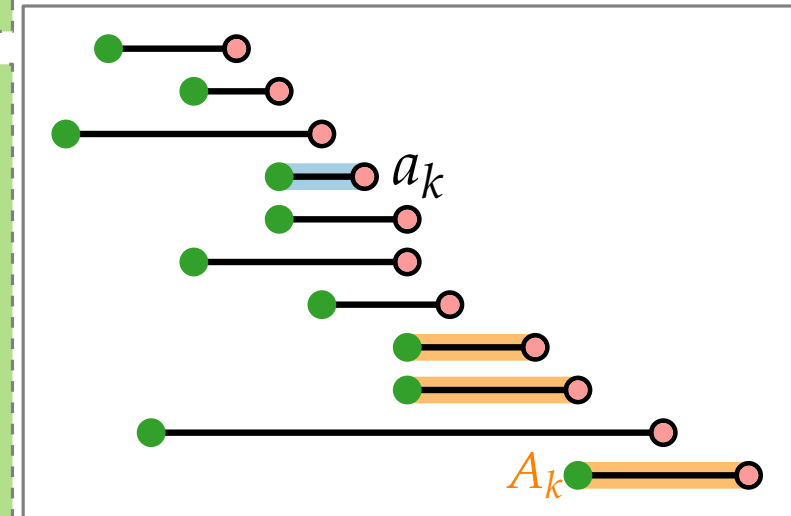
// Finde Aktivität a_m mit kleinster Endzeit in A_k

while $m \leq n$ **and** $s[m] < e[k]$ **do**

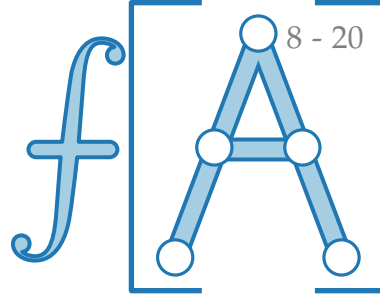
$m = m + 1$

if $m > n$ **then return** $\emptyset$

else return $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

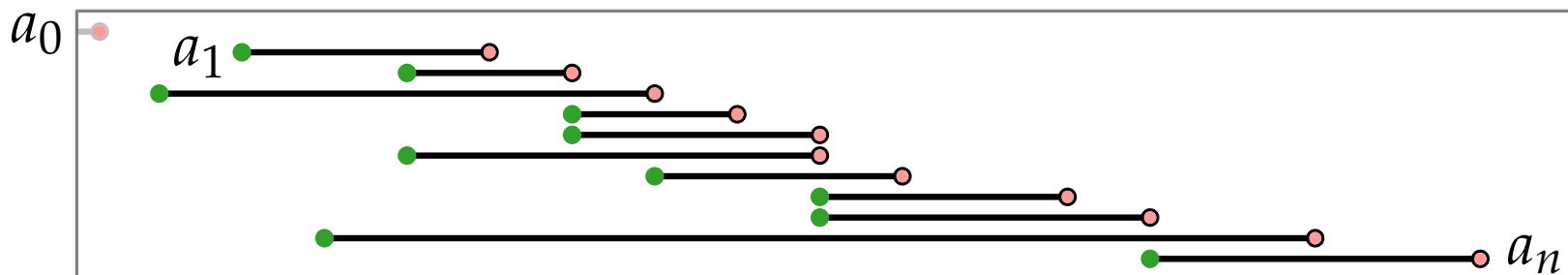
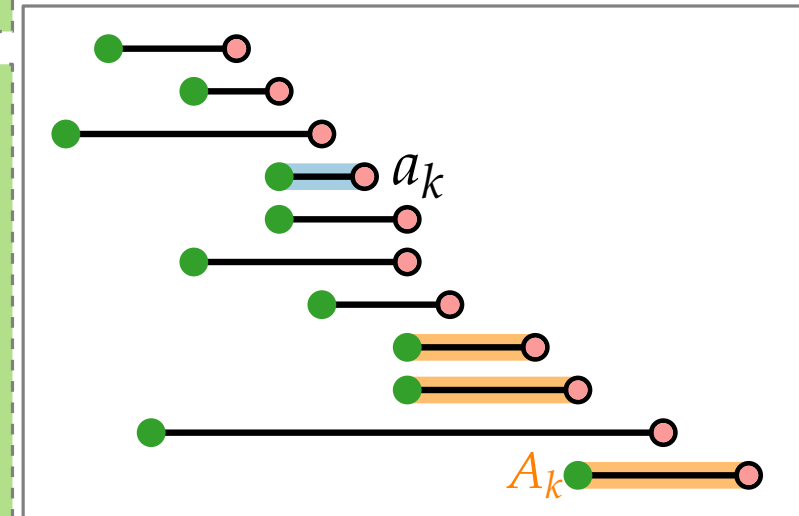
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

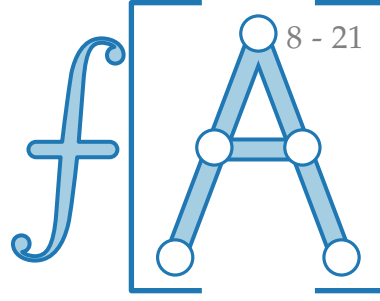
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

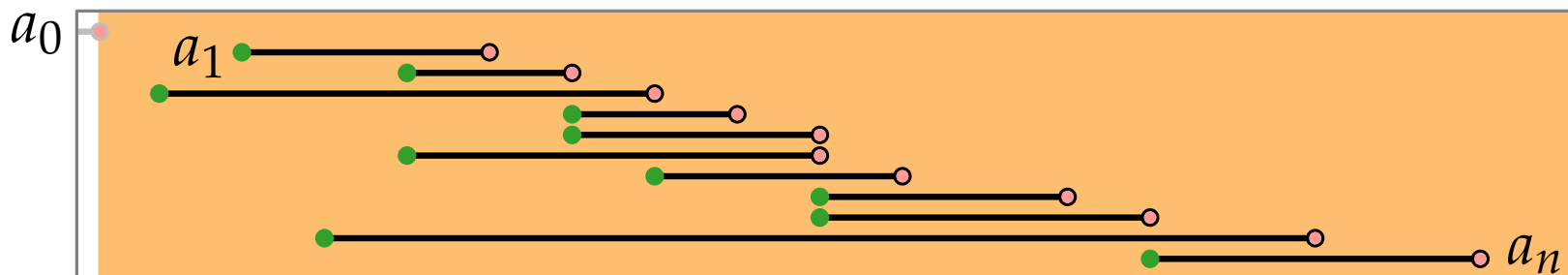
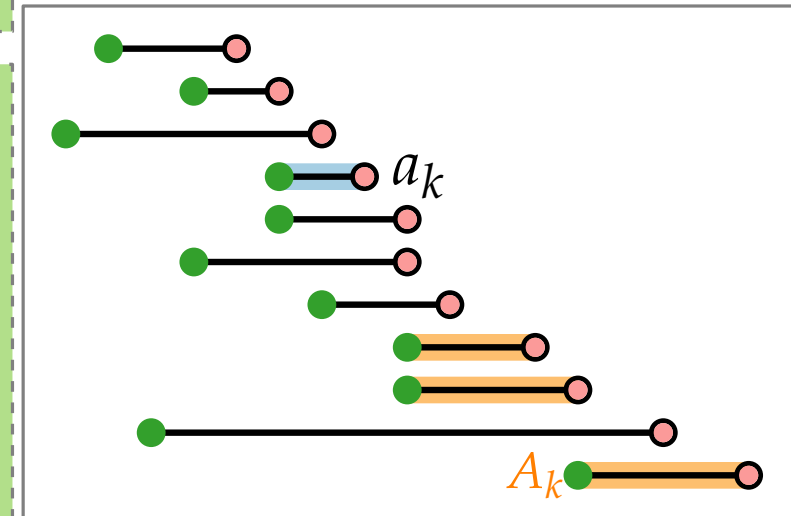
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

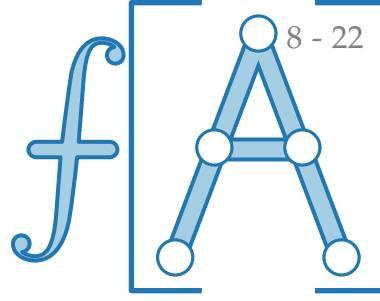
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

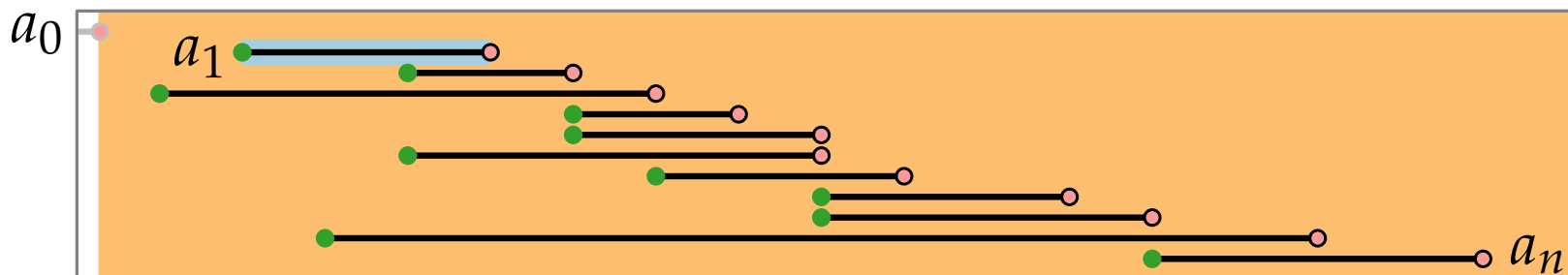
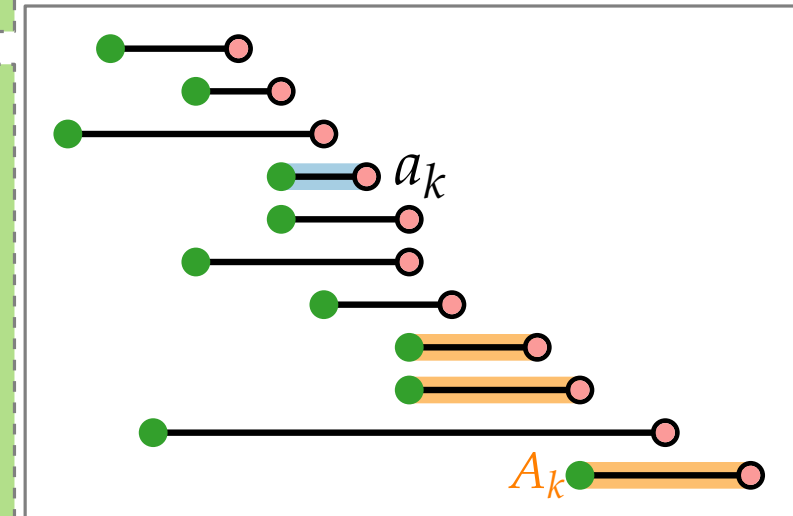
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

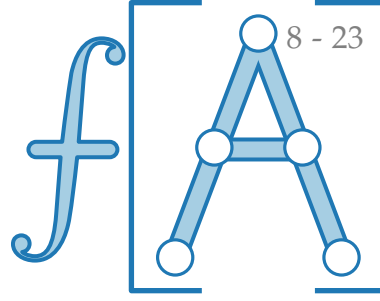
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

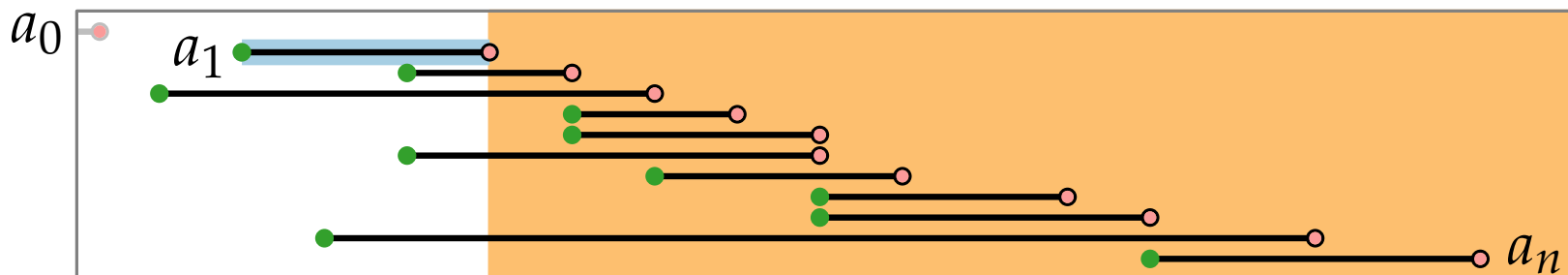
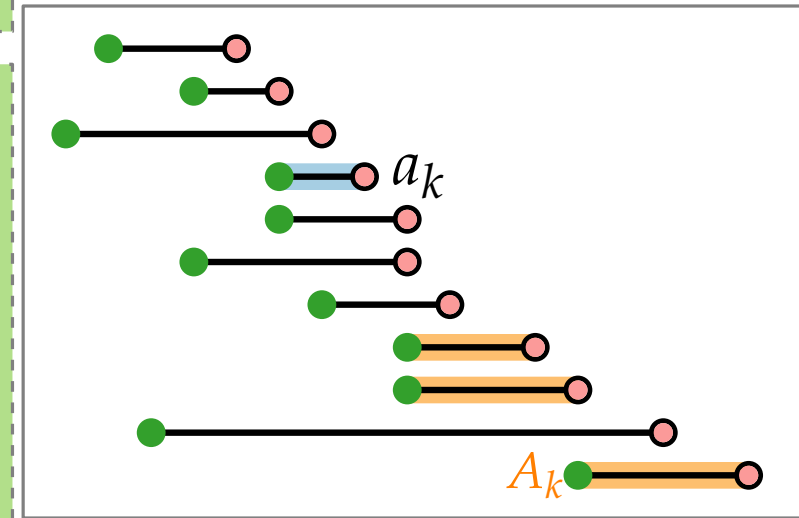
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

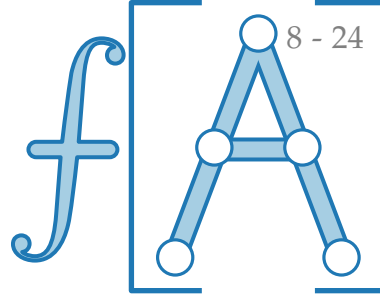
```
   $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

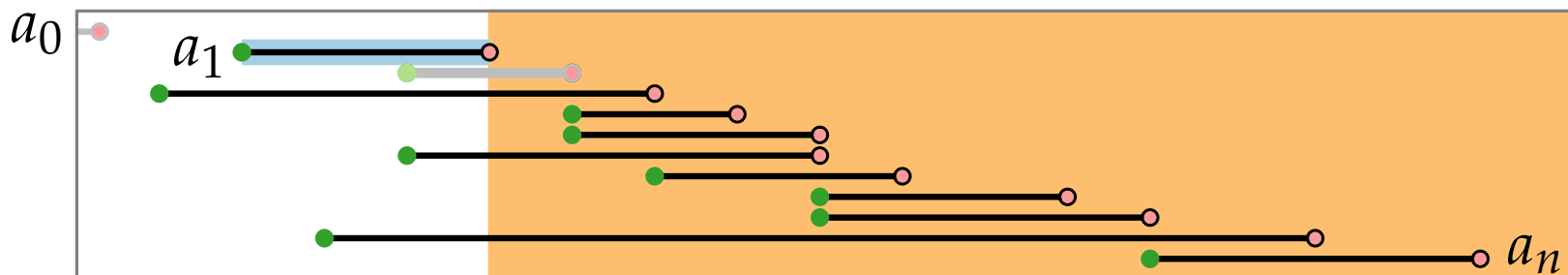
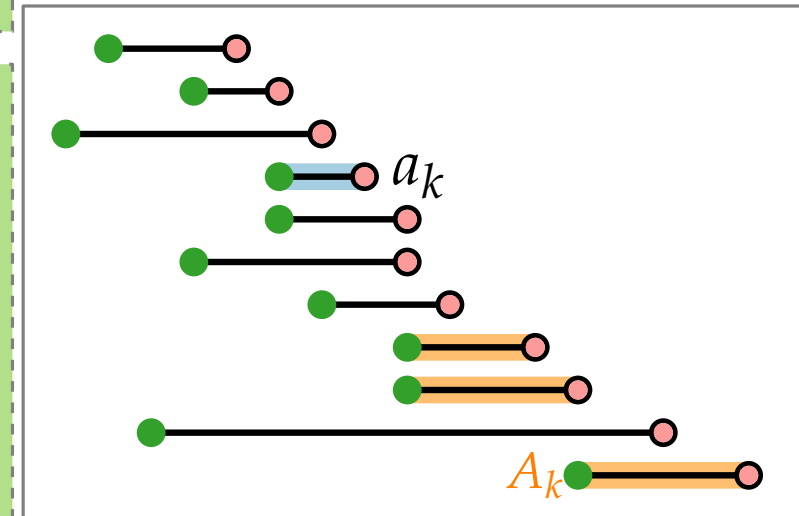
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

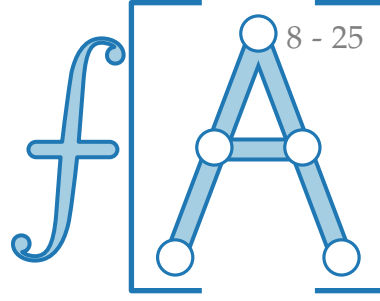
```
   $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

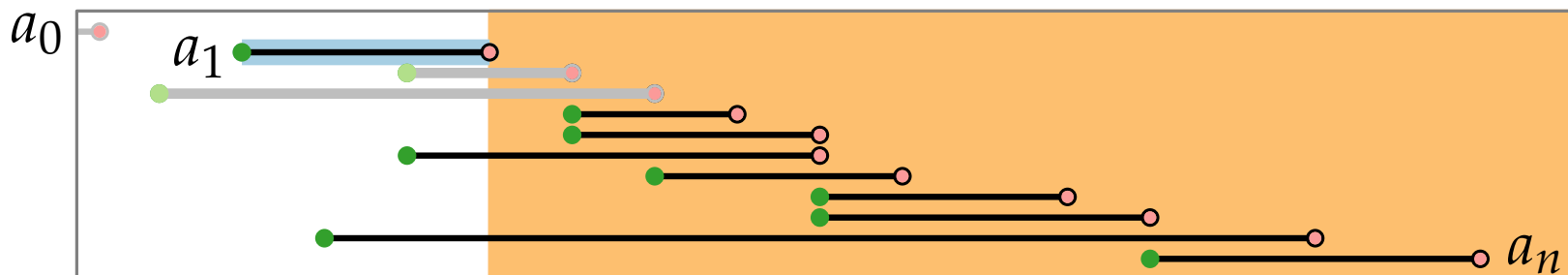
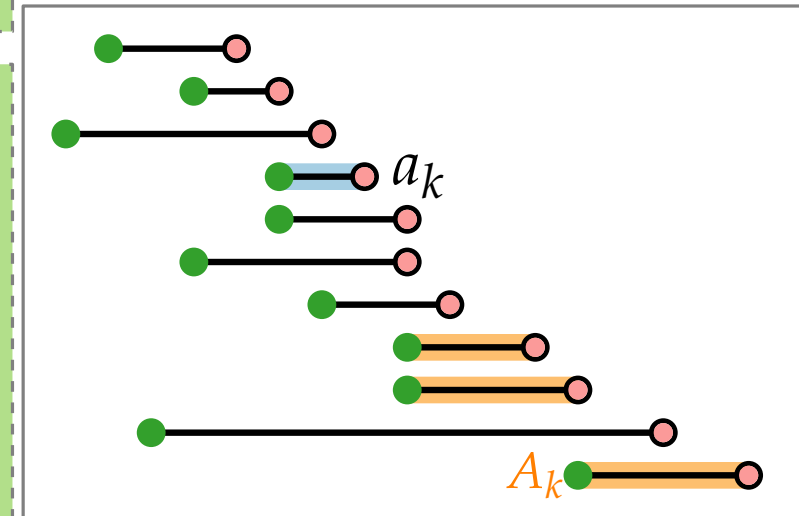
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

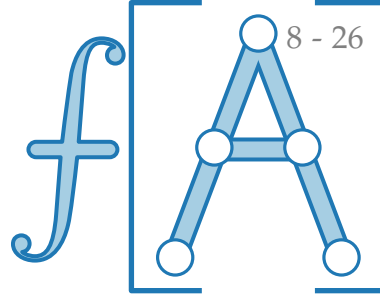
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

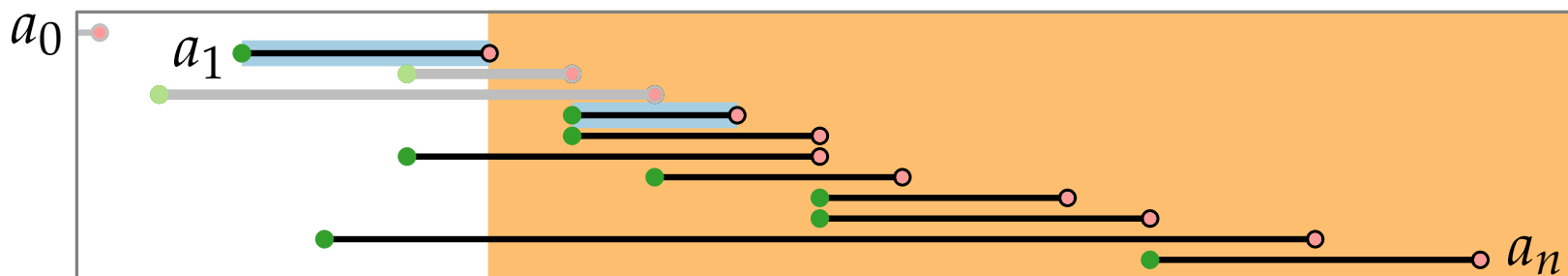
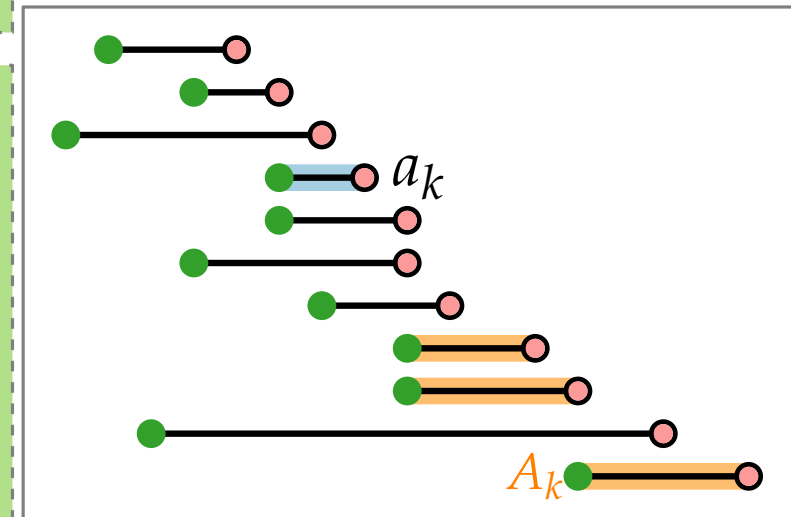
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

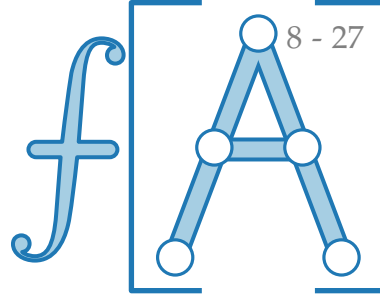
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

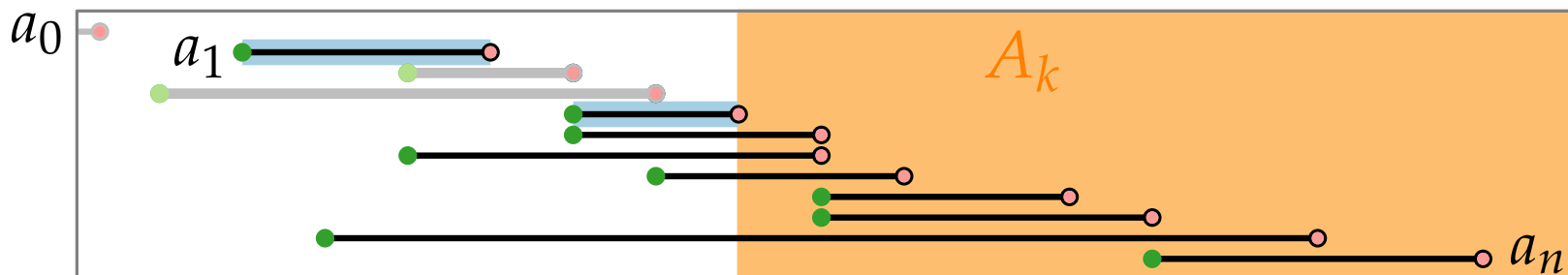
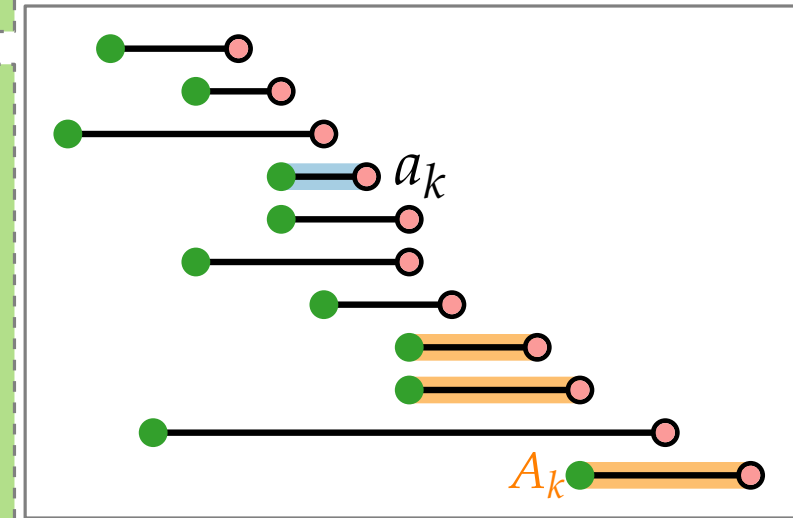
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

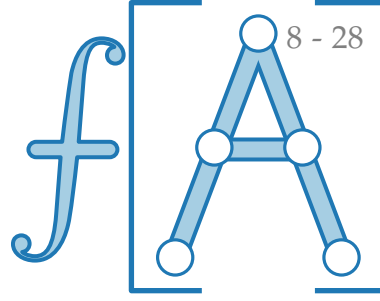
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

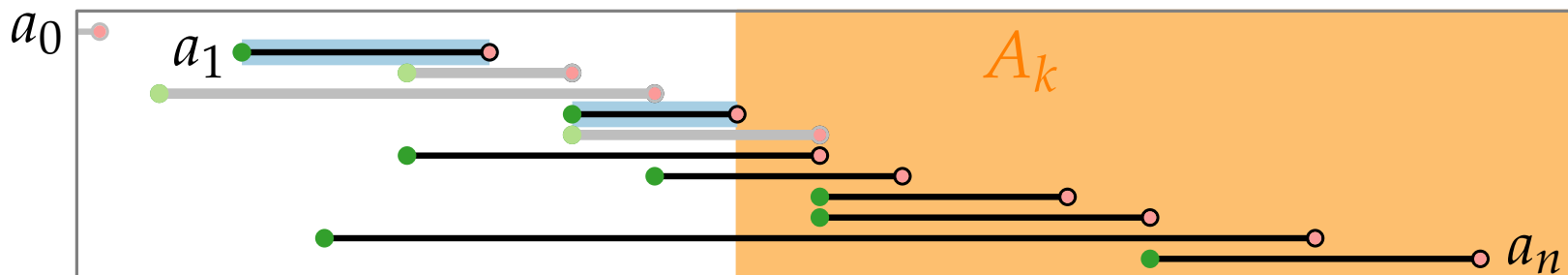
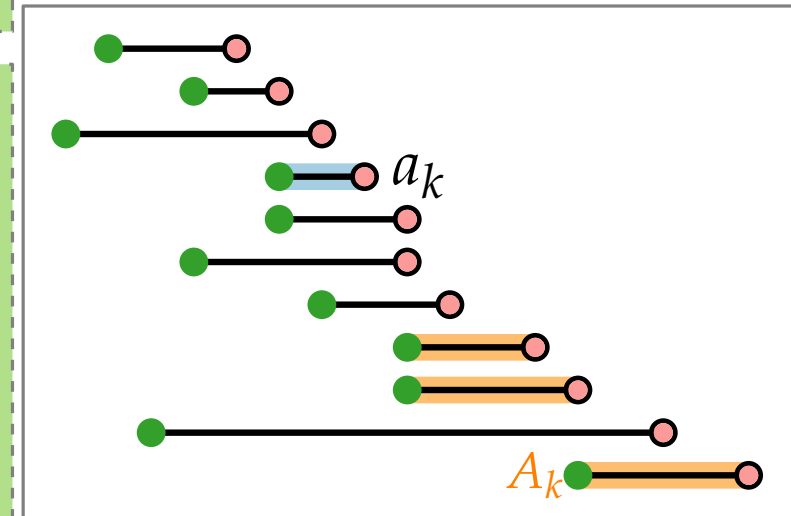
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

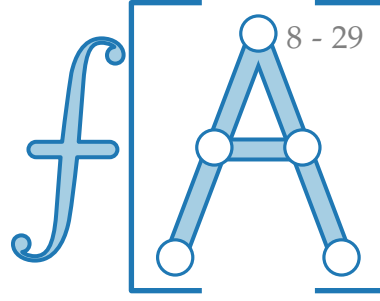
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

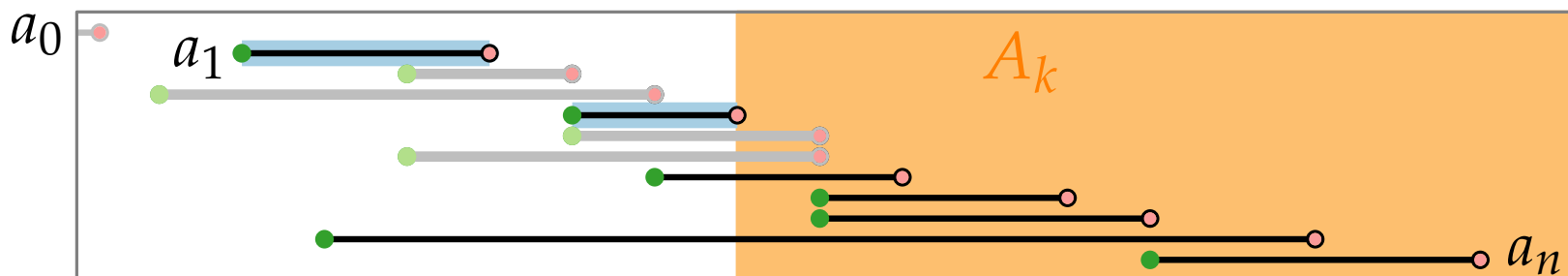
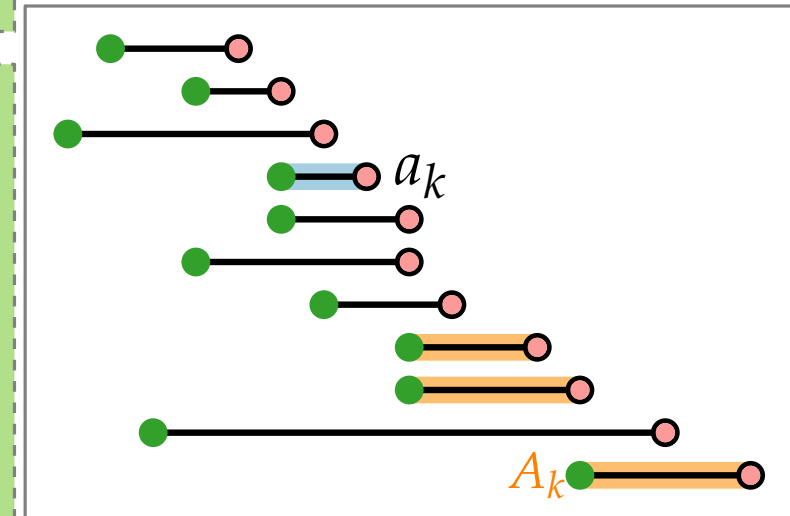
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

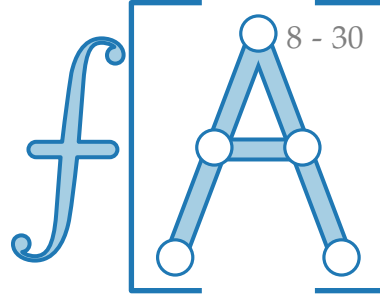
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

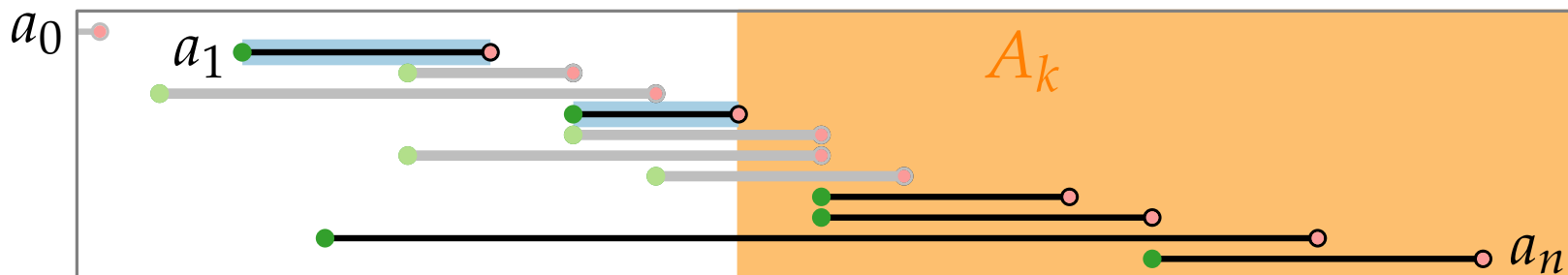
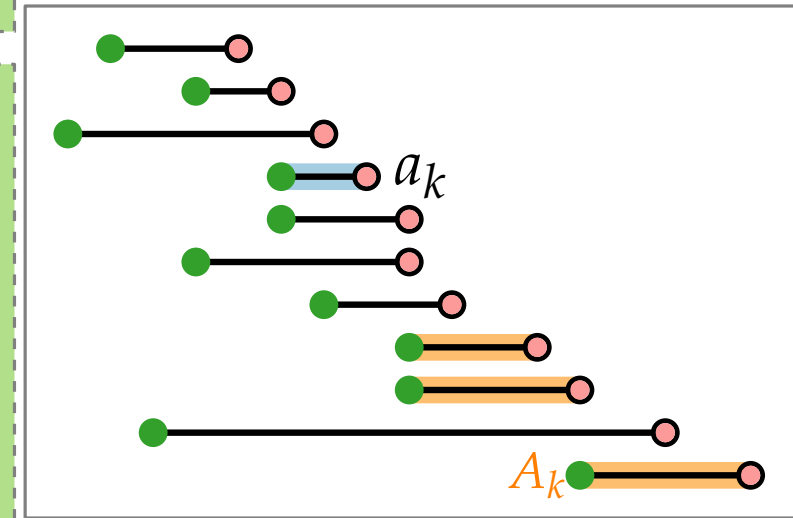
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

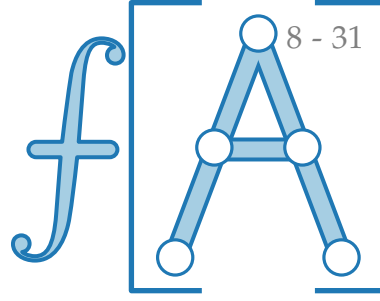
```
   $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

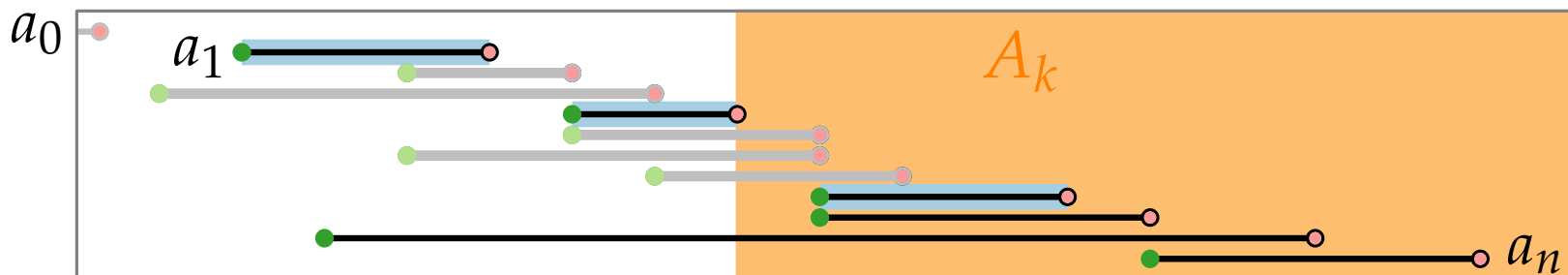
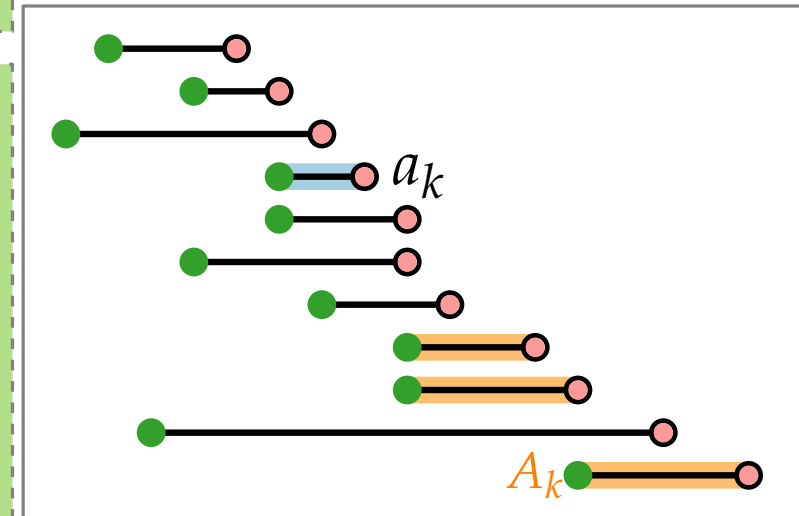
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

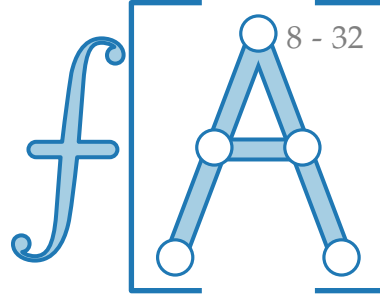
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

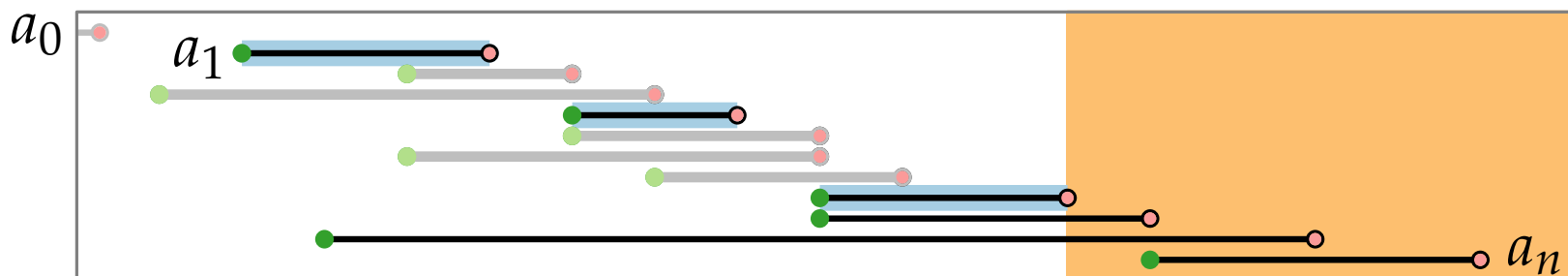
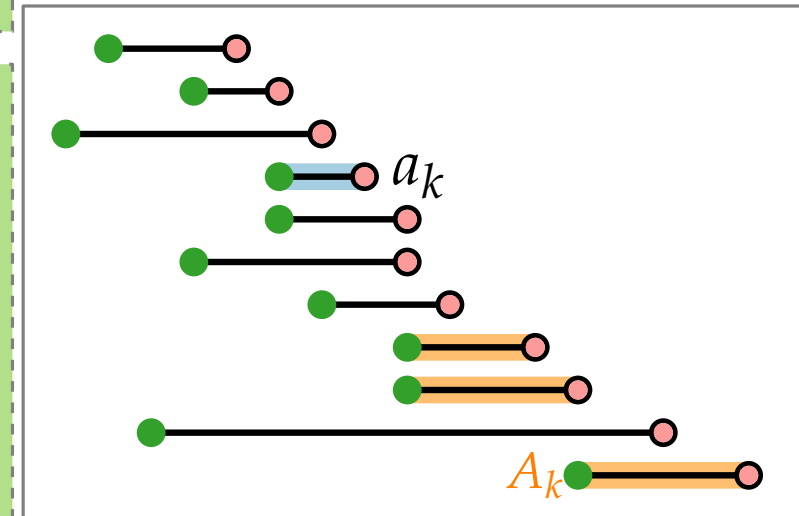
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

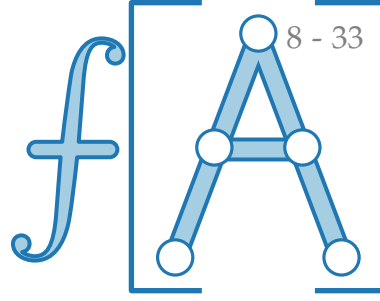
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

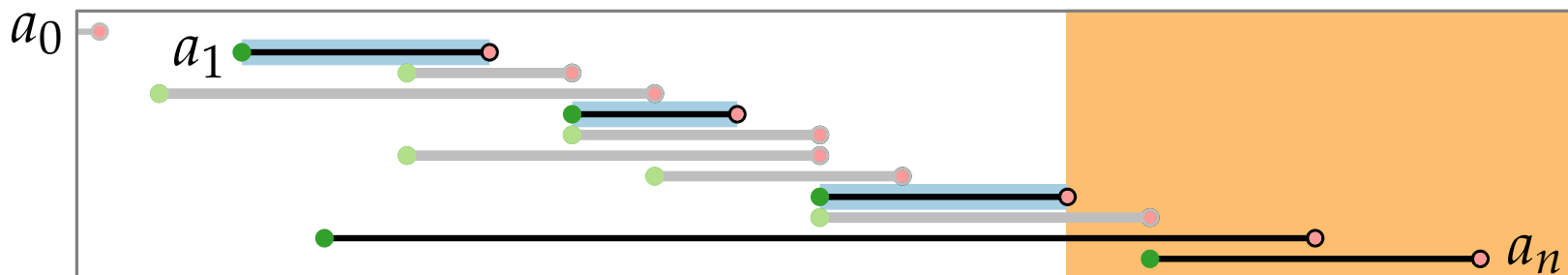
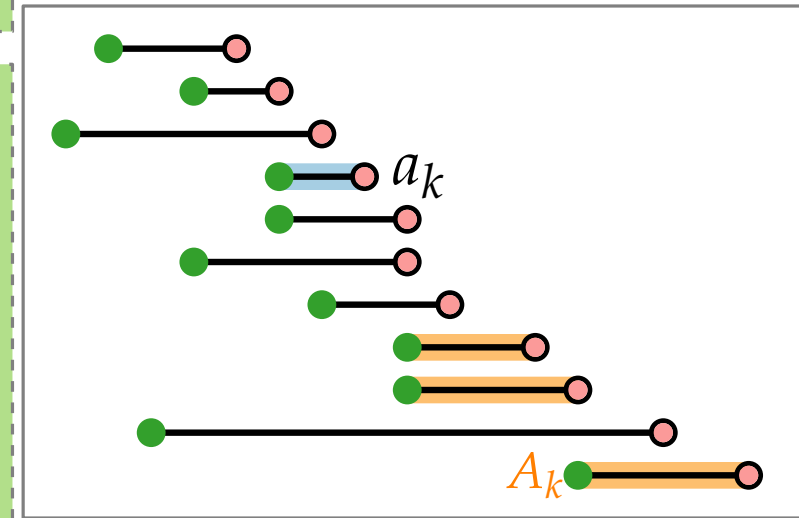
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

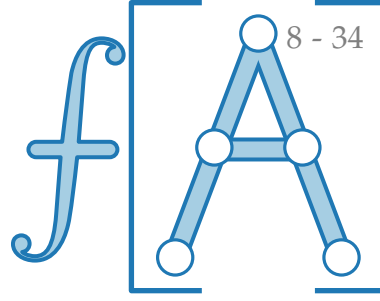
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

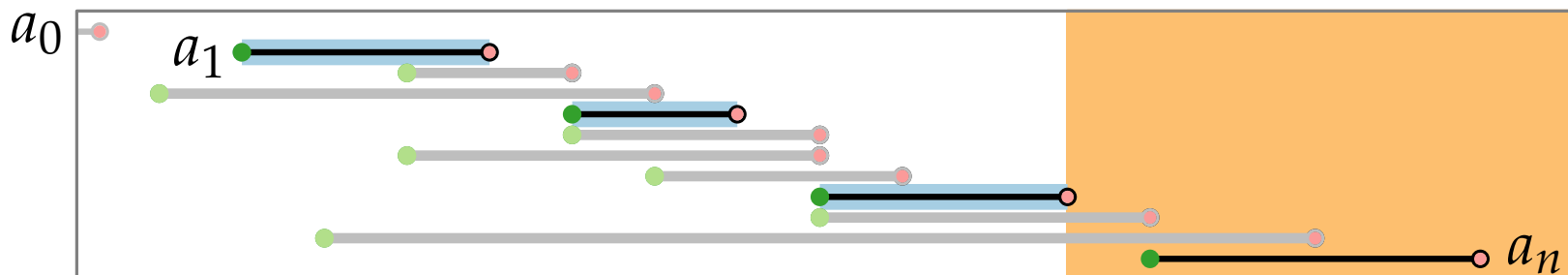
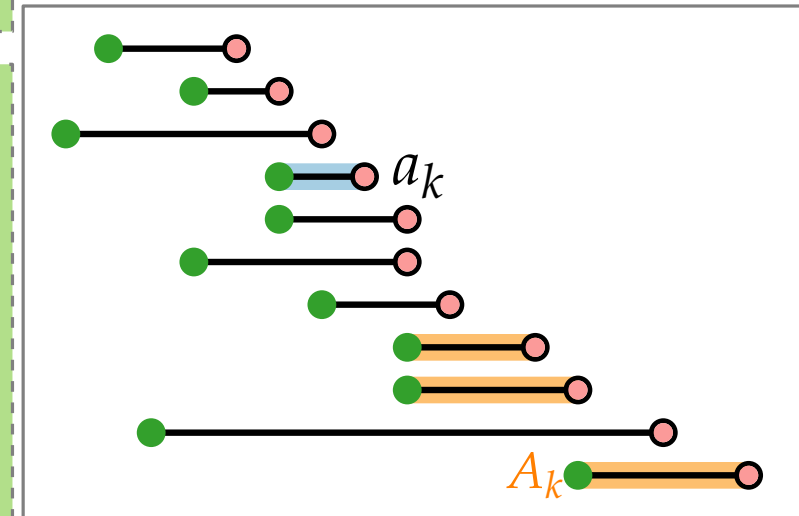
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

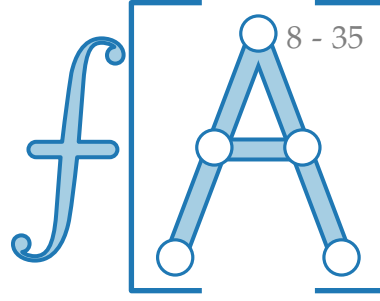
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

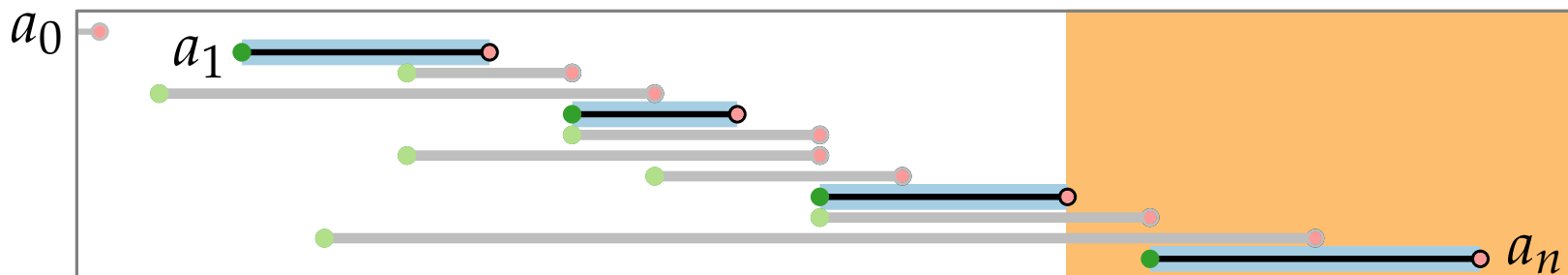
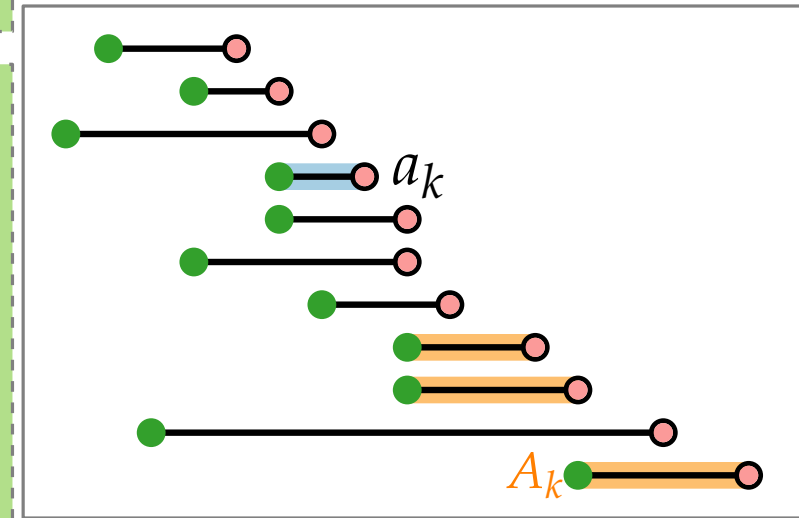
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

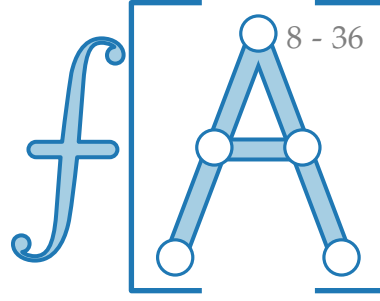
```
     $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1; n = s.length$

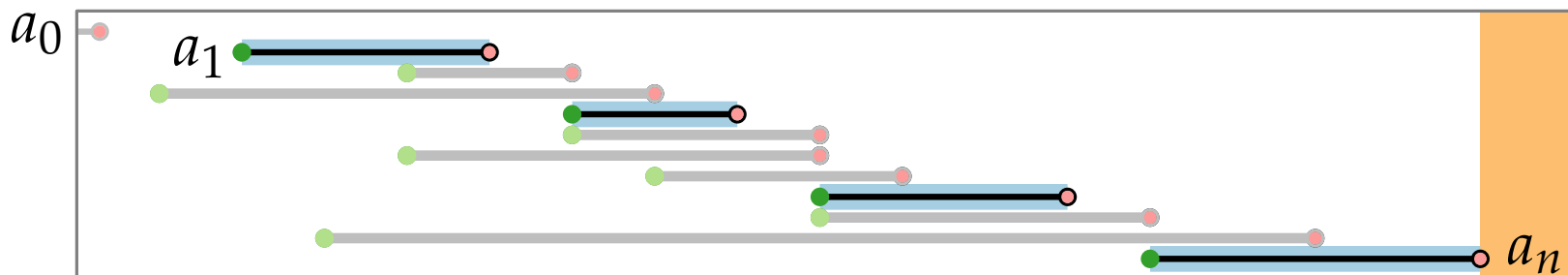
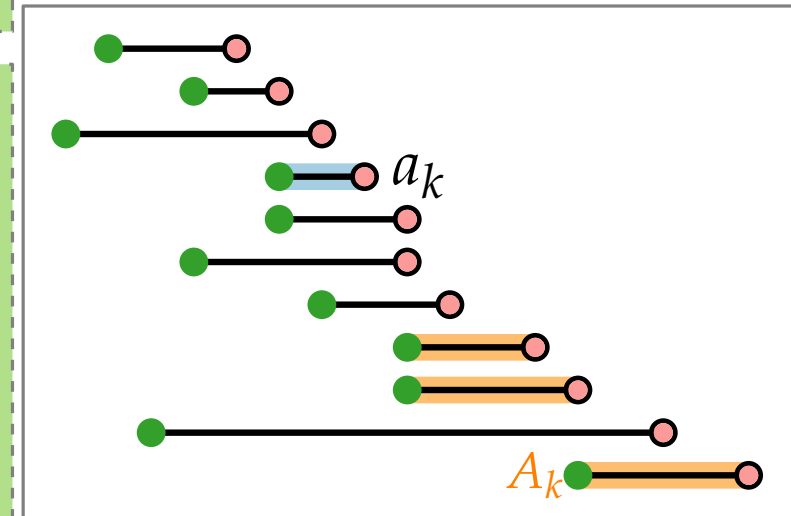
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

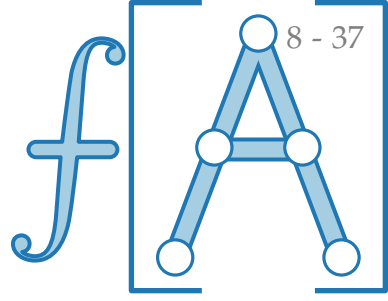
```
   $m = m + 1$ 
```

```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$ // $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

$m = k + 1$; $n = s.length$

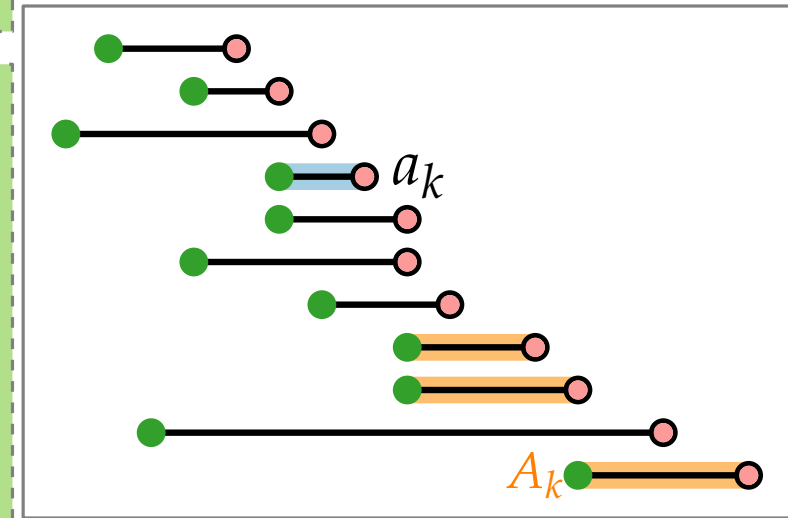
// Finde Aktivität a_m mit kleinster Endzeit in A_k

```
while  $m \leq n$  and  $s[m] < e[k]$  do
```

```
   $m = m + 1$ 
```

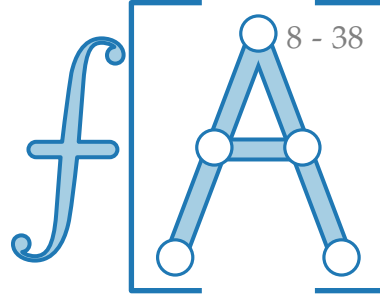
```
if  $m > n$  then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(s, e, m)$ 
```



Laufzeit?

Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

```
m = k + 1; n = s.length
```

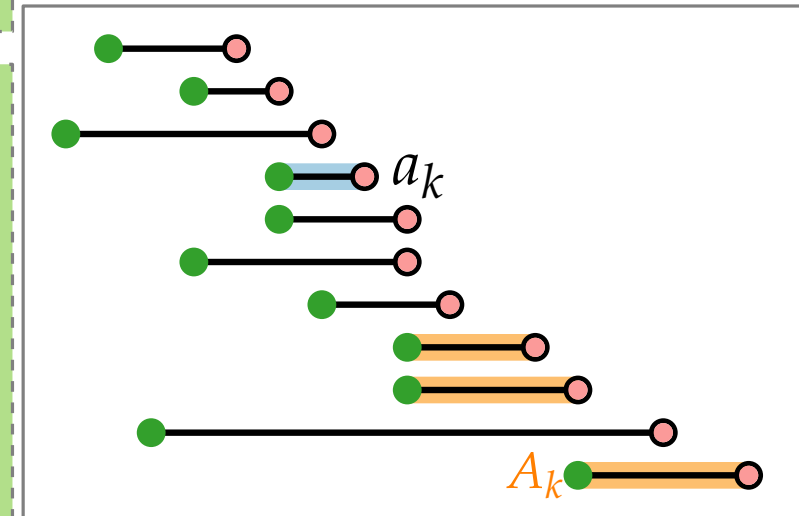
```
// Finde Aktivität  $a_m$  mit kleinster Endzeit in  $A_k$ 
```

```
while m ≤ n and s[m] < e[k] do
```

```
    m = m + 1
```

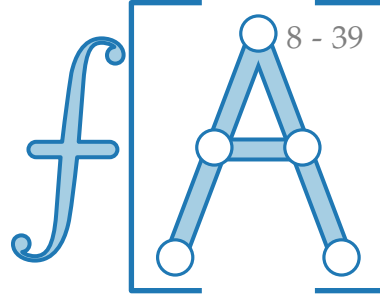
```
if m > n then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(\mathbf{s}, \mathbf{e}, \mathbf{m})$ 
```



Laufzeit? Wie oft wird *m* inkrementiert?

Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

$e_0 = -\infty$

// $\Rightarrow A_0 = A$

Aktivitäten nach Endzeiten sortieren, falls nötig

```
return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für  $A_k$ 
```

```
m = k + 1; n = s.length
```

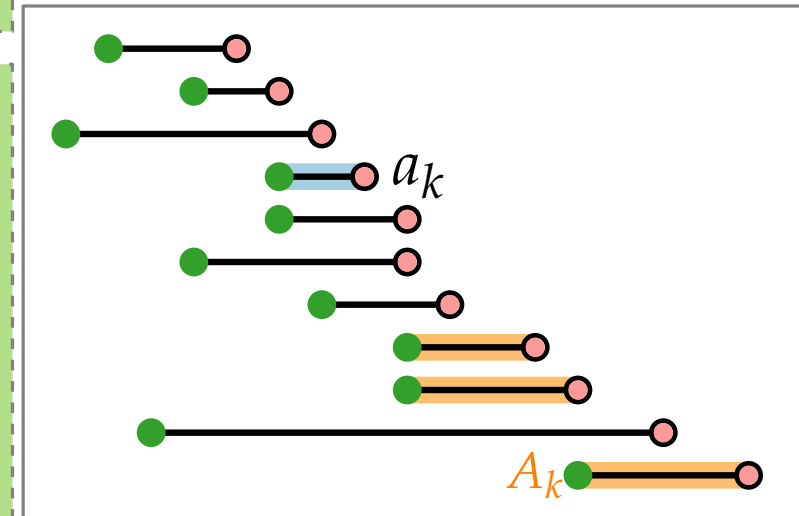
```
// Finde Aktivität  $a_m$  mit kleinster Endzeit in  $A_k$ 
```

```
while m ≤ n and s[m] < e[k] do
```

```
└ m = m + 1
```

```
if m > n then return  $\emptyset$ 
```

```
else return  $\{a_m\} \cup \text{GREEDYRECURSIVEMAIN}(\mathbf{s}, \mathbf{e}, \mathbf{m})$ 
```

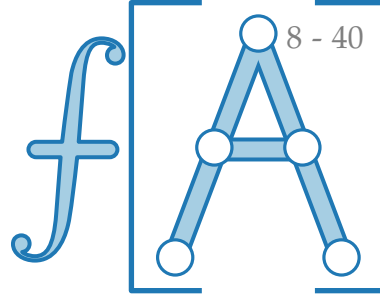


Laufzeit?

Wie oft wird *m* inkrementiert?

Insgesamt, über alle rekursiven Aufrufe, *n* Mal.

Greedy – rekursiv



```
GREEDYRECURSIVE(int[ ] s, int[ ] e)
```

```
    e0 = -∞
```

```
    // ⇒ A0 = A
```

```
    Aktivitäten nach Endzeiten sortieren, falls nötig
```

```
    return GREEDYRECURSIVEMAIN(s, e, 0)
```

```
GREEDYRECURSIVEMAIN(int[ ] s, int[ ] e, int k) // opt. Lsg. für Ak
```

```
    m = k + 1; n = s.length
```

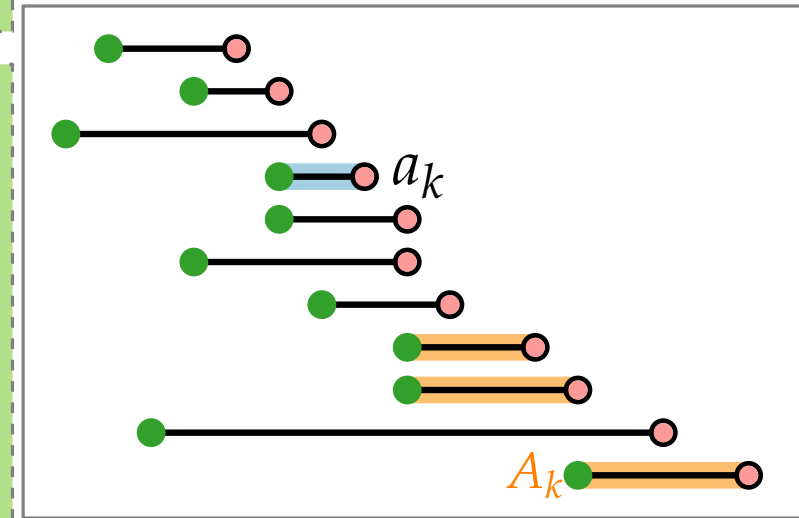
```
    // Finde Aktivität am mit kleinster Endzeit in Ak
```

```
    while m ≤ n and s[m] < e[k] do
```

```
        m = m + 1
```

```
    if m > n then return ∅
```

```
    else return {am} ∪ GREEDYRECURSIVEMAIN(s, e, m)
```



Laufzeit?

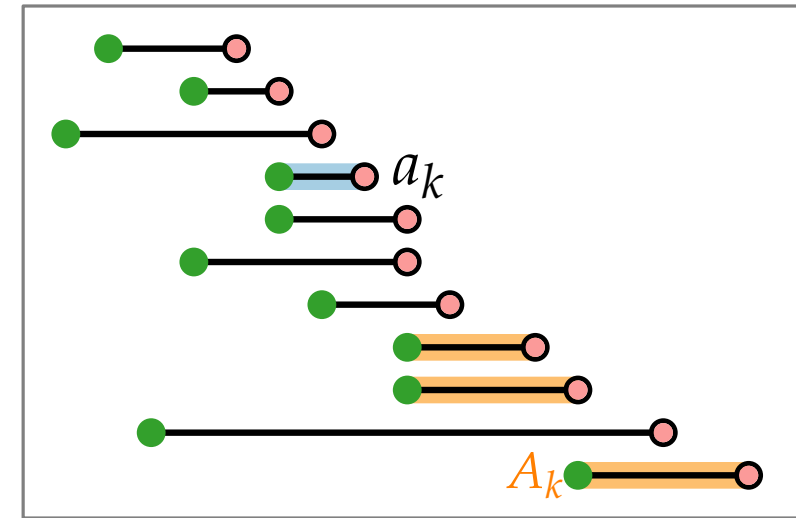
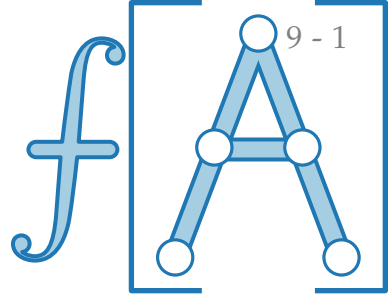
Wie oft wird *m* inkrementiert?

Insgesamt, über alle rekursiven Aufrufe, *n* Mal.

D.h. GREEDYRECURSIVE läuft (ohne Sortieren) in $\Theta(n)$ Zeit.

Greedy – iterativ

```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
   $n = s.length$ 
```

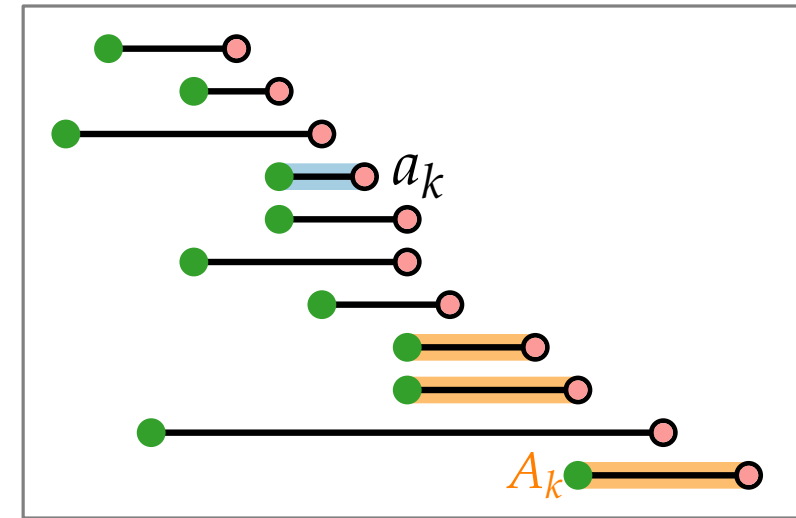
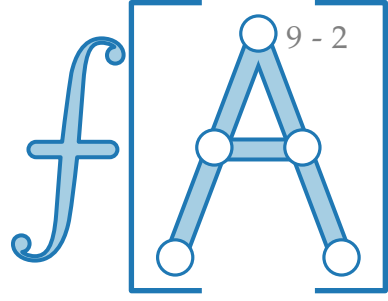


Greedy – iterativ

```
GREEDYITERATIVE(int[ ] s, int[ ] e)
```

```
   $n = s.length$ 
```

```
  if  $n = 0$  then return  $\emptyset$ 
```



Greedy – iterativ

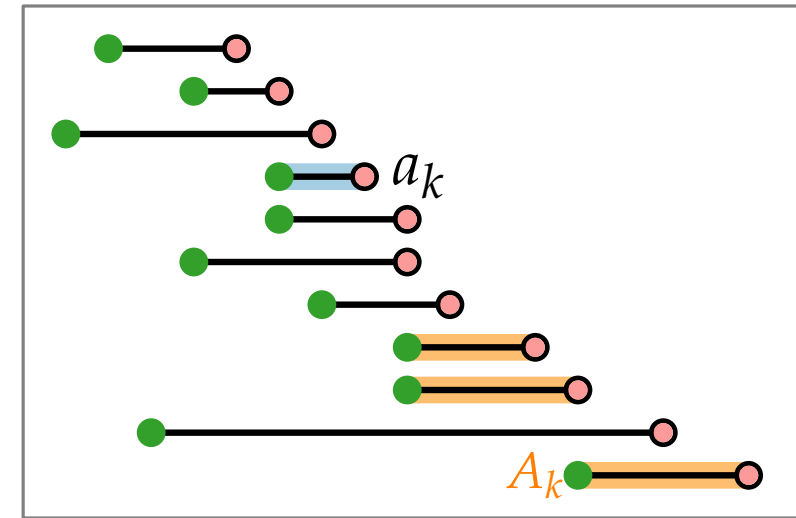
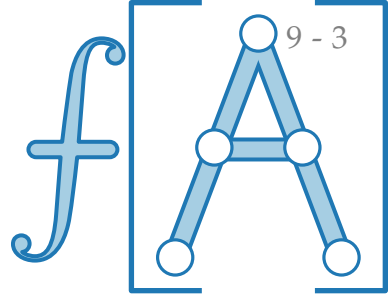
```
GREEDYITERATIVE(int[ ] s, int[ ] e)
```

```
  n = s.length
```

```
  if n = 0 then return  $\emptyset$ 
```

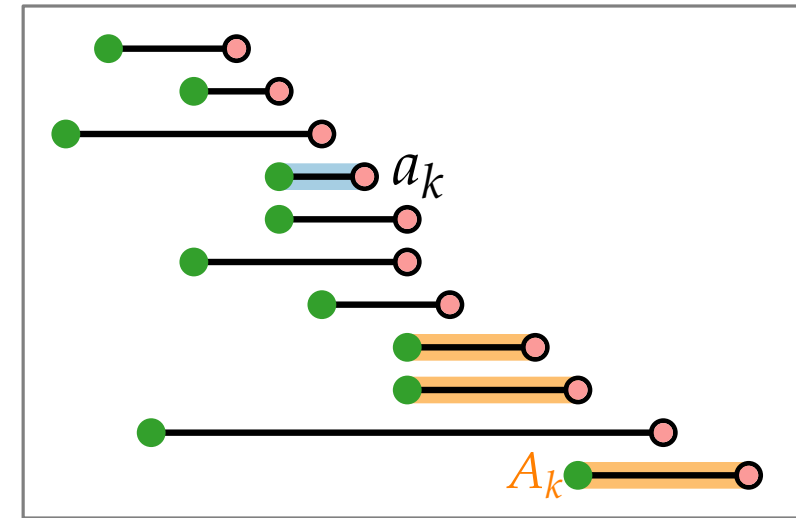
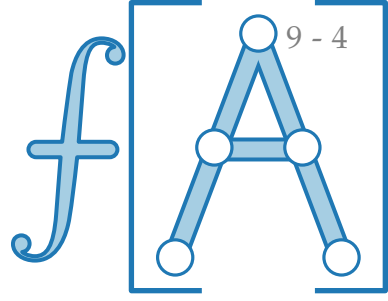
```
  L = {a1}
```

```
  return L
```



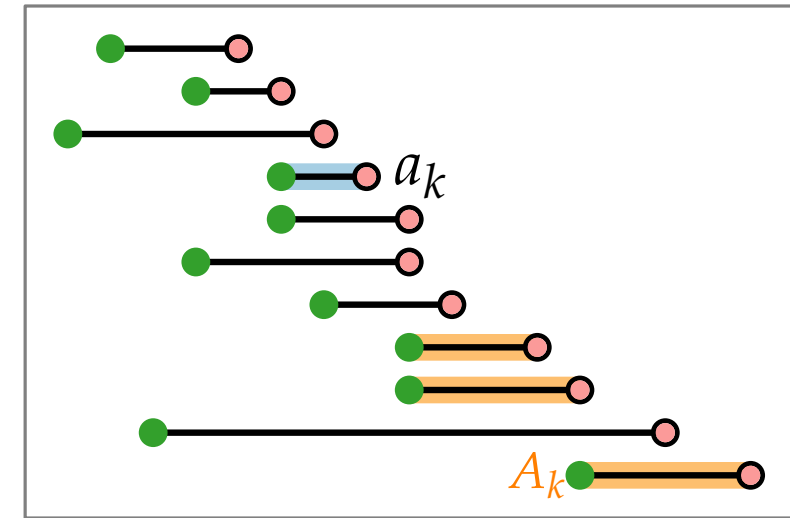
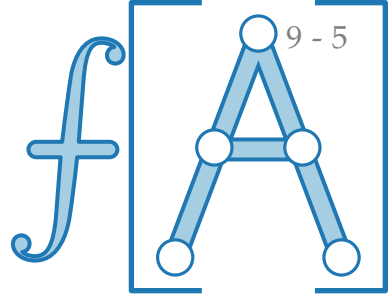
Greedy – iterativ

```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
  n = s.length  
  if n = 0 then return  $\emptyset$   
  L = {a1}  
  k = 1 // höchster Index in L  
  
  return L
```



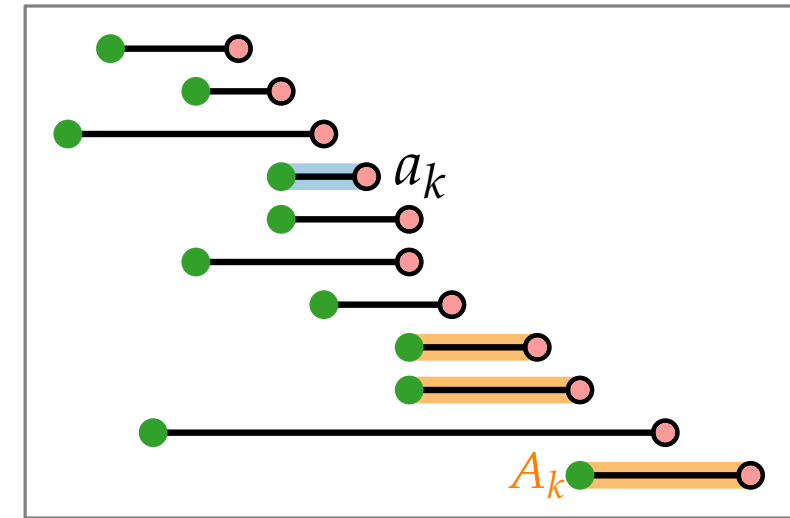
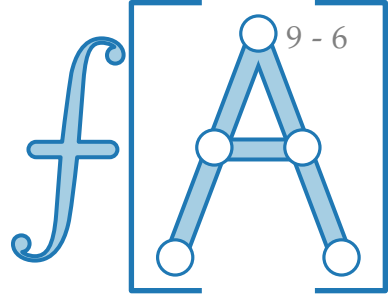
Greedy – iterativ

```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
  n = s.length  
  if n = 0 then return  $\emptyset$   
  L = {a1}  
  k = 1 // höchster Index in L  
  for m = 2 to n do  
    |  
  return L
```



Greedy – iterativ

```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
  n = s.length  
  if n = 0 then return  $\emptyset$   
  L = {a1}  
  k = 1 // höchster Index in L  
  for m = 2 to n do  
    if s[m] ≥ e[k] then  
        
  return L
```



Greedy – iterativ

GREEDYITERATIVE(int[] s , int[] e)

$n = s.length$

if $n = 0$ then return $\emptyset$

$L = \{a_1\}$

$k = 1$ // höchster Index in L

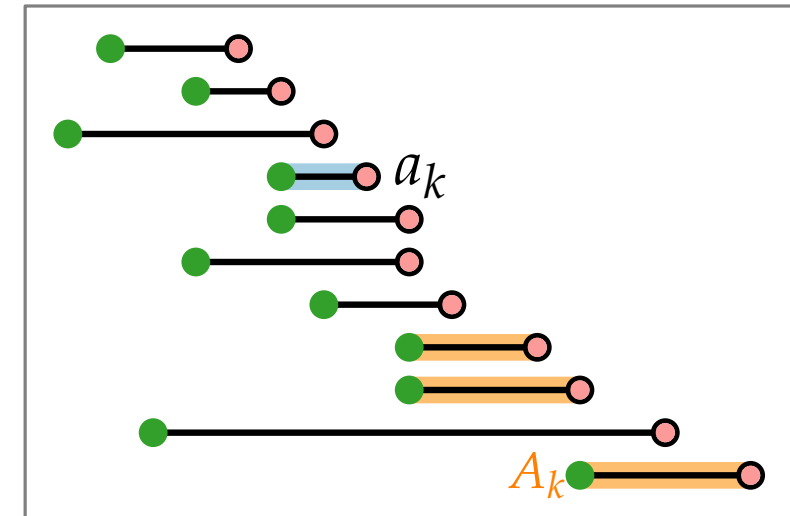
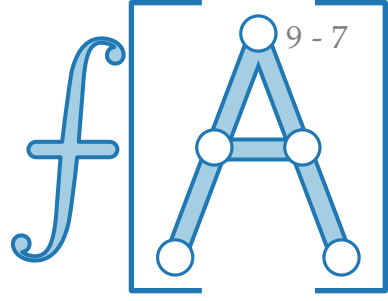
for $m = 2$ to n do

 if $s[m] \geq e[k]$ then

$L = L \cup \{a_m\}$

$k = m$

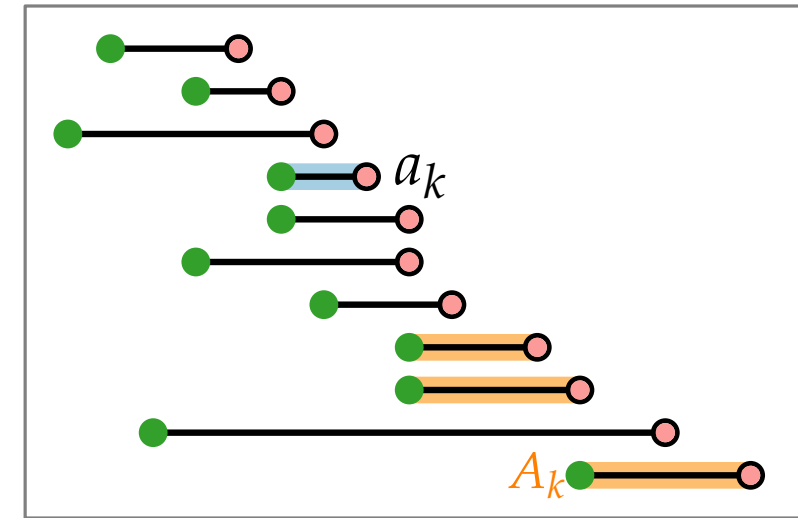
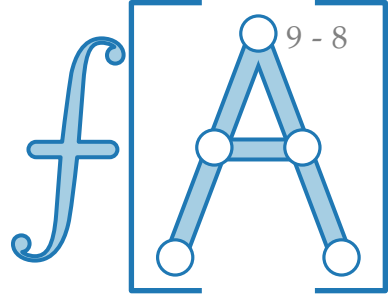
return L



Greedy – iterativ

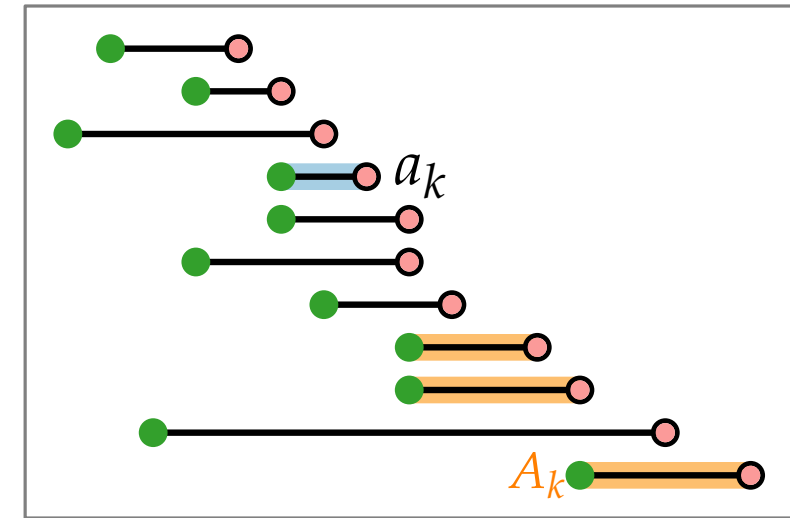
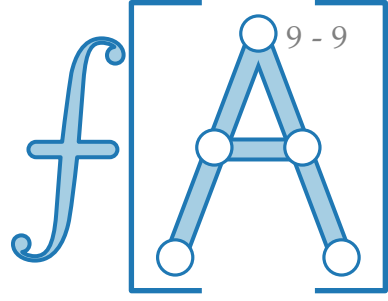
```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
  n = s.length  
  if n = 0 then return  $\emptyset$   
  L = {a1}  
  k = 1 // höchster Index in L  
  for m = 2 to n do  
    if s[m] ≥ e[k] then  
      L = L ∪ {am}  
      k = m  
  return L
```

Laufzeit?



Greedy – iterativ

```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
  n = s.length  
  if n = 0 then return  $\emptyset$   
  L = {a1}  
  k = 1 // höchster Index in L  
  for m = 2 to n do  
    if s[m] ≥ e[k] then  
      L = L ∪ {am}  
      k = m  
  return L
```

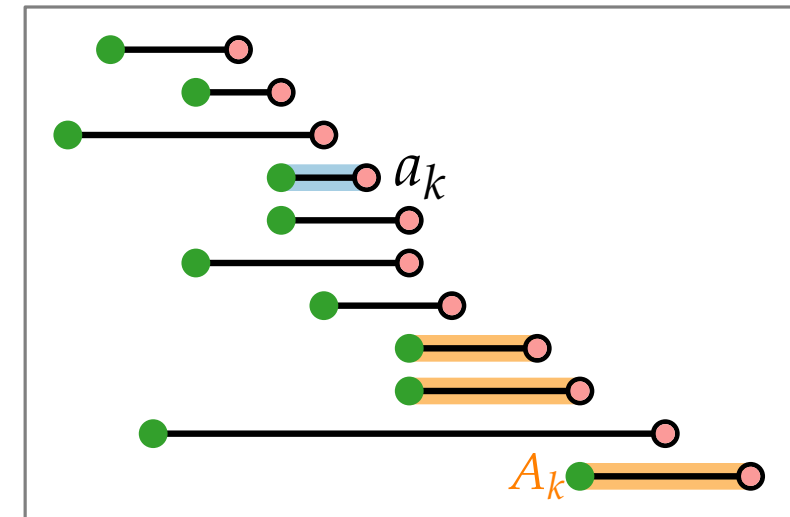
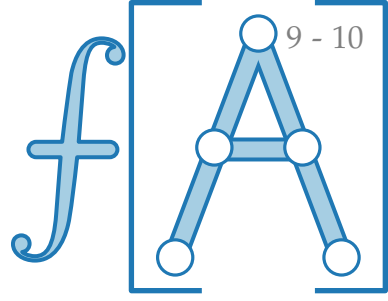


Laufzeit?

GREEDYITERATIVE läuft ebenfalls in $\Theta(n)$ Zeit.

Greedy – iterativ

```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
  n = s.length  
  if n = 0 then return  $\emptyset$   
  L = {a1}  
  k = 1 // höchster Index in L  
  for m = 2 to n do  
    if s[m] ≥ e[k] then  
      L = L ∪ {am}  
      k = m  
  return L
```

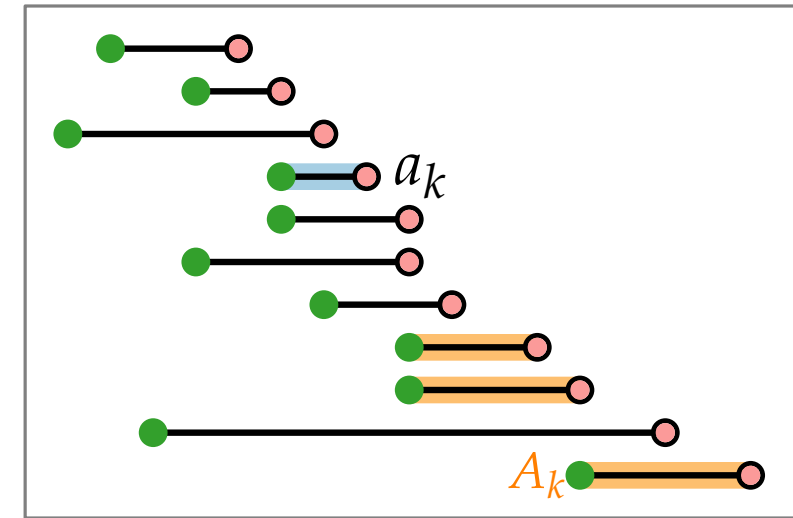
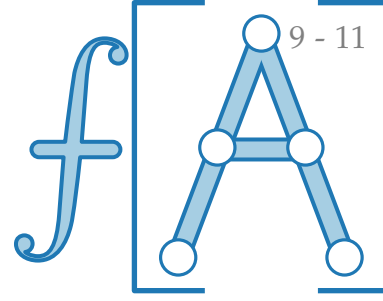


Laufzeit? GREEDYITERATIVE läuft ebenfalls in $\Theta(n)$ Zeit.

Bemerkung. GREEDYITERATIVE berechnet dieselbe optimale Lösung wie GREEDYRECURSIVE

Greedy – iterativ

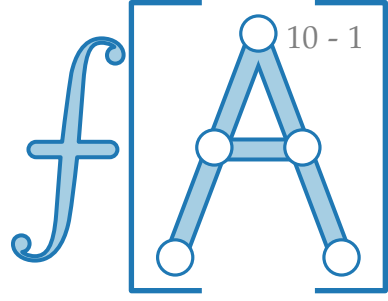
```
GREEDYITERATIVE(int[ ] s, int[ ] e)  
  n = s.length  
  if n = 0 then return  $\emptyset$   
  L = {a1}  
  k = 1 // höchster Index in L  
  for m = 2 to n do  
    if s[m] ≥ e[k] then  
      L = L ∪ {am}  
      k = m  
  return L
```



Laufzeit? GREEDYITERATIVE läuft ebenfalls in $\Theta(n)$ Zeit.

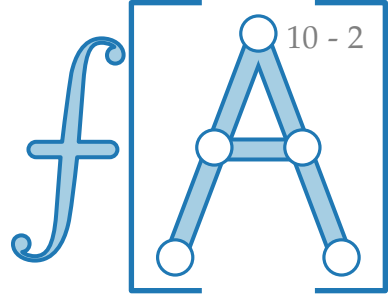
Bemerkung. GREEDYITERATIVE berechnet dieselbe optimale Lösung wie GREEDYRECURSIVE – die „linkeste“.

Die Greedy-Strategie



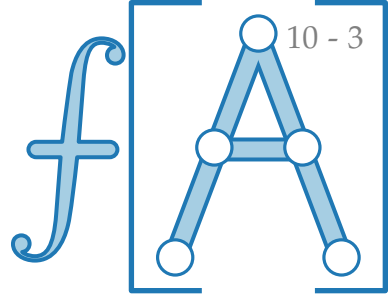
Die Greedy-Strategie

1. Teste, ob das Problem optimale Teilstruktur aufweist.



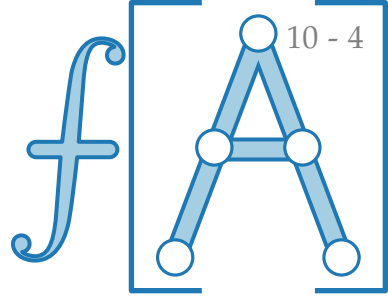
Die Greedy-Strategie

1. Teste, ob das Problem optimale Teilstruktur aufweist.
2. Entwickle eine rekursive Lösung



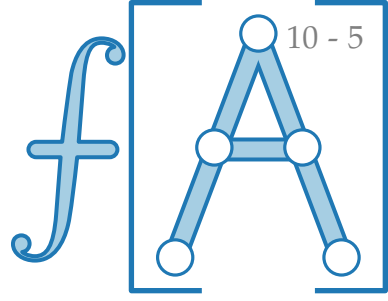
Die Greedy-Strategie

1. Teste, ob das Problem optimale Teilstruktur aufweist.
2. Entwickle eine rekursive Lösung
3. Zeige, dass bei einer Greedy-Entscheidung nur *ein* Teilproblem bleibt



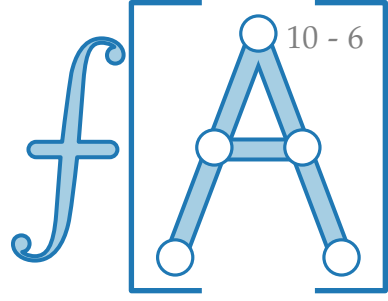
Die Greedy-Strategie

1. Teste, ob das Problem optimale Teilstruktur aufweist.
2. Entwickle eine rekursive Lösung
3. Zeige, dass bei einer Greedy-Entscheidung nur *ein* Teilproblem bleibt
4. Beweise, dass die Greedy-Wahl „sicher“ ist

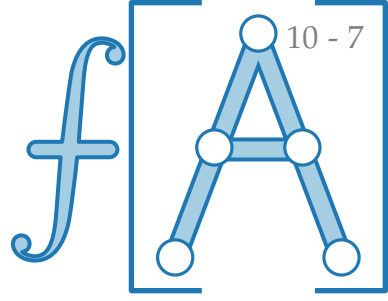


Die Greedy-Strategie

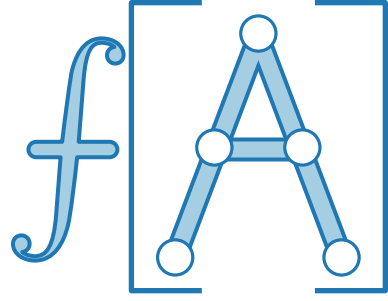
1. Teste, ob das Problem optimale Teilstruktur aufweist.
2. Entwickle eine rekursive Lösung
3. Zeige, dass bei einer Greedy-Entscheidung nur *ein* Teilproblem bleibt
4. Beweise, dass die Greedy-Wahl „sicher“ ist
5. Entwickle einen rekursiven Greedy-Algorithmus



Die Greedy-Strategie



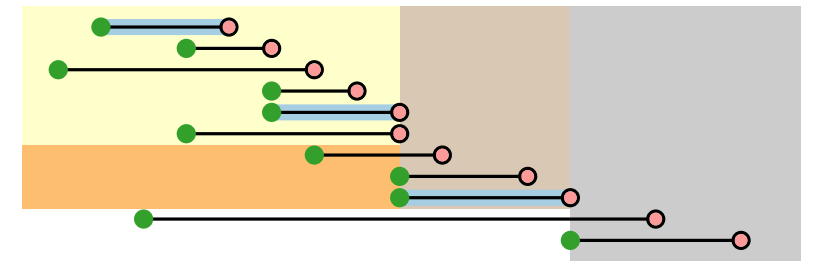
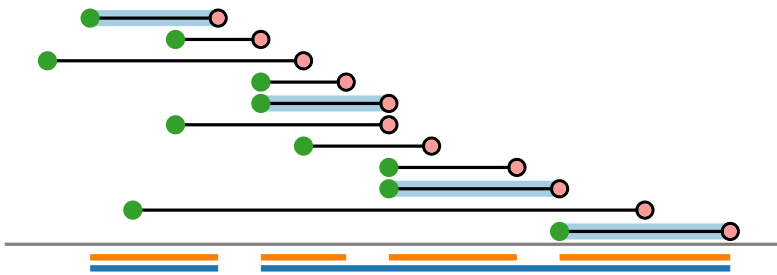
1. Teste, ob das Problem optimale Teilstruktur aufweist.
2. Entwickle eine rekursive Lösung
3. Zeige, dass bei einer Greedy-Entscheidung nur *ein* Teilproblem bleibt
4. Beweise, dass die Greedy-Wahl „sicher“ ist
5. Entwickle einen rekursiven Greedy-Algorithmus
6. Konvertiere den rekursiven in einen iterativen Algorithmus



Fortgeschrittene Algorithmen

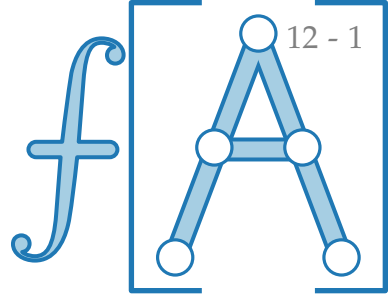
1. Vorlesung Greedy Algorithmen

Teil III: Verallgemeinerungen



Andere Probleme

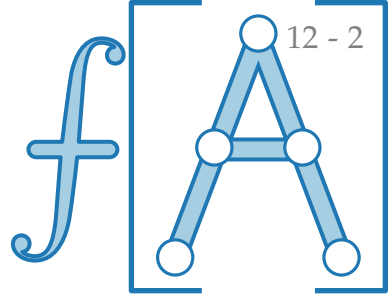
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?



Andere Probleme

1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

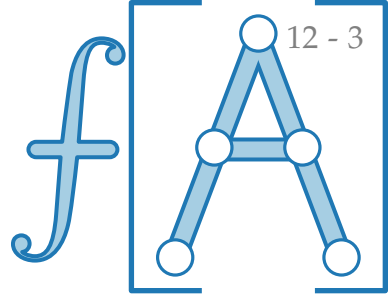


Andere Probleme

1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.

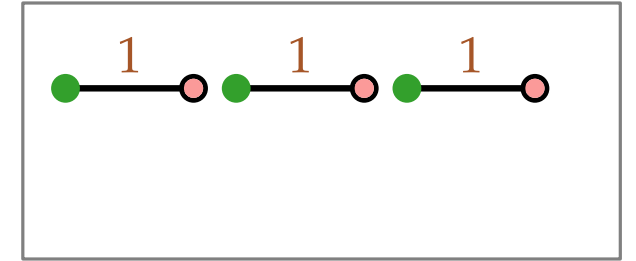
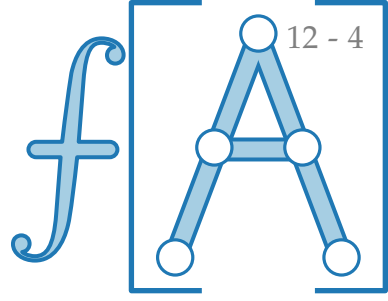


Andere Probleme

1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.

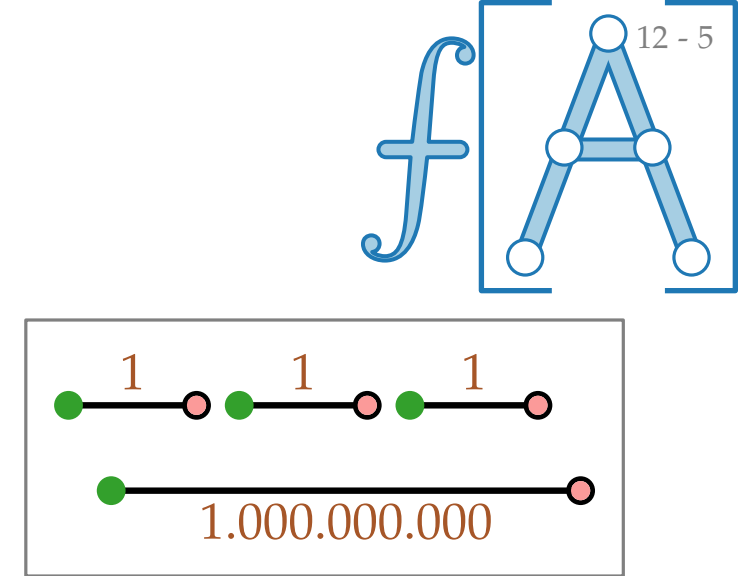


Andere Probleme

1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



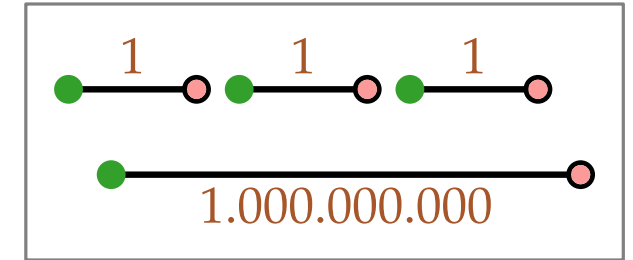
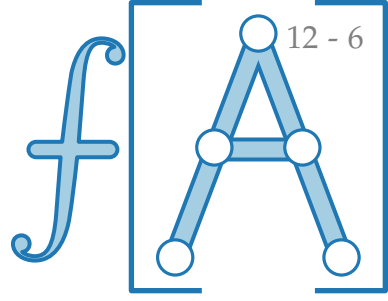
Andere Probleme

1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.

2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Andere Probleme

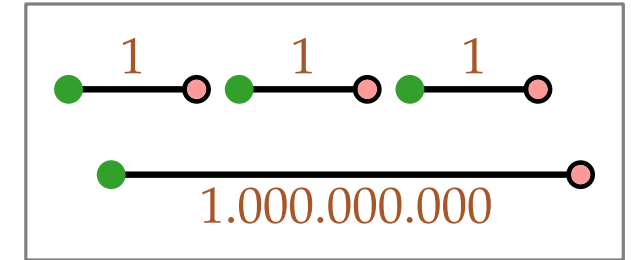
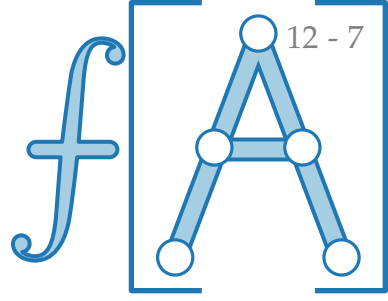
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.

2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:

Problem. MAXIMUMINDEPENDENTSET



Andere Probleme

1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

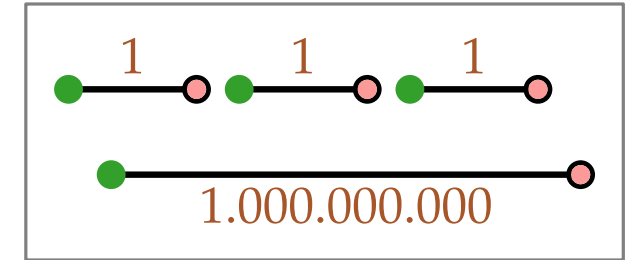
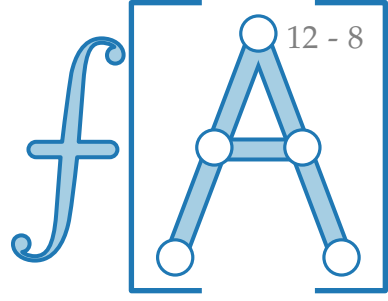
Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.

2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:

Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten



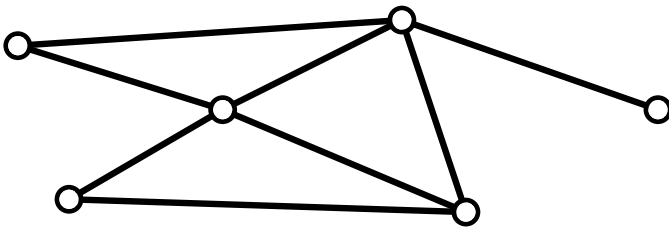
Andere Probleme

1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

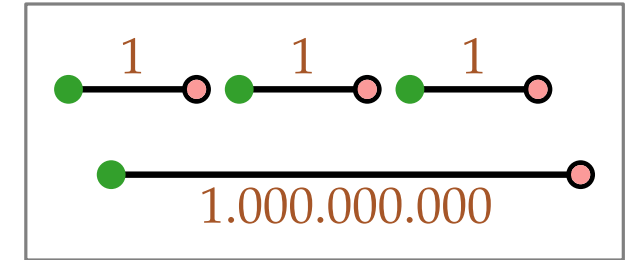
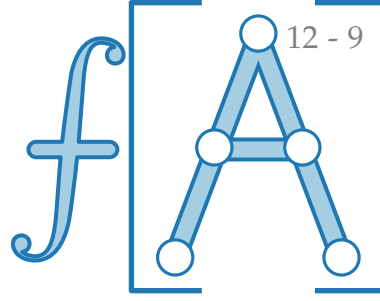
Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.

2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

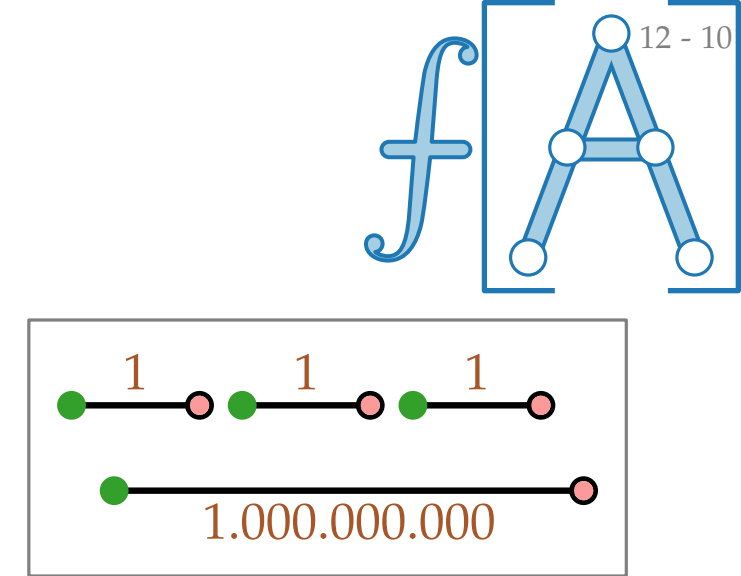


Andere Probleme

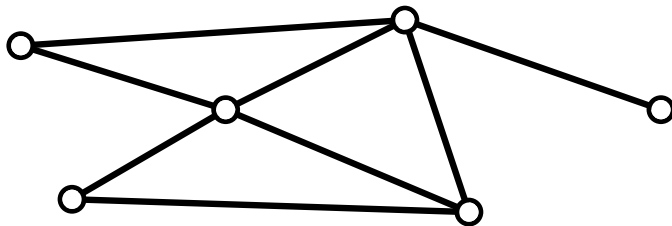
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

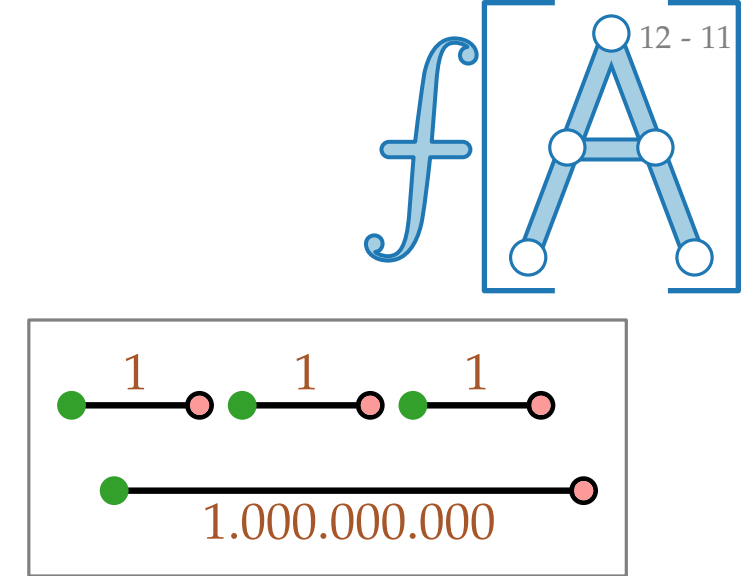
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

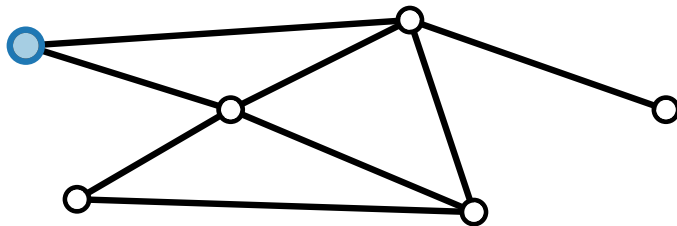
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

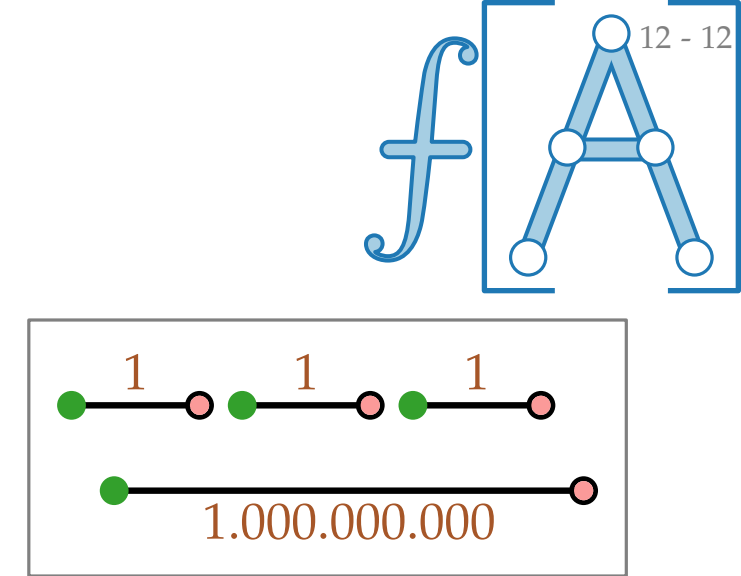
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

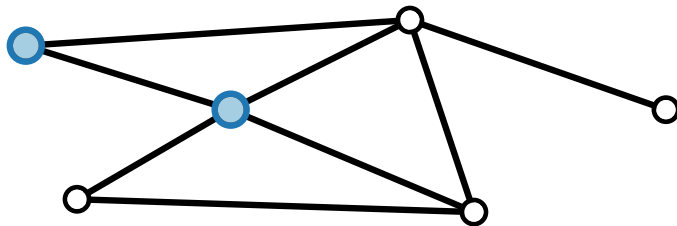
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

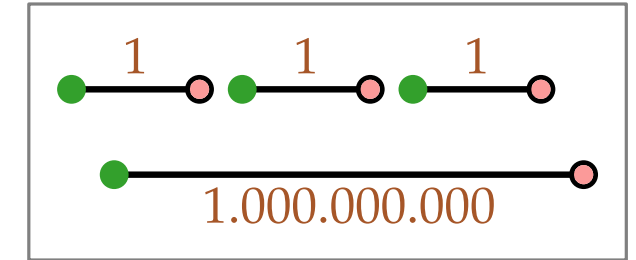
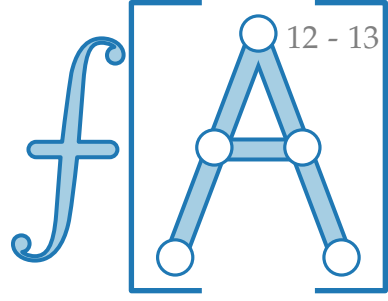
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

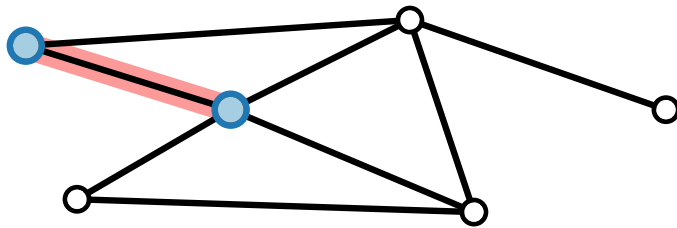
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

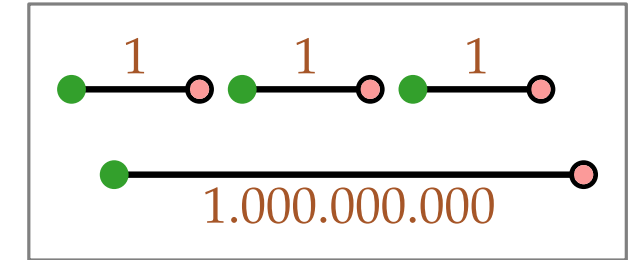
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

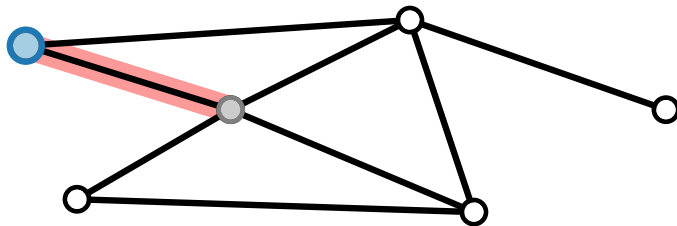
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

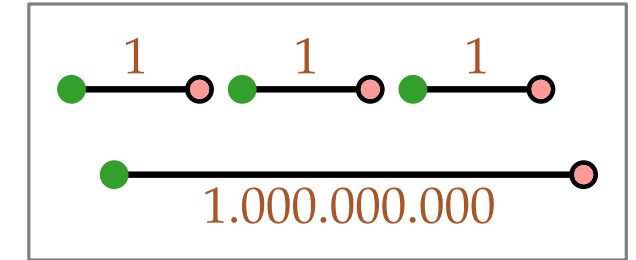
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

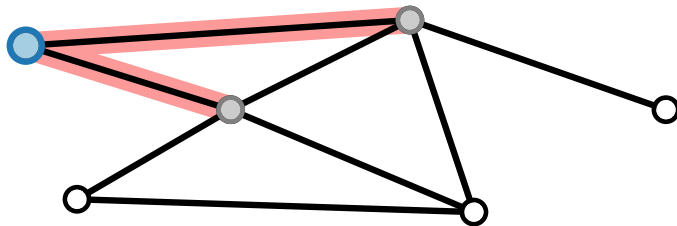
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

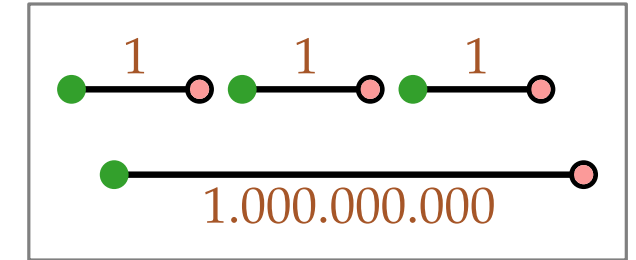
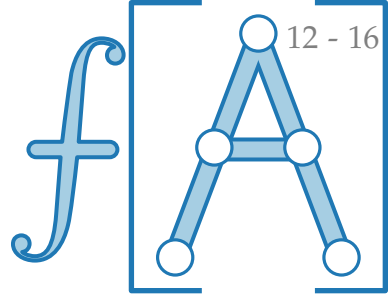
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

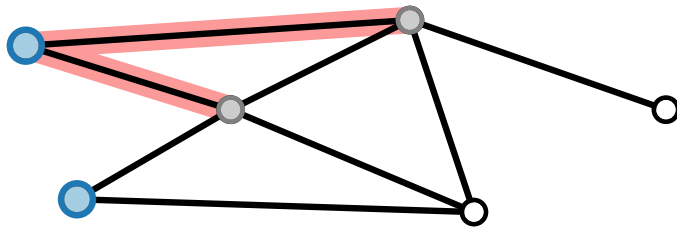
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

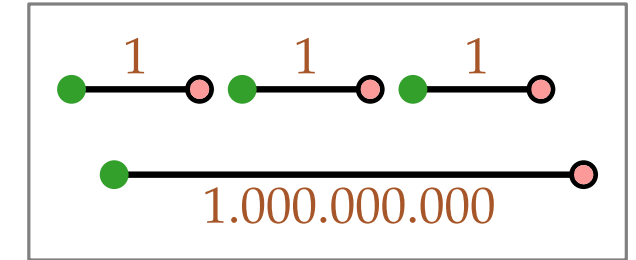
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

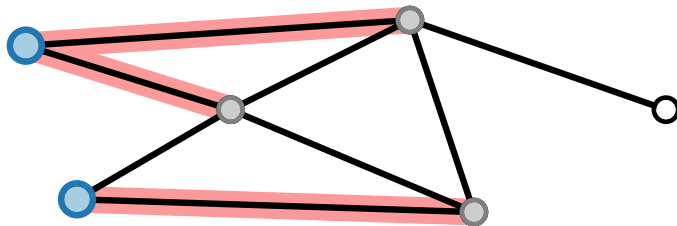
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

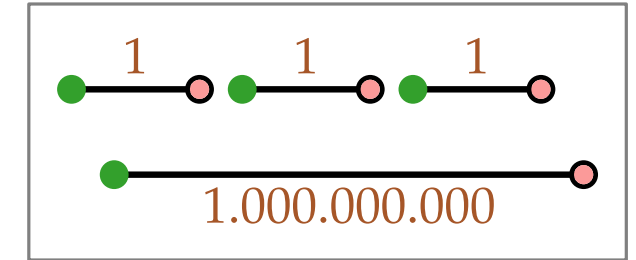
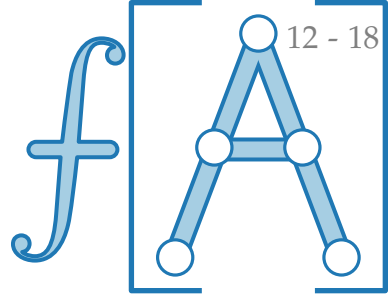
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

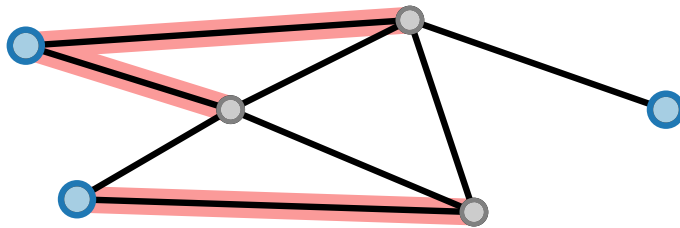
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

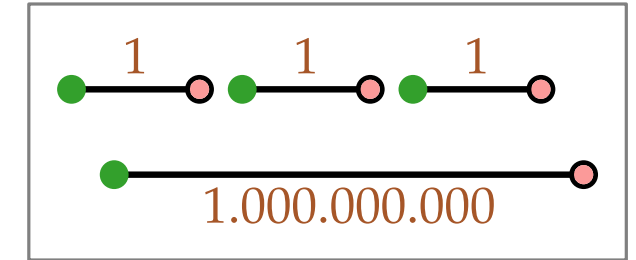
Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

Andere Probleme

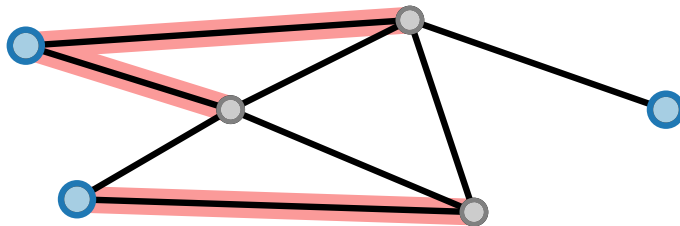
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

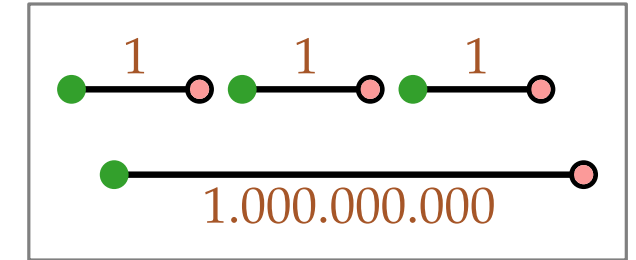
- Was hat MIS mit unserem Ablaufplanungsproblem zu tun?

Andere Probleme

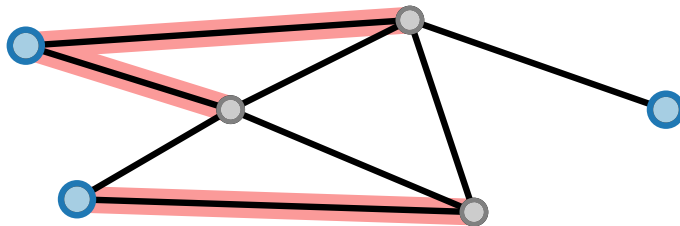
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

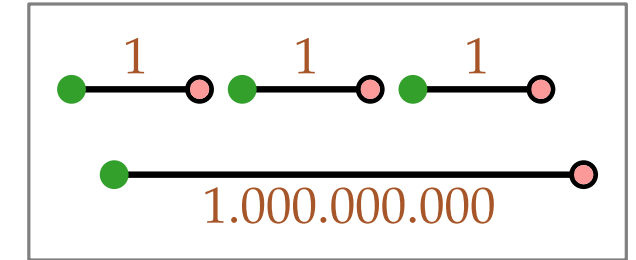
- Was hat MIS mit unserem Ablaufplanungsproblem zu tun?
- Welche Graphen kommen bei der Ablaufplanung nicht vor?

Andere Probleme

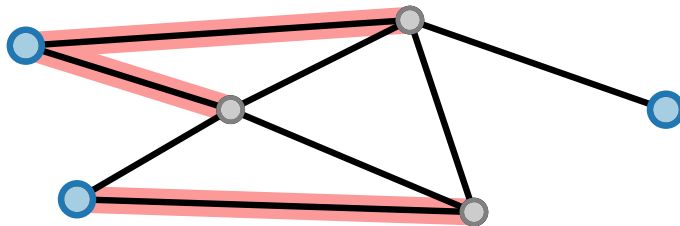
1. Welches allgemeinere Ablaufproblem kann der Greedy-Algorithmus (GA) nicht lösen?

Wenn jede Aktivität $a \in A$ ihren eigenen Ertrag $w(a)$ erbringt:

Finde $L \subseteq A$ mit L kompatibel und $w(L) := \sum_{a \in L} w(a)$ max.



2. Problem **größte unabhängige Menge (MIS – Max. Independent Set)** in Graphen:



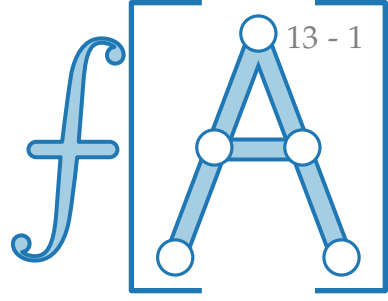
Problem. MAXIMUMINDEPENDENTSET

Geg. Graph $G = (V, E)$ mit n Knoten

Ges. Größte **unabhängige** Menge, also eine größte Menge $U \subseteq V$, so dass kein Knotenpaar in U adjazent in G ist.

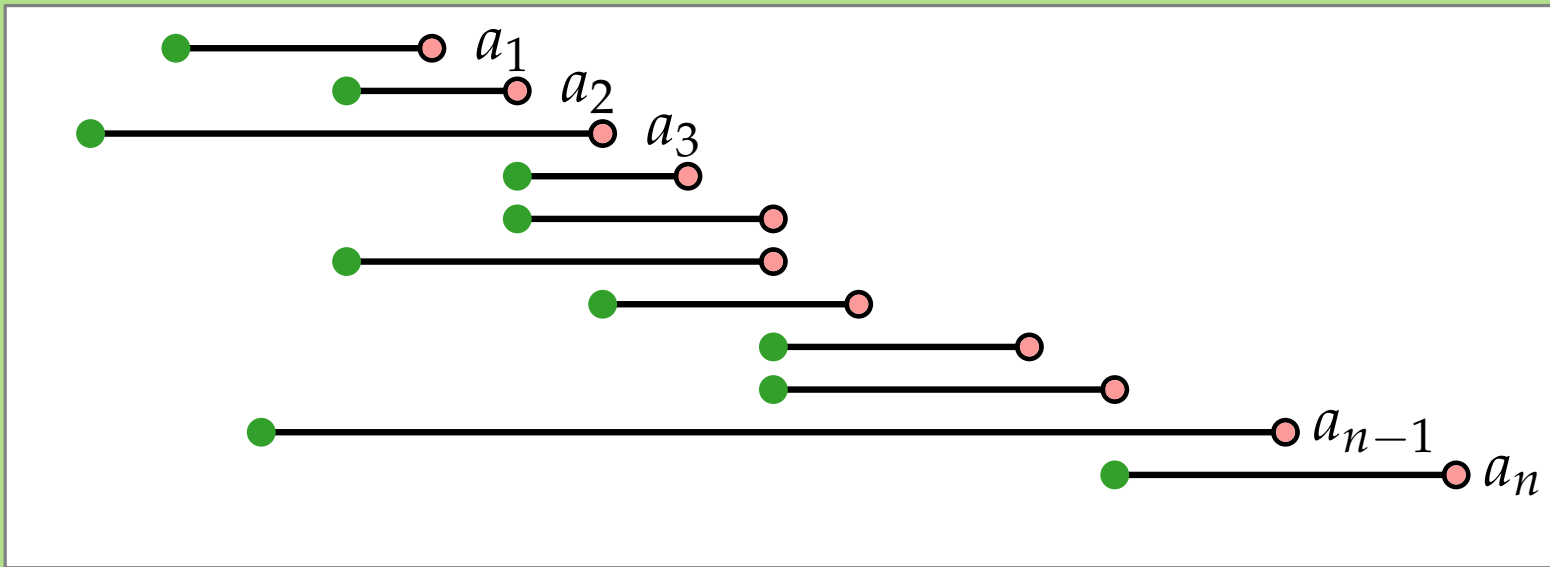
- Was hat MIS mit unserem Ablaufplanungsproblem zu tun?
- Welche Graphen kommen bei der Ablaufplanung nicht vor?
- Kann man MIS mit Dynam. Progr. (DP) oder GA lösen?

Ein ähnliches Problem der Ablaufplanung

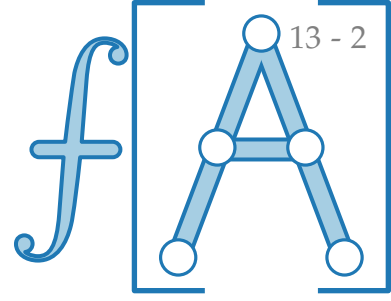


Problem. ABLAUFPLANUNG'

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**,
wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

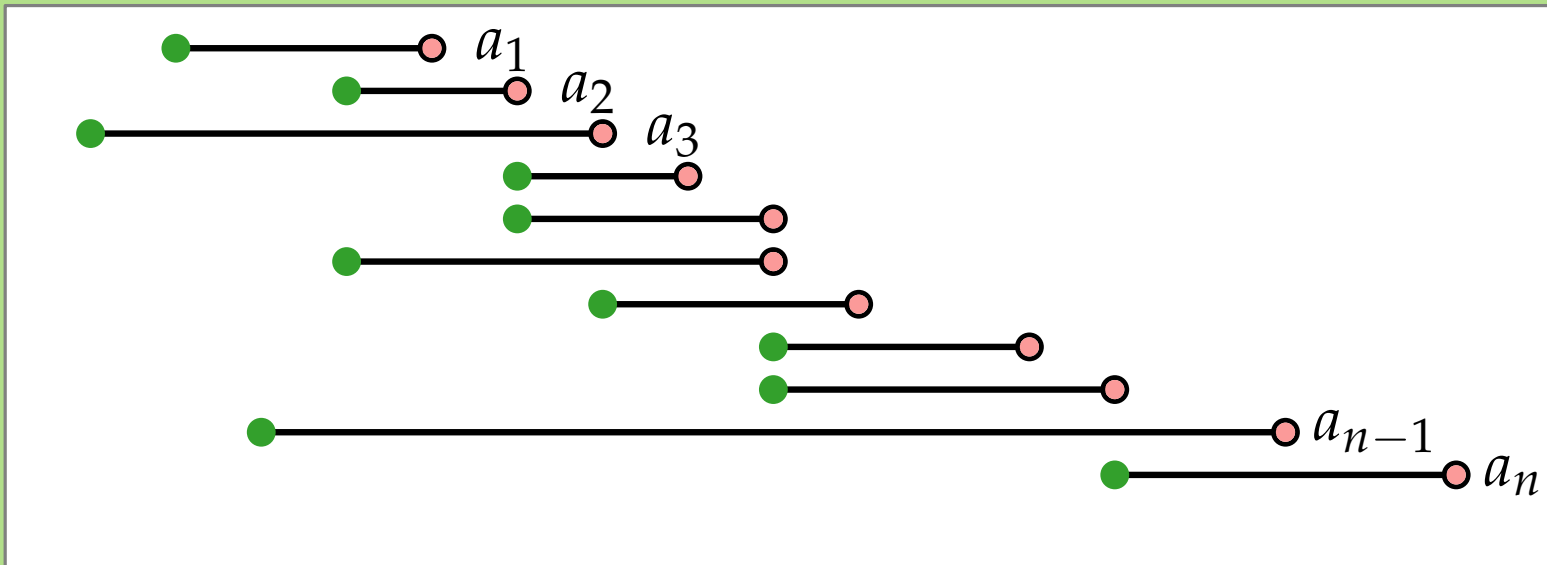


Ein ähnliches Problem der Ablaufplanung



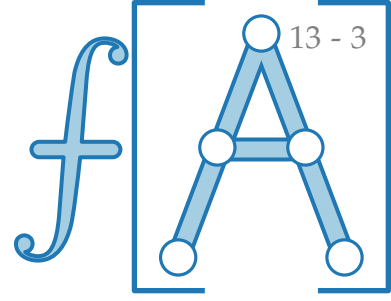
Problem. ABLAUFPLANUNG'

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



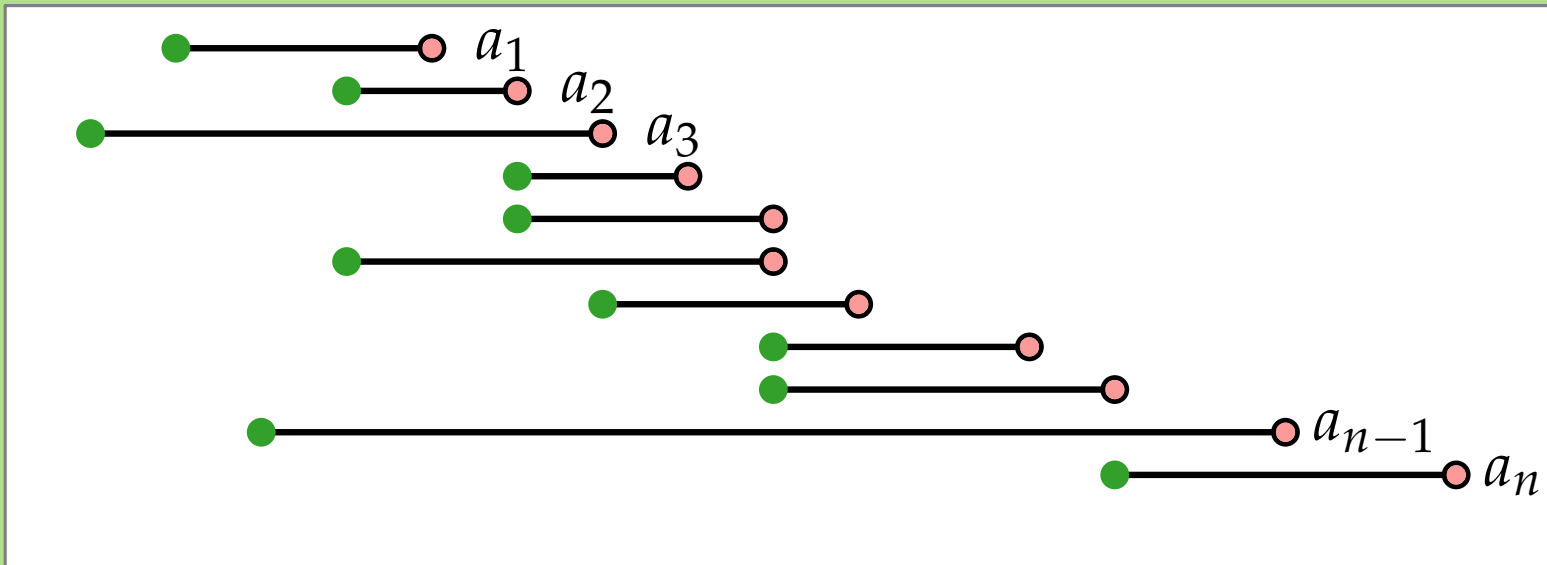
Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

Ein ähnliches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG'

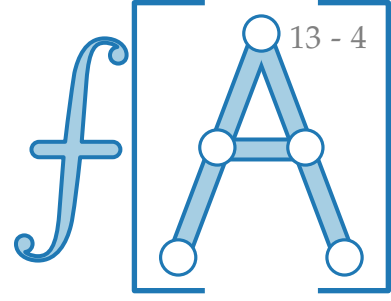
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

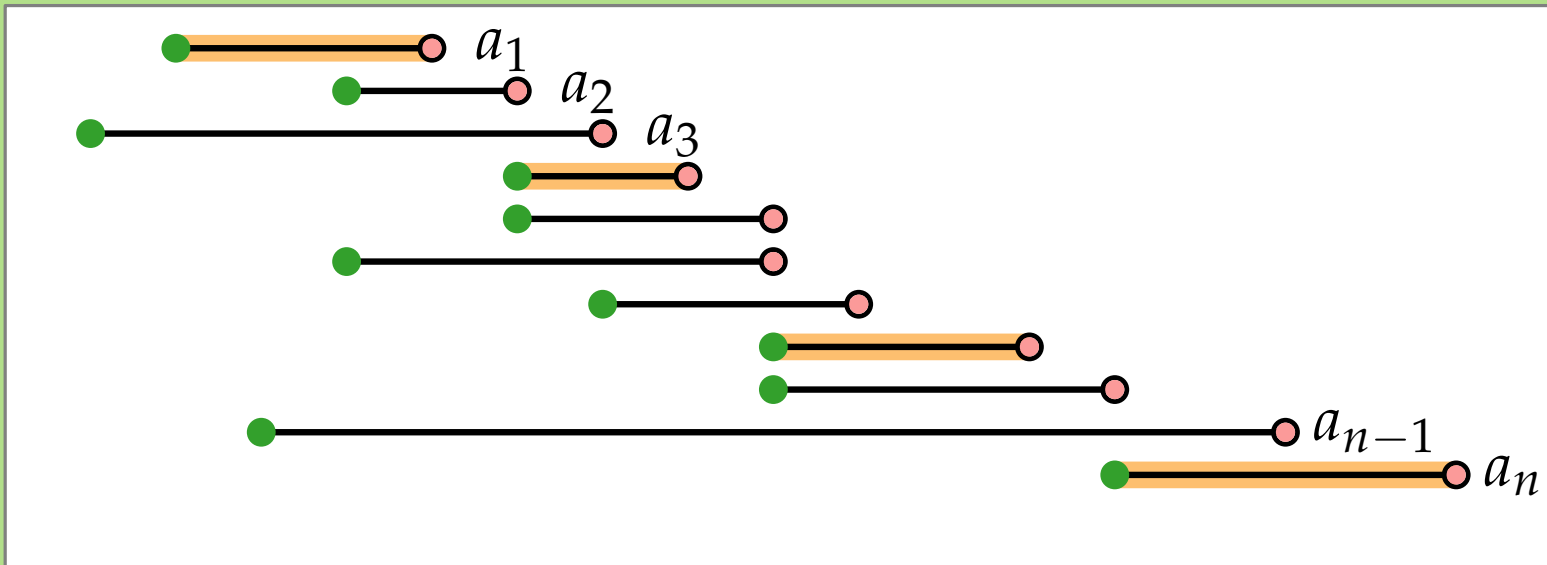
Grund. Intervalle $\hat{=}$ Prozesse, die die gleiche Ressource nutzen; der Gesamtertrag ist proportional zur Auslastung.

Ein ähnliches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG'

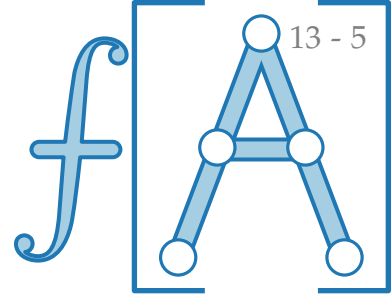
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

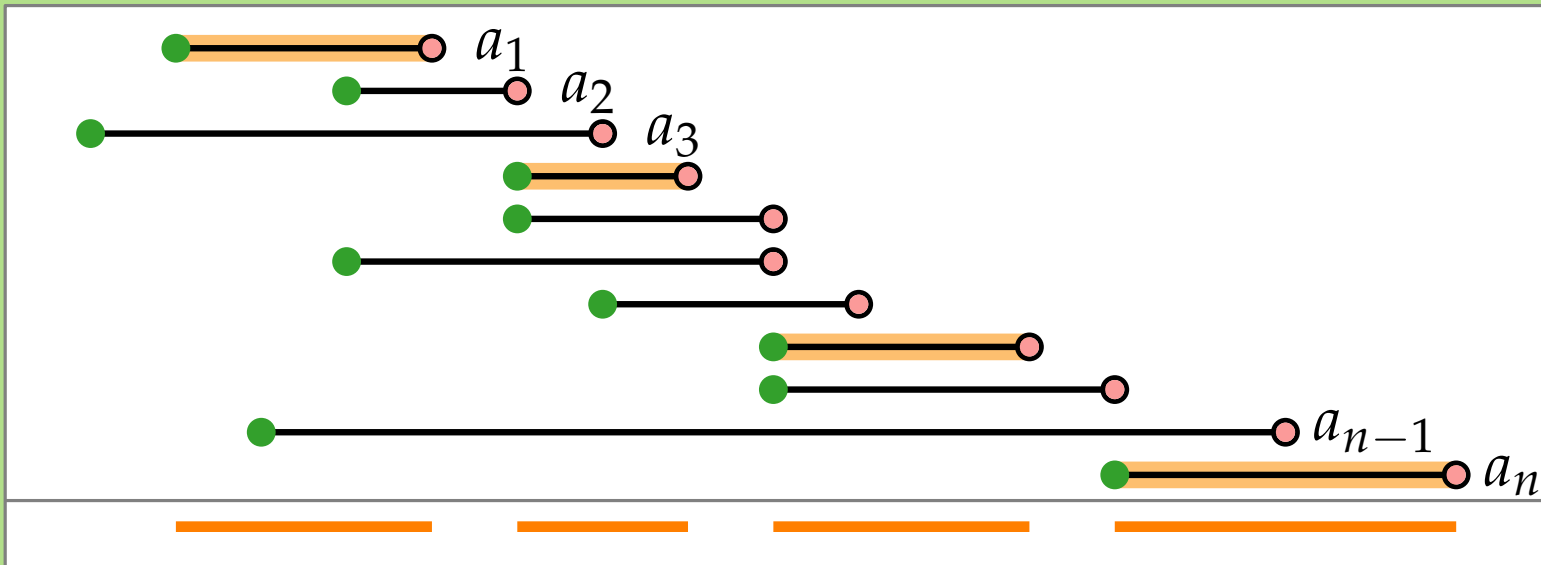
Grund. Intervalle $\hat{=}$ Prozesse, die die gleiche Ressource nutzen; der Gesamtertrag ist proportional zur Auslastung.

Ein ähnliches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG'

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



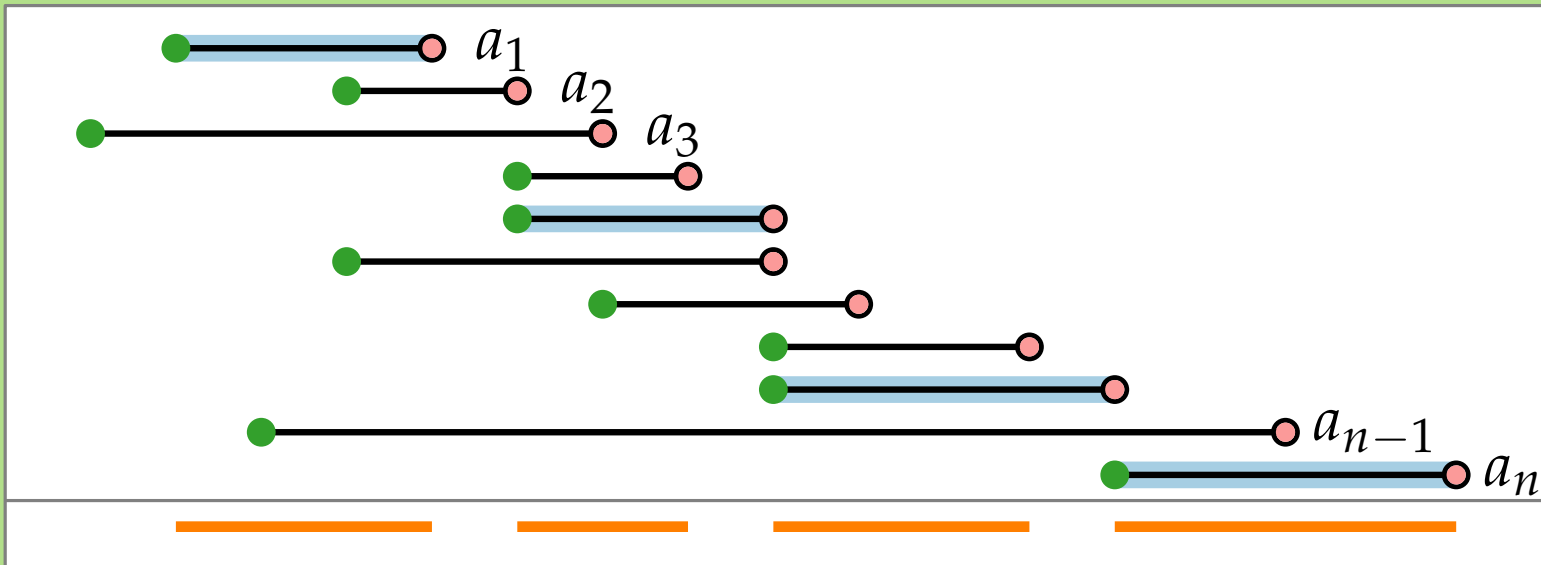
Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

Grund. Intervalle $\hat{=}$ Prozesse, die die gleiche Ressource nutzen; der Gesamtertrag ist proportional zur Auslastung.

Ein ähnliches Problem der Ablaufplanung

Problem. ABLAUFPLANUNG'

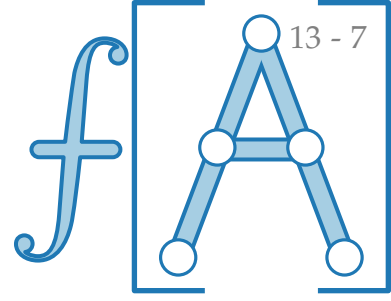
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

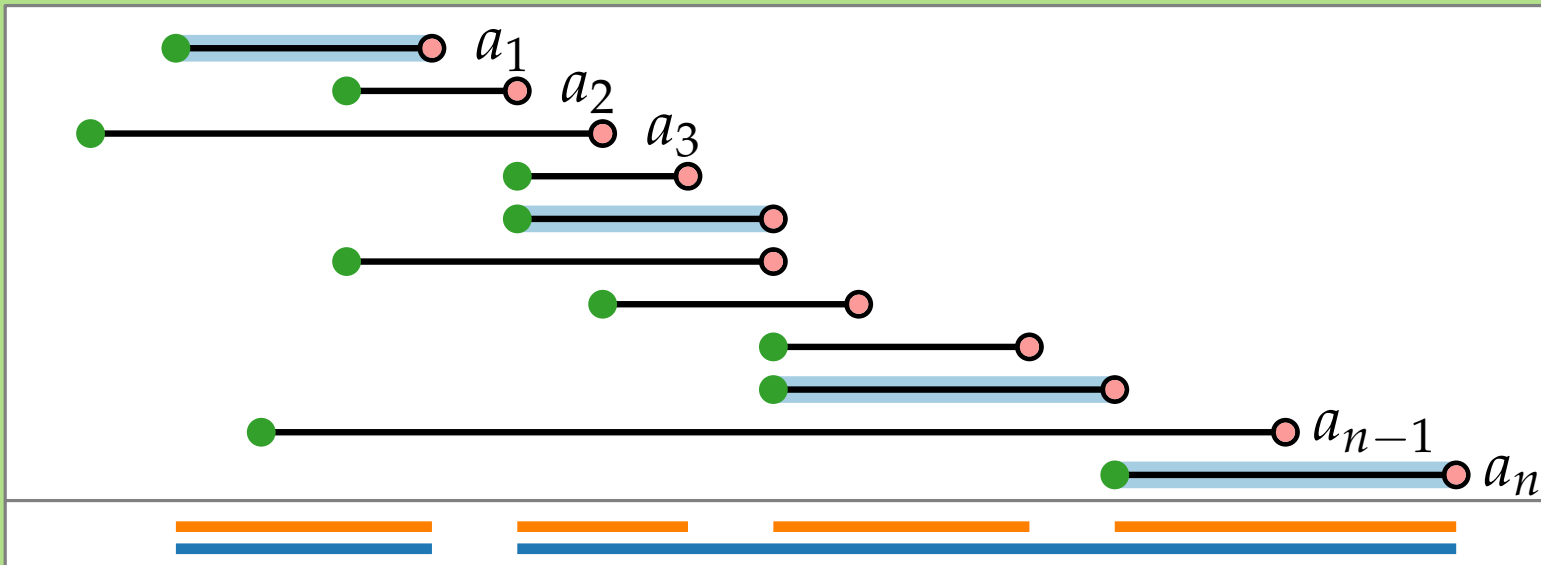
Grund. Intervalle $\hat{=}$ Prozesse, die die gleiche Ressource nutzen; der Gesamtertrag ist proportional zur Auslastung.

Ein ähnliches Problem der Ablaufplanung



Problem. ABLAUFPLANUNG'

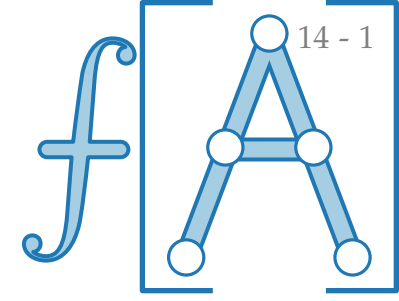
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

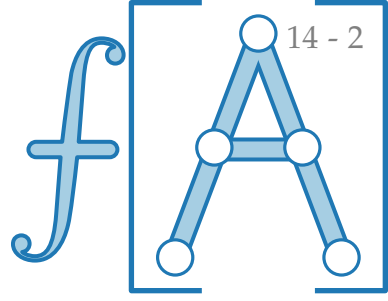
Grund. Intervalle $\hat{=}$ Prozesse, die die gleiche Ressource nutzen; der Gesamtertrag ist proportional zur Auslastung.

Greedy?



Greedy?

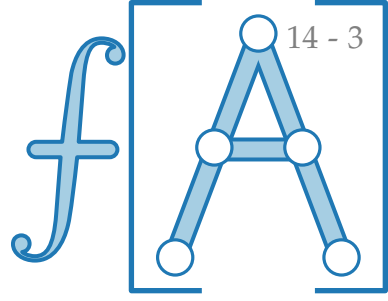
- 1. Versuch.** Nimm Aktivität mit frühestem Endtermin,
streiche dazu inkompatible Aktivitäten und iteriere.



Greedy?

1. Versuch. Nimm Aktivität mit frühestem Endtermin,
streiche dazu inkompatible Aktivitäten und iteriere.

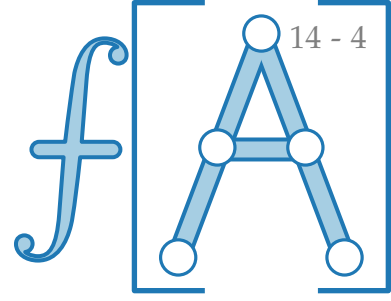
Gegenbsp.



Greedy?

1. Versuch. Nimm Aktivität mit frühestem Endtermin,
streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.



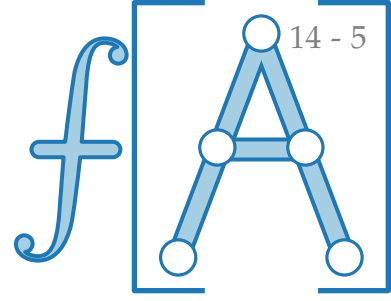
Greedy?

1. Versuch. Nimm Aktivität mit frühestem Endtermin,
streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.



Ergebnis kann **beliebig schlecht** werden.



Greedy?

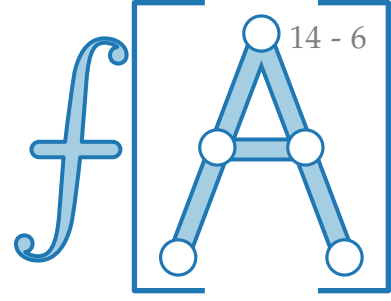
1. Versuch. Nimm Aktivität mit frühestem Endtermin,
streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.



Ergebnis kann **beliebig schlecht** werden.

2. Versuch.



Greedy?

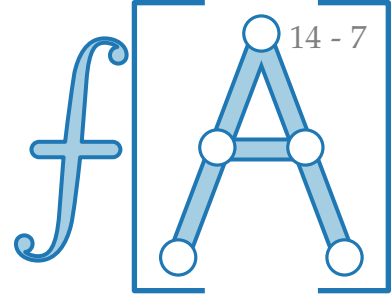
1. Versuch. Nimm Aktivität mit frühestem Endtermin,
streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.

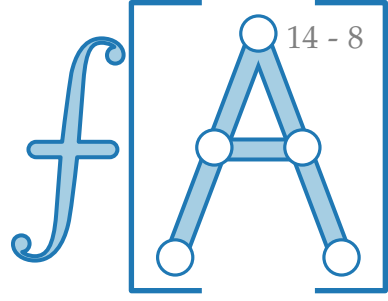


Ergebnis kann **beliebig schlecht** werden.

2. Versuch. Nimm längste Aktivität,
streiche dazu inkompatible Aktivitäten und iteriere.

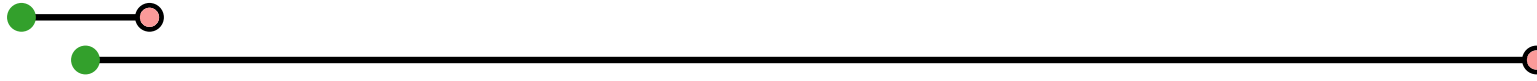


Greedy?



1. Versuch. Nimm Aktivität mit frühestem Endtermin,
streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.

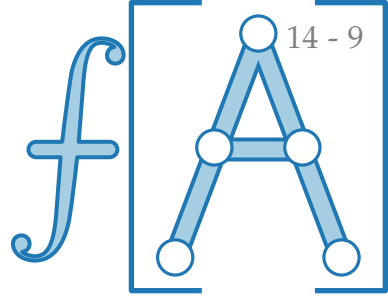


Ergebnis kann **beliebig schlecht** werden.

2. Versuch. Nimm längste Aktivität,
streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.

Greedy?

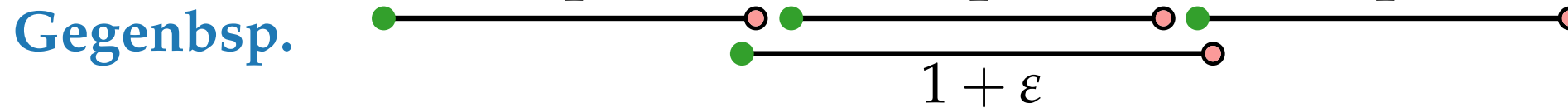


1. Versuch. Nimm Aktivität mit frühestem Endtermin, streiche dazu inkompatible Aktivitäten und iteriere.

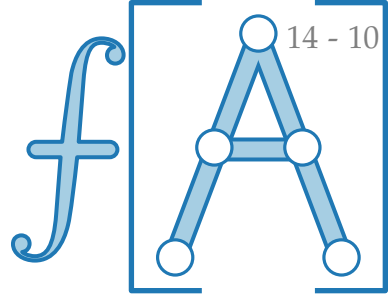


Ergebnis kann **beliebig schlecht** werden.

2. Versuch. Nimm längste Aktivität, streiche dazu inkompatible Aktivitäten und iteriere.



Greedy?

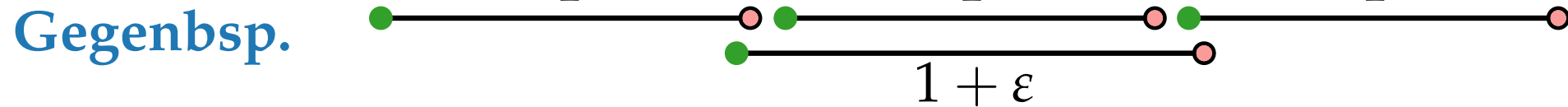


1. Versuch. Nimm Aktivität mit frühestem Endtermin, streiche dazu inkompatible Aktivitäten und iteriere.



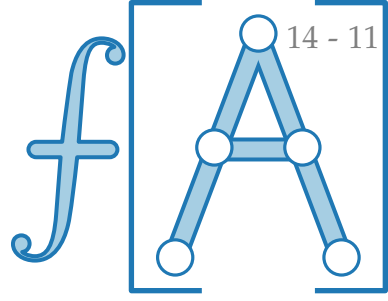
Ergebnis kann **beliebig schlecht** werden.

2. Versuch. Nimm längste Aktivität, streiche dazu inkompatible Aktivitäten und iteriere.



Ergebnis kann **3 mal so schlecht** werden.

Greedy?



1. Versuch. Nimm Aktivität mit frühestem Endtermin, streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.

Ergebnis kann **beliebig schlecht** werden.

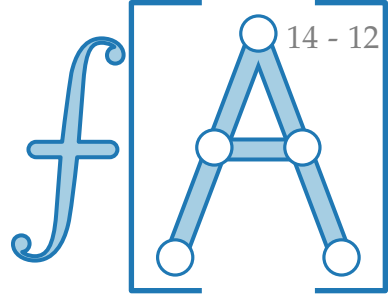
2. Versuch. Nimm längste Aktivität, streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp.

Ergebnis kann **3 mal so schlecht** werden.

Ergebnis wird höchstens 3 mal so schlecht. (Beweis ein anderes Mal)

Greedy?

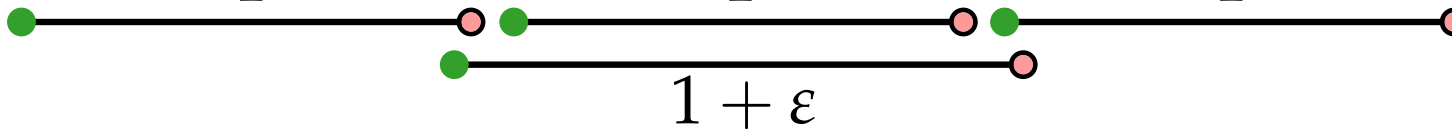


1. Versuch. Nimm Aktivität mit frühestem Endtermin, streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp. 

Ergebnis kann **beliebig schlecht** werden.

2. Versuch. Nimm längste Aktivität, streiche dazu inkompatible Aktivitäten und iteriere.

Gegenbsp. 

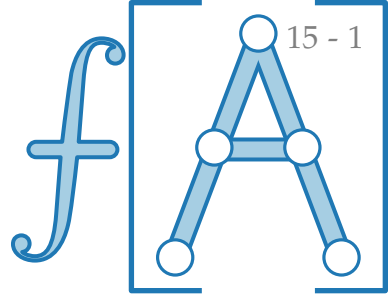
Ergebnis kann **3 mal so schlecht** werden.

Ergebnis wird höchstens 3 mal so schlecht. (Beweis ein anderes Mal)

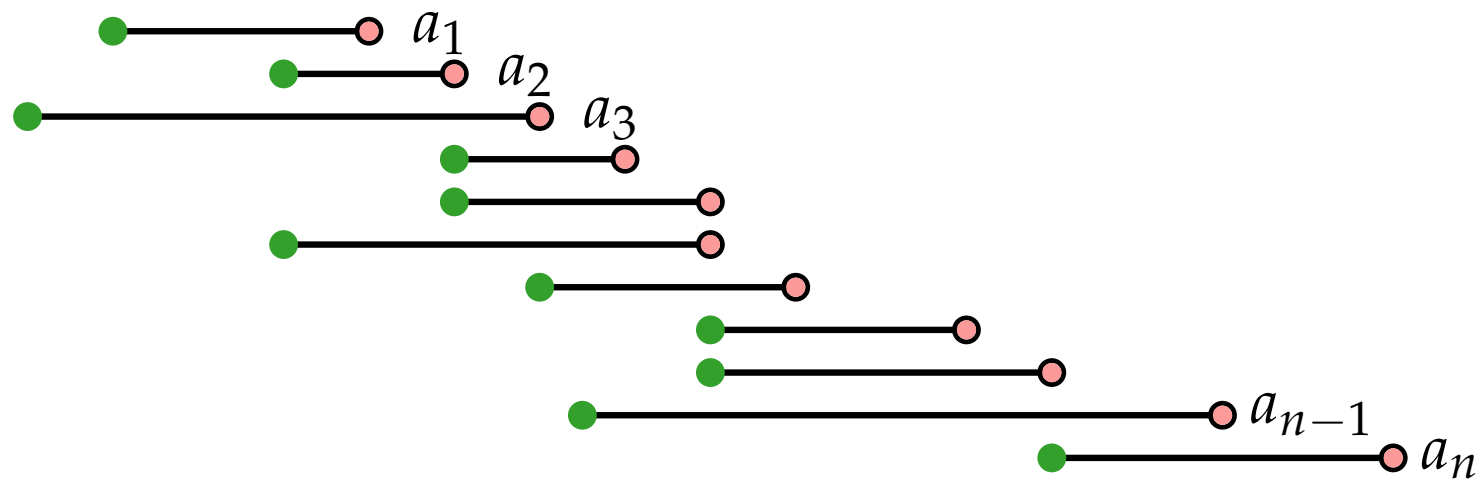
Frage.

Wie können wir den 2. GA in $\mathcal{O}(n \log n)$ Zeit implementieren?

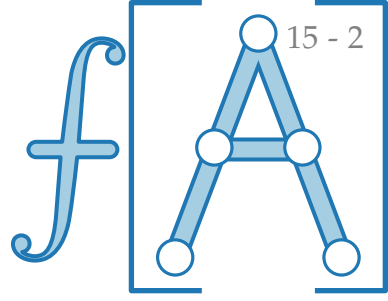
2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

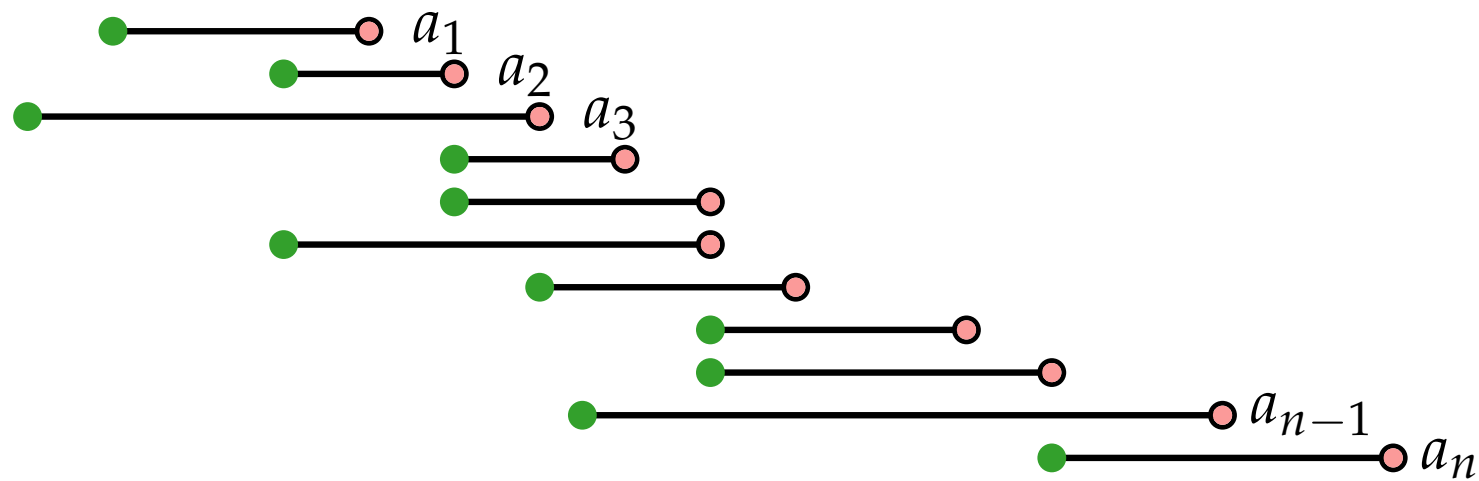


2. GA in $\mathcal{O}(n \log n)$ Zeit

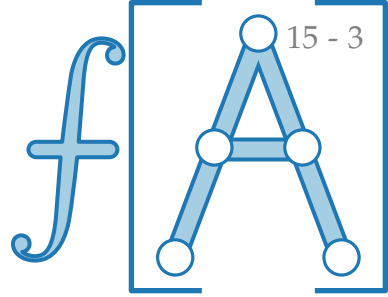


ZWEITERGA(int[] a)

Sortiere a nach Länge



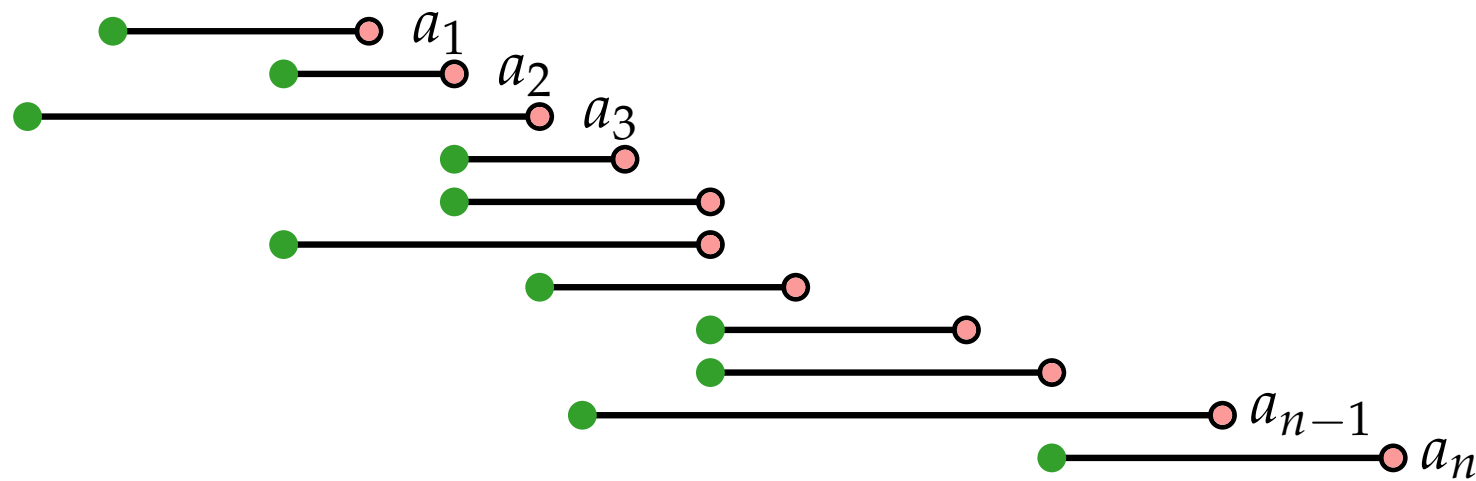
2. GA in $\mathcal{O}(n \log n)$ Zeit



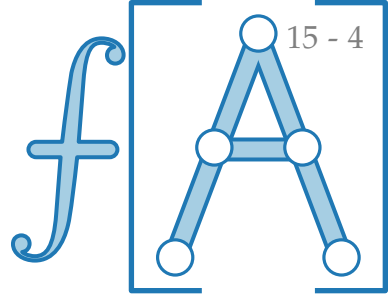
ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$



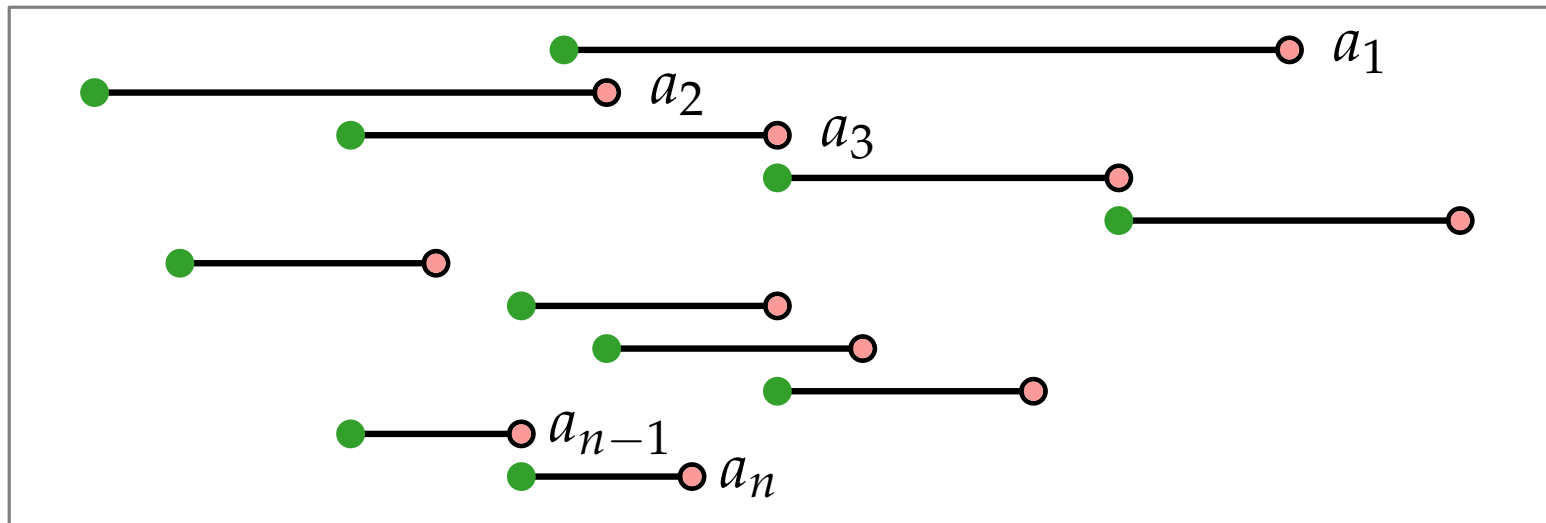
2. GA in $\mathcal{O}(n \log n)$ Zeit



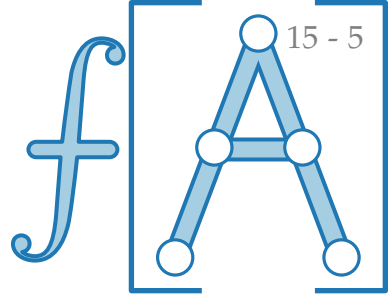
ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$



2. GA in $\mathcal{O}(n \log n)$ Zeit

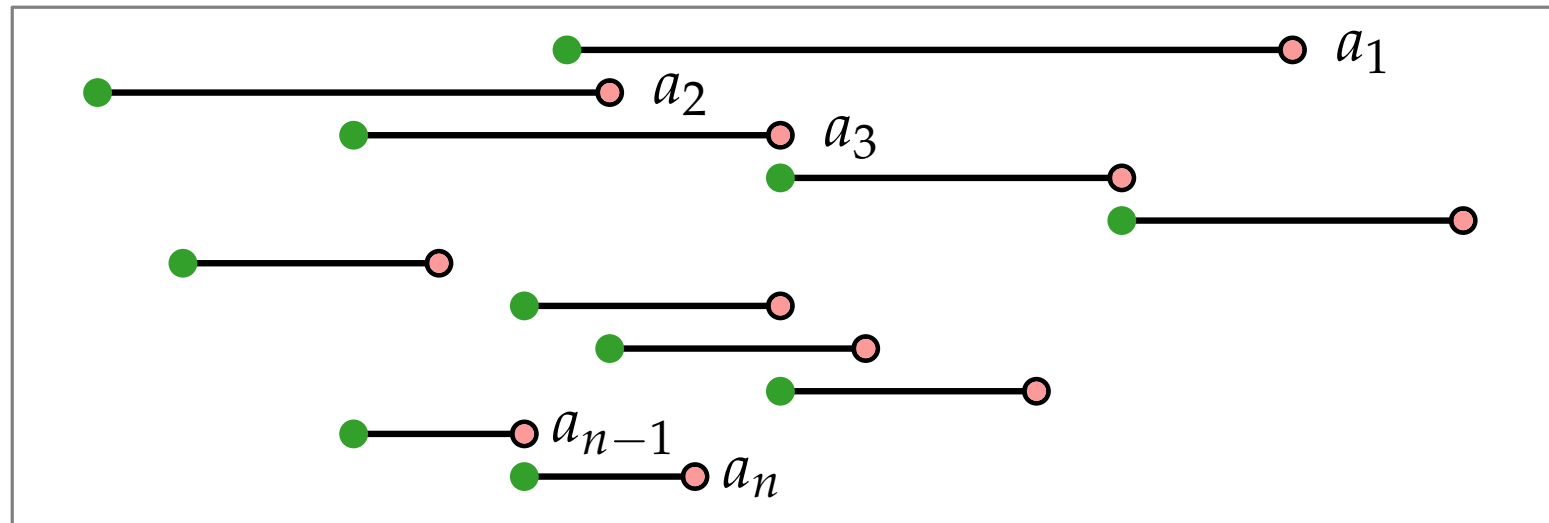


ZWEITERGA(int[] a)

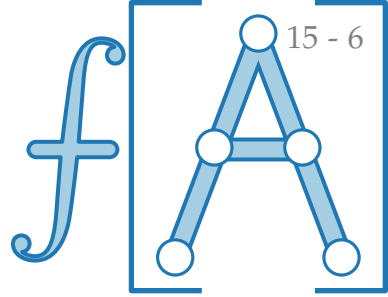
Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.



2. GA in $\mathcal{O}(n \log n)$ Zeit



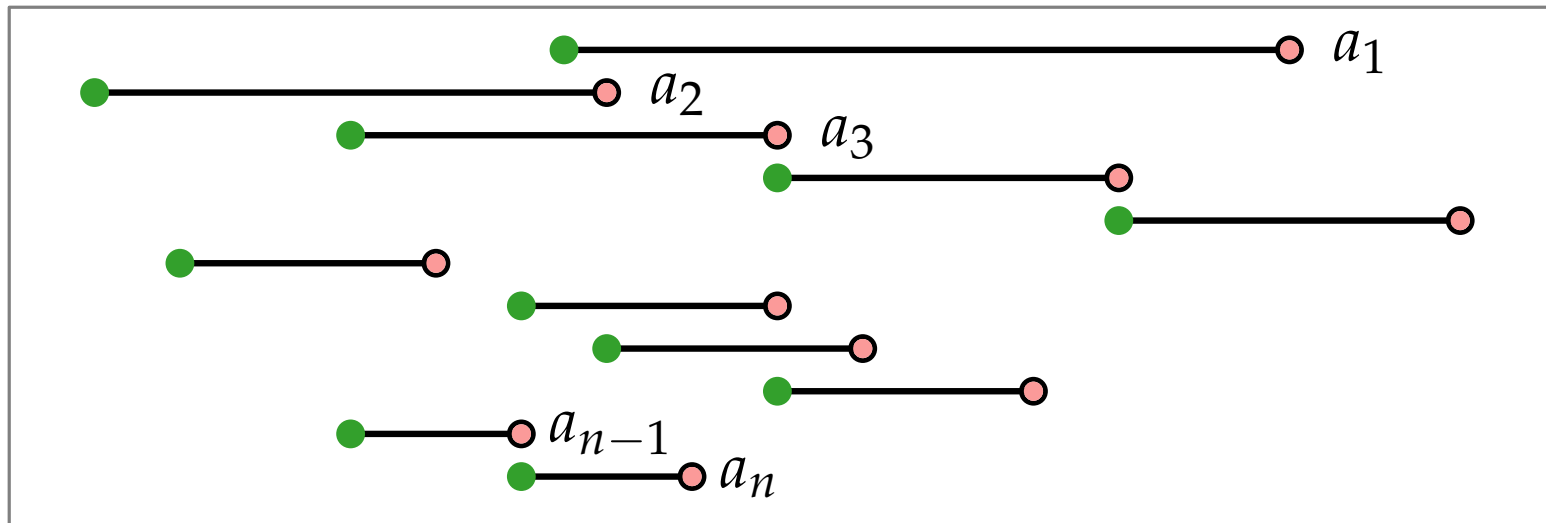
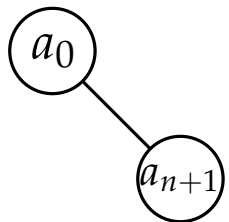
ZWEITERGA(int[] a)

Sortiere a nach Länge

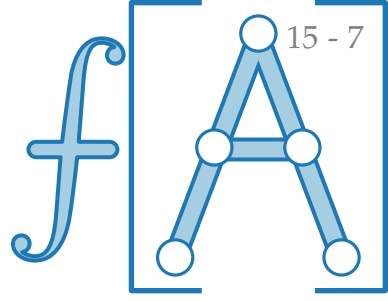
// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

T :



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

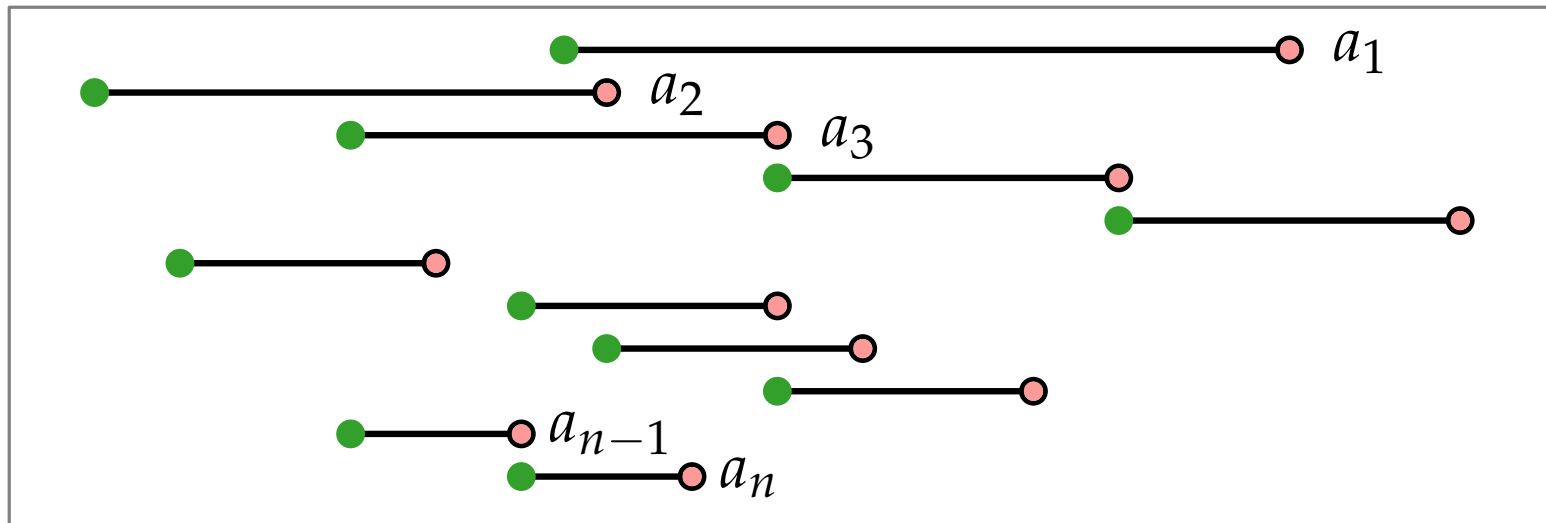
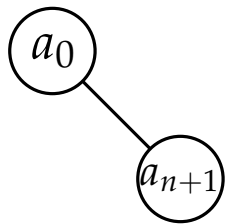
Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

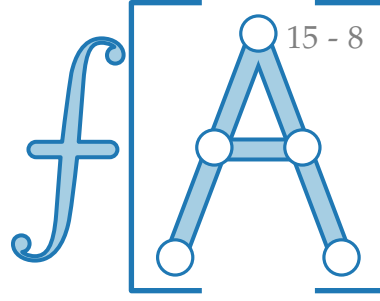
$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

T :



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

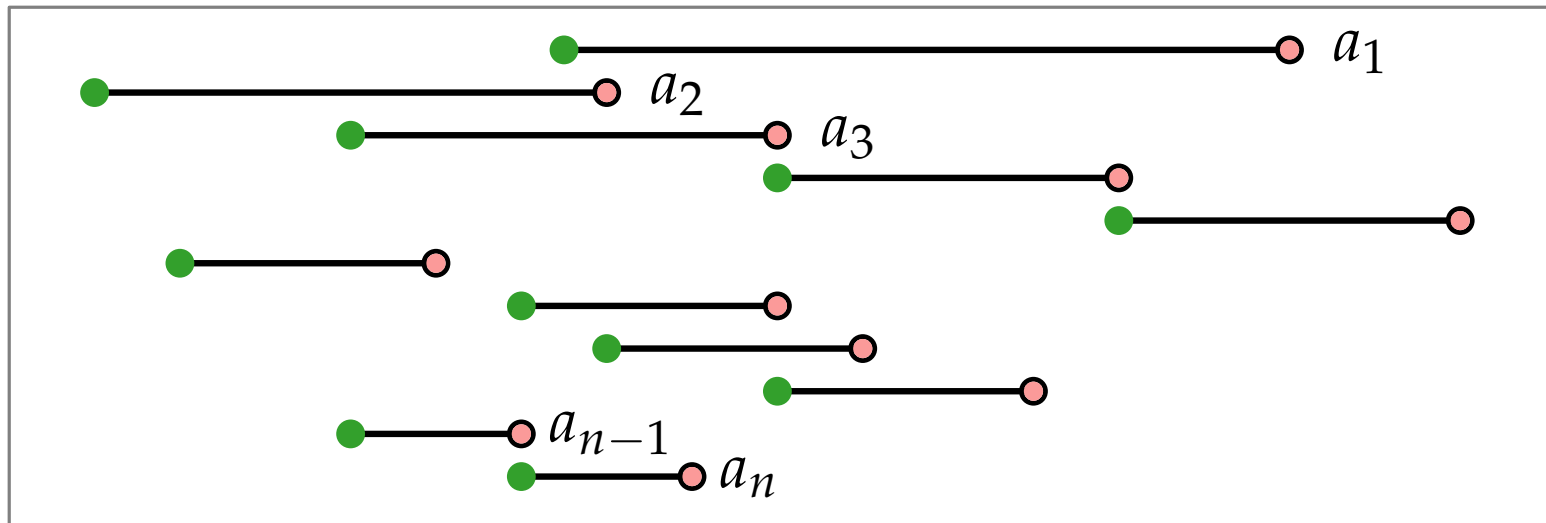
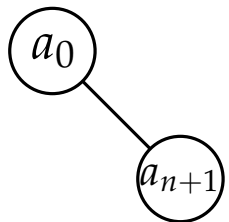
// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

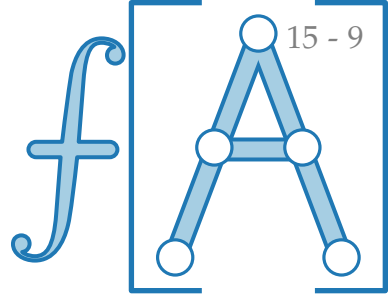
for $i = 1$ to n do

return T ;

T :



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

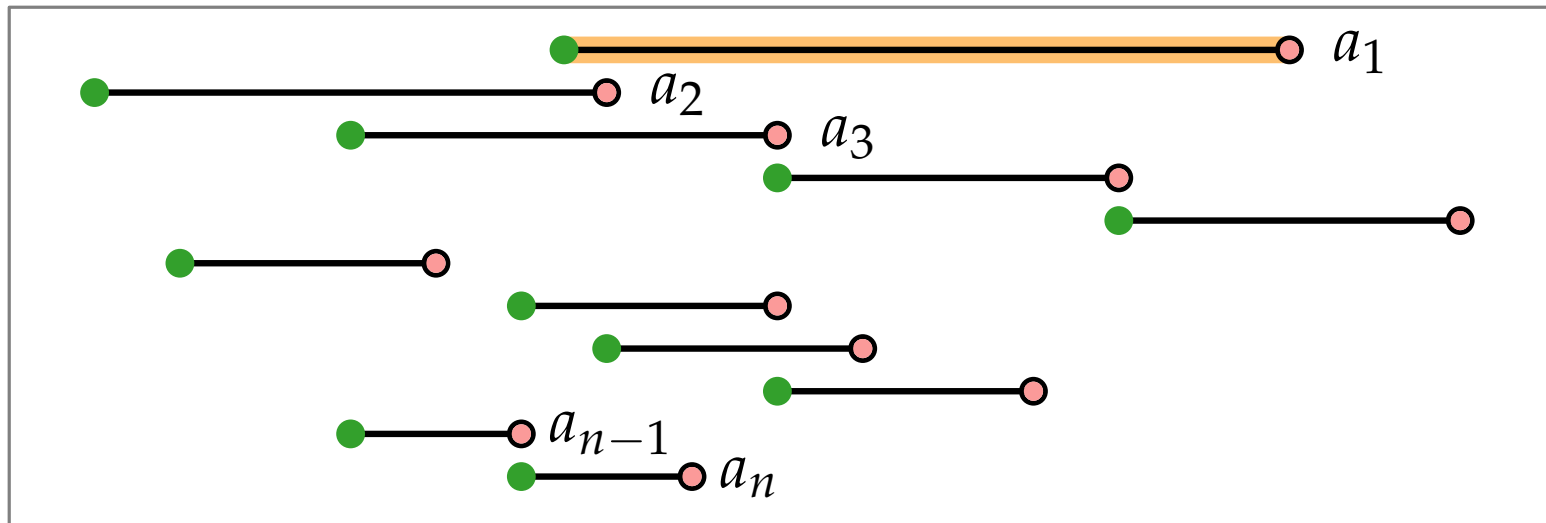
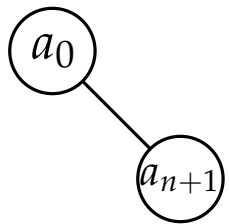
// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

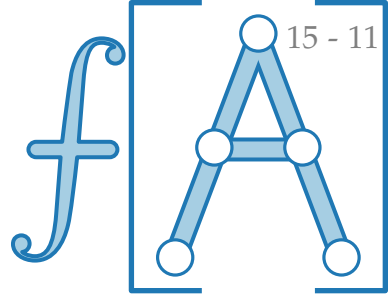
for $i = 1$ to n do

return T ;

T :



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

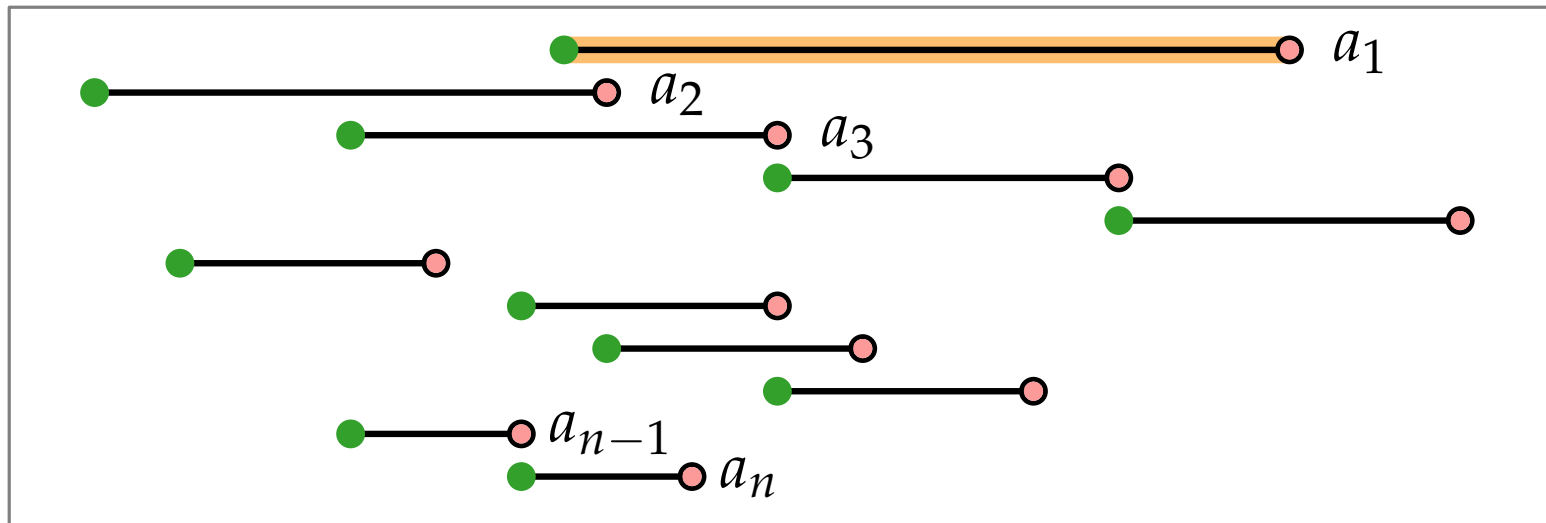
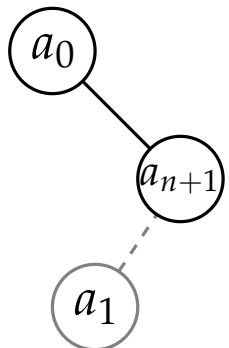
$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

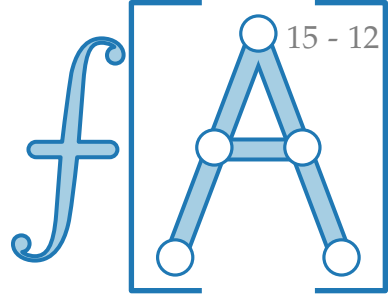
 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

return T ;

T :



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

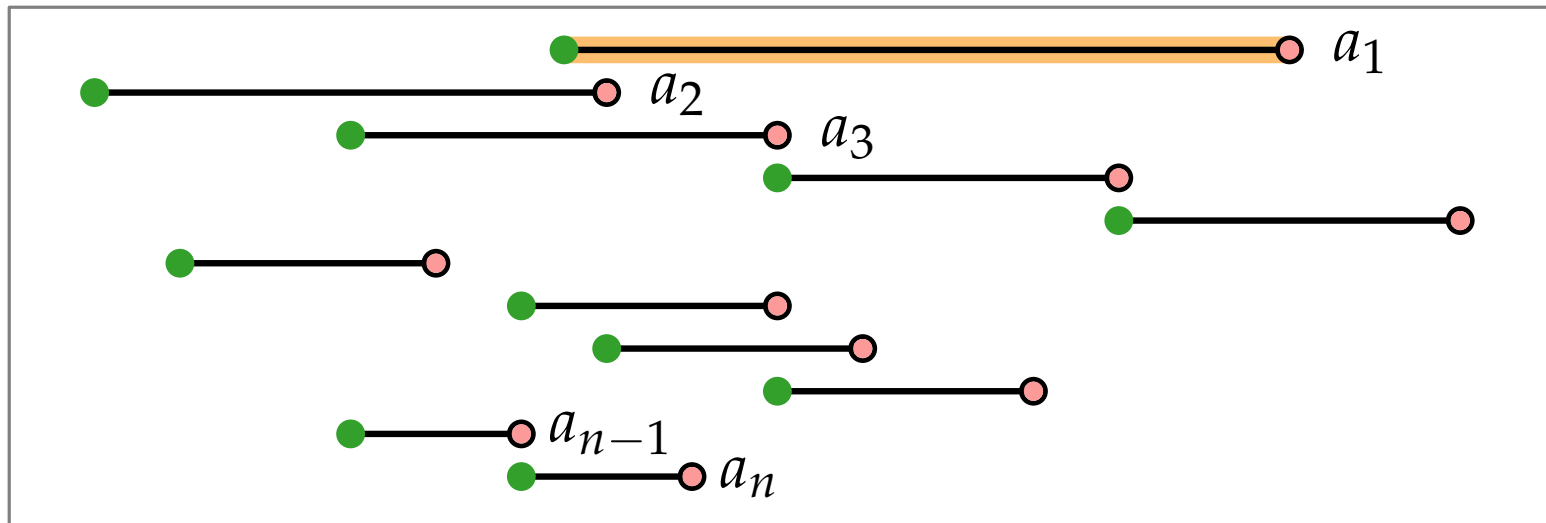
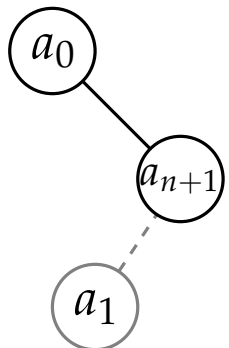
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

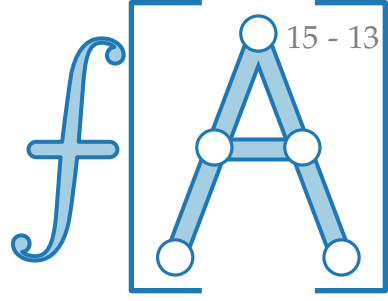
if $e_j < s_i$ **and** $s_k > e_i$ **then**

return T ;

T :



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

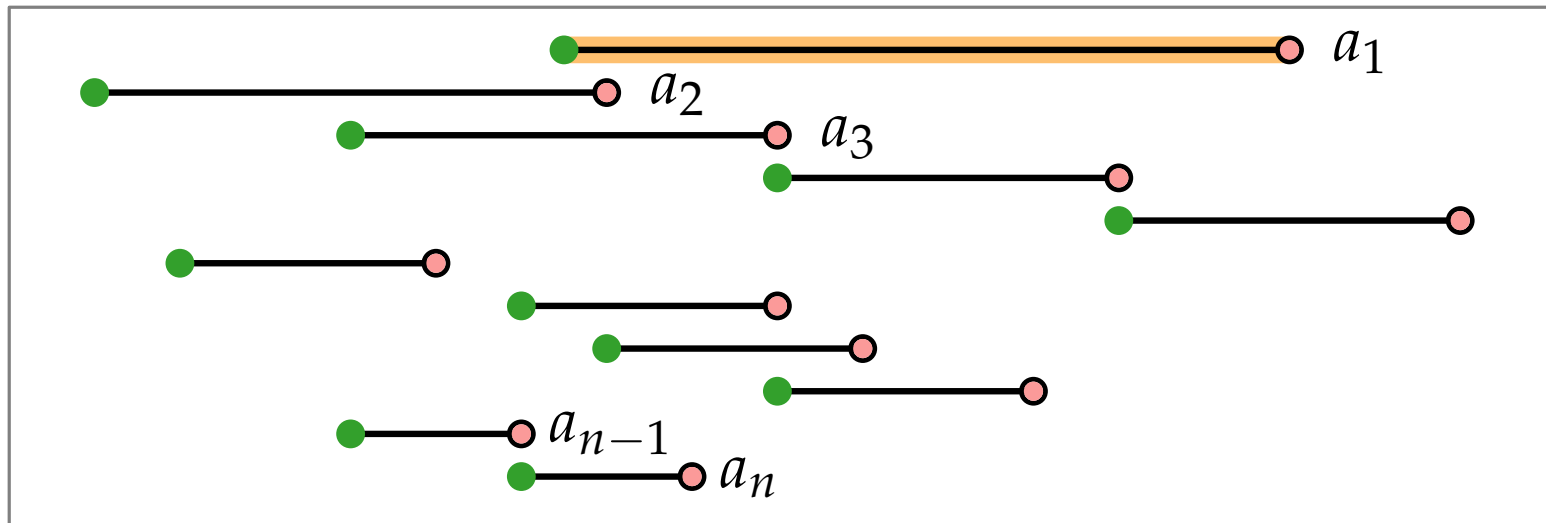
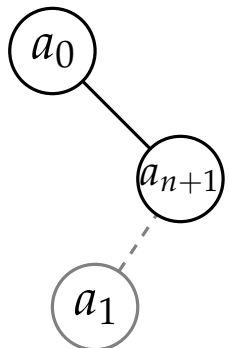
 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

if $e_j < s_i$ **and** $s_k > e_i$ **then**

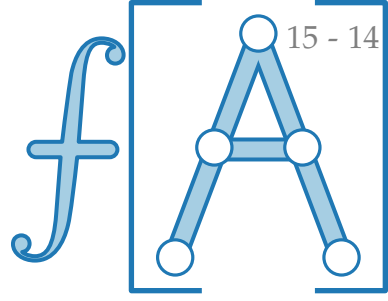
 Füge a_i in T ein

return T ;

T :



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

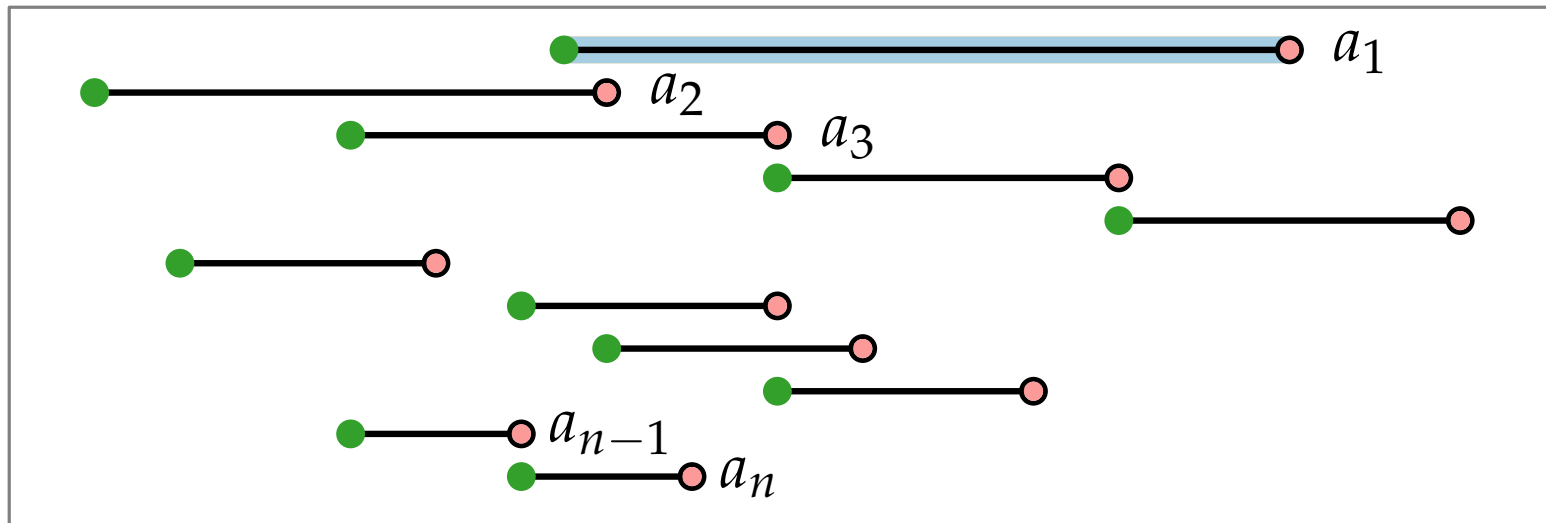
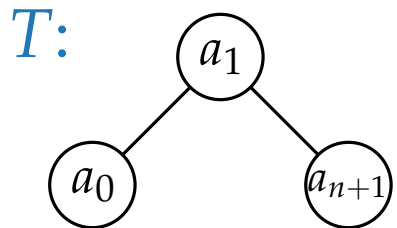
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

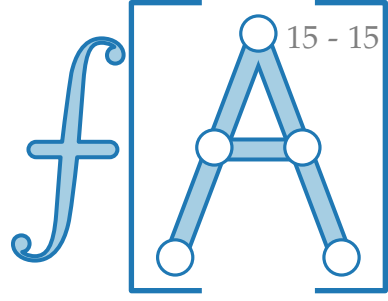
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

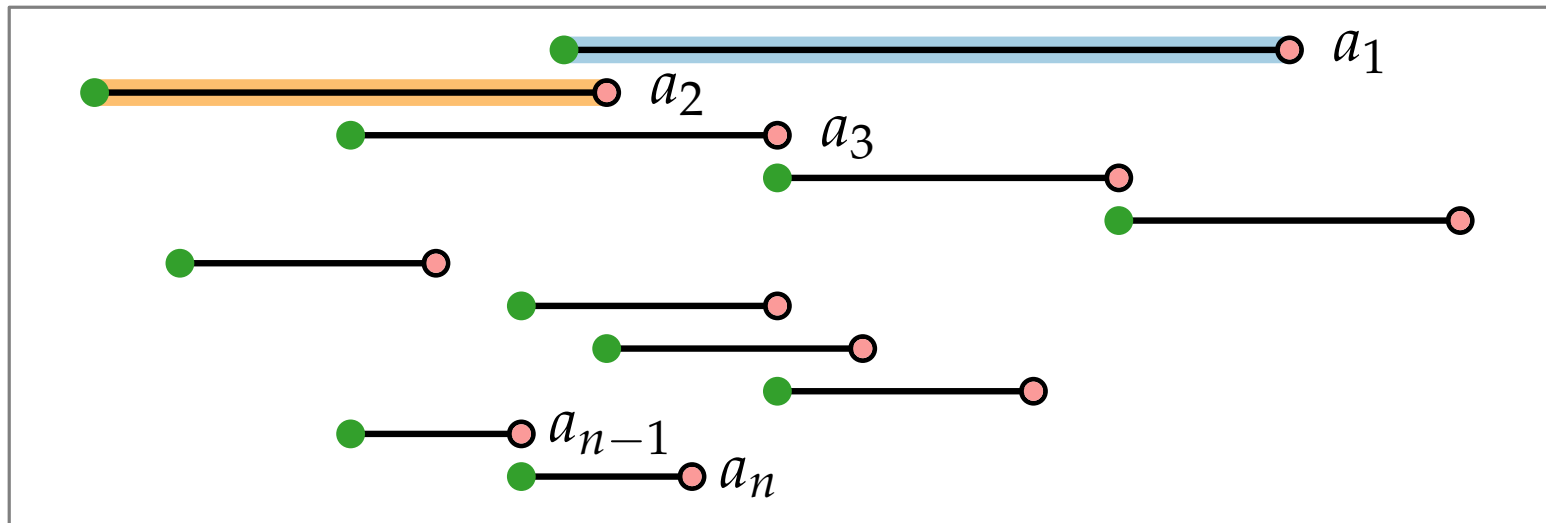
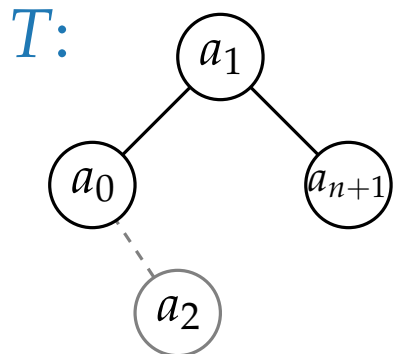
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

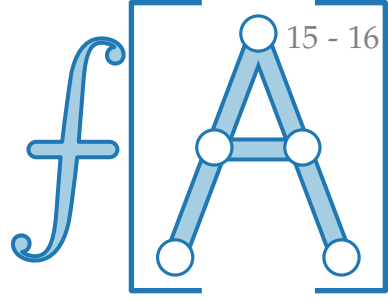
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

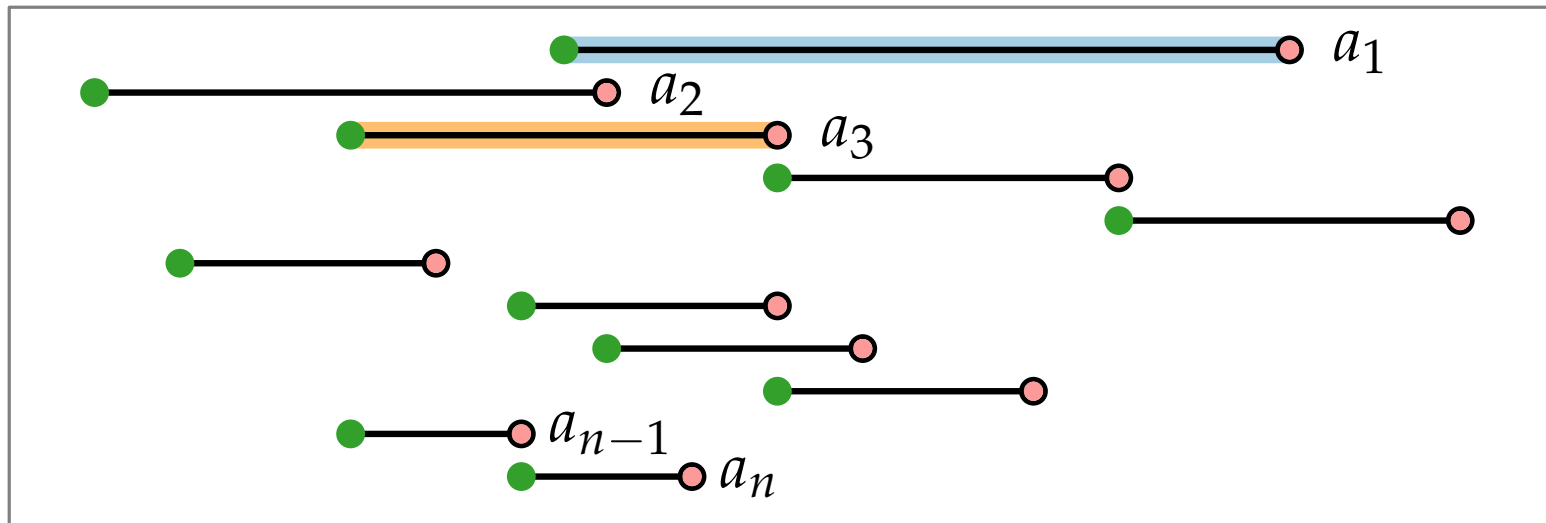
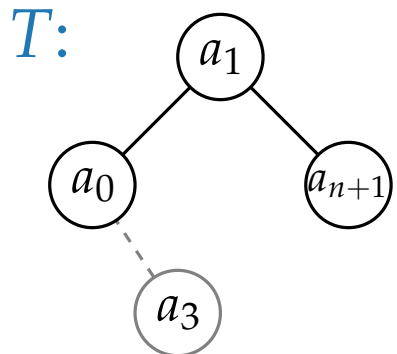
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

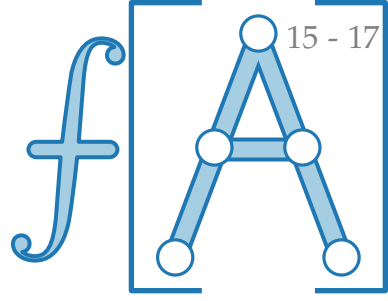
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

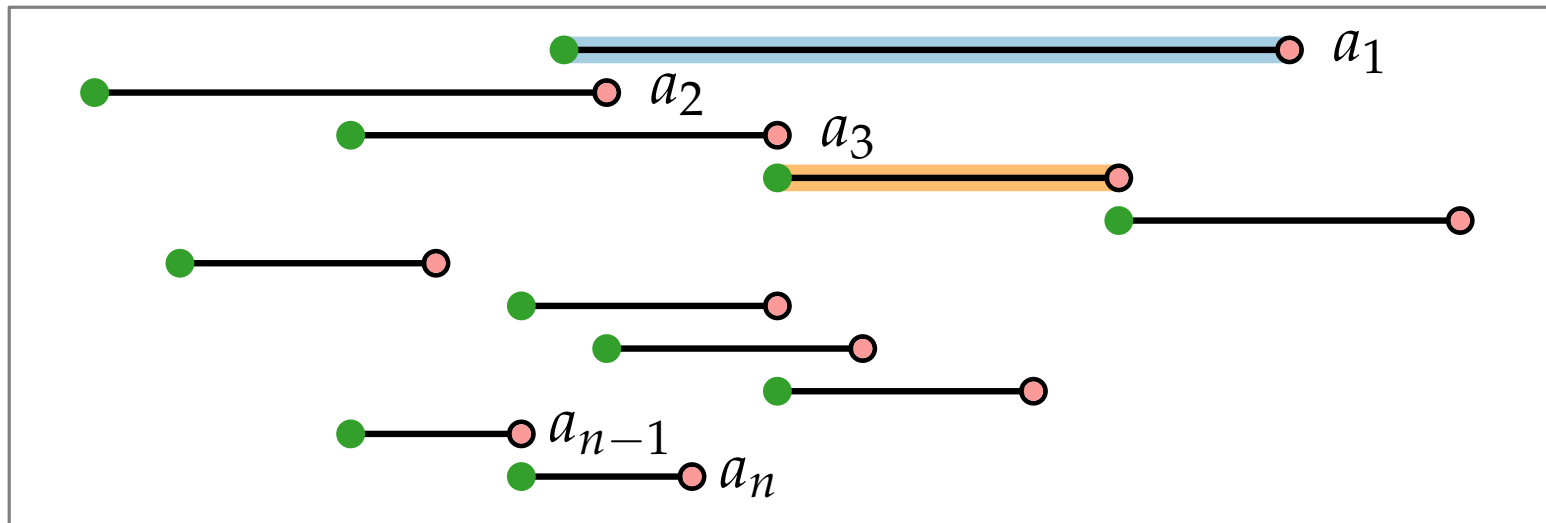
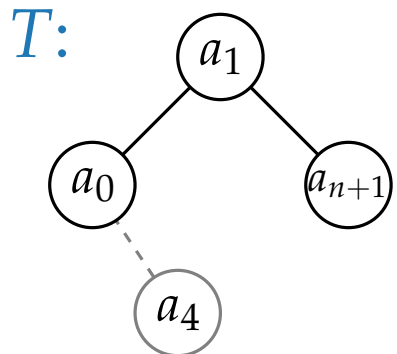
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

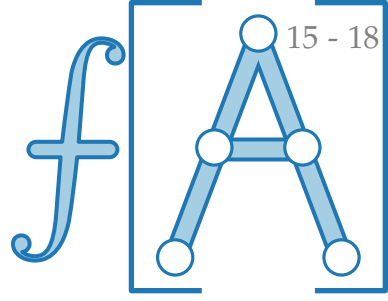
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

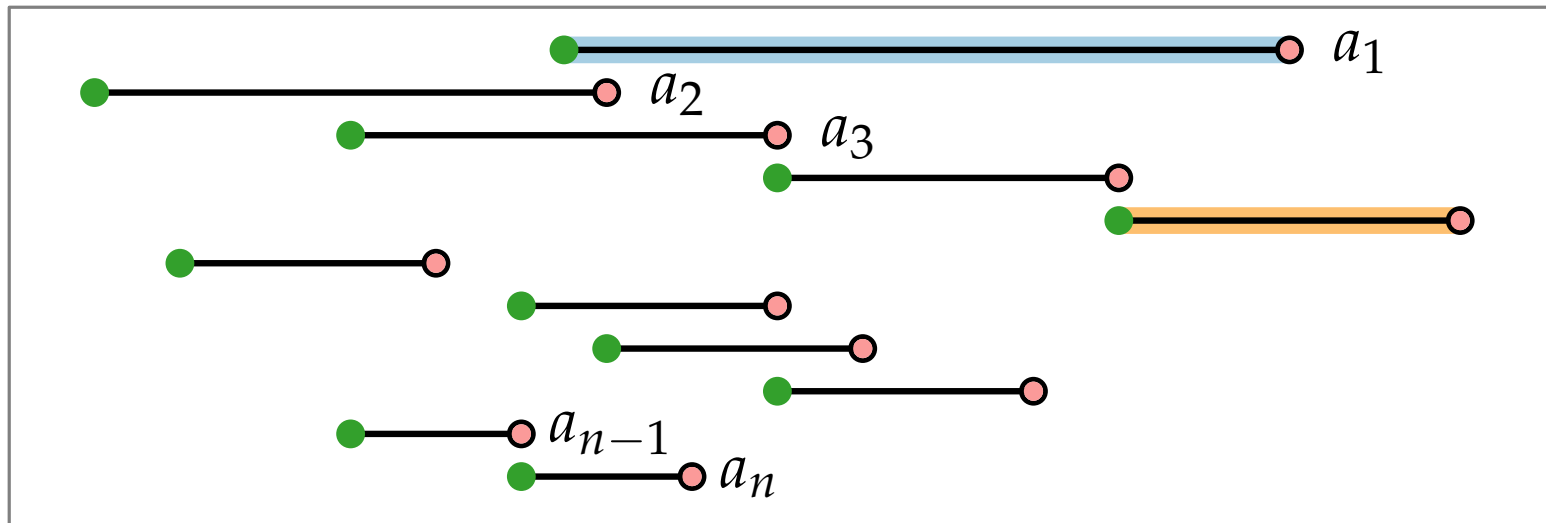
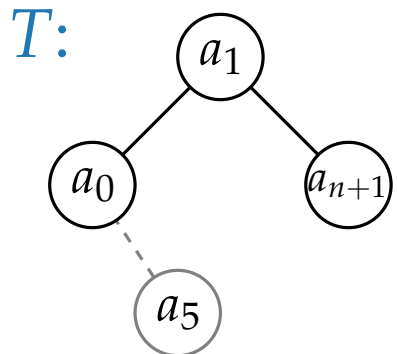
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

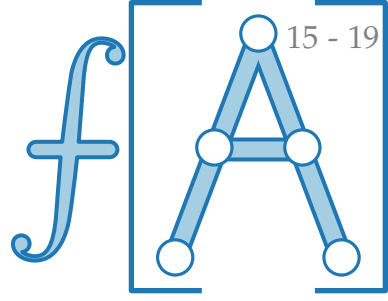
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

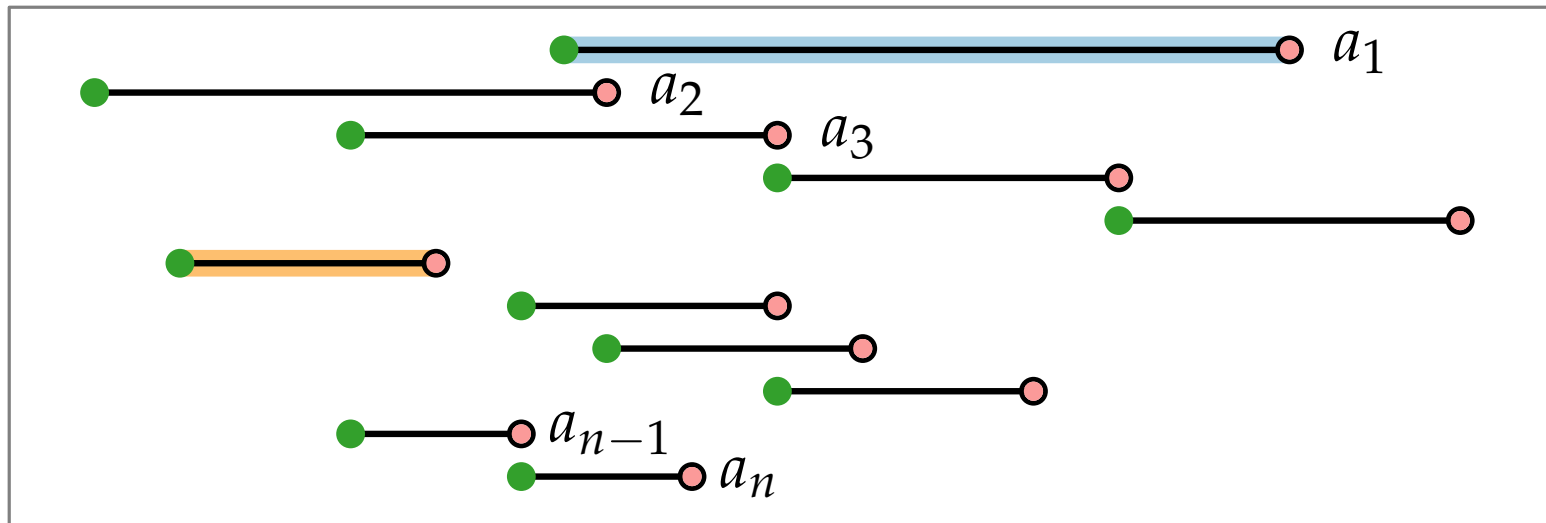
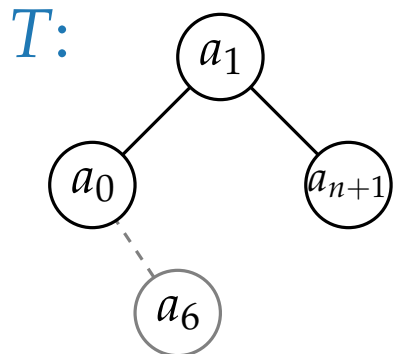
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

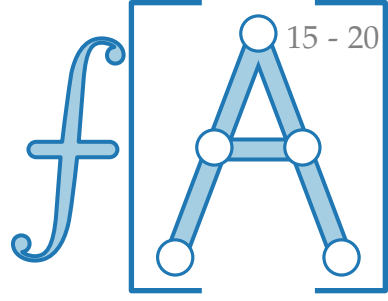
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

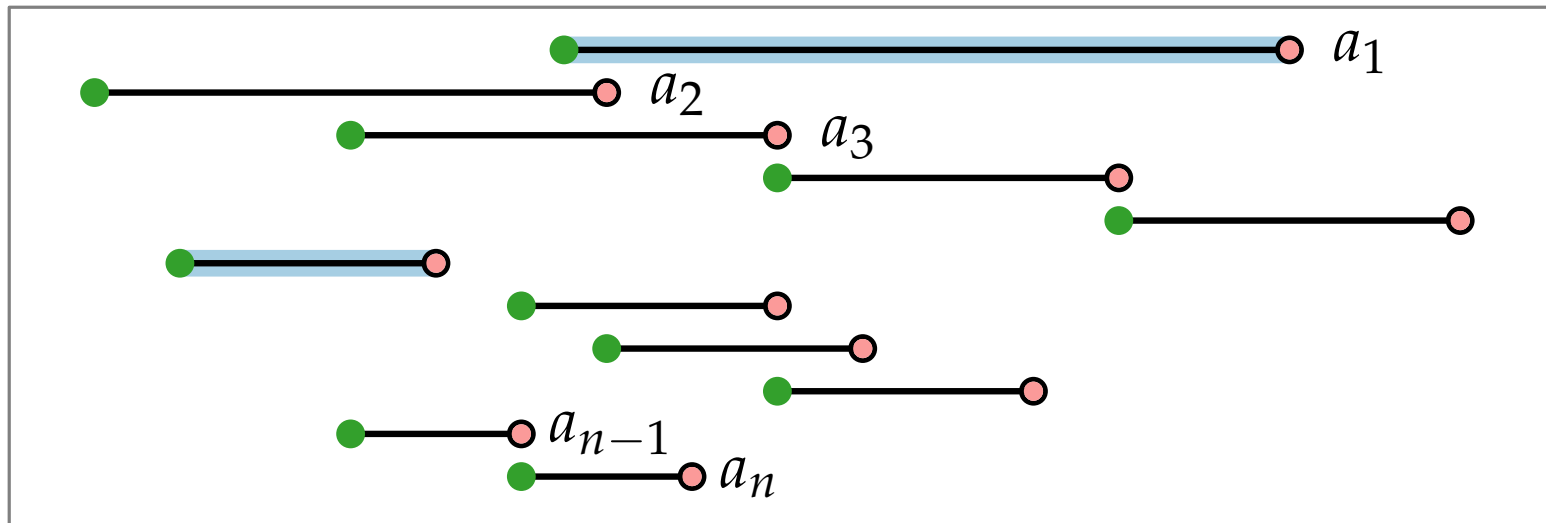
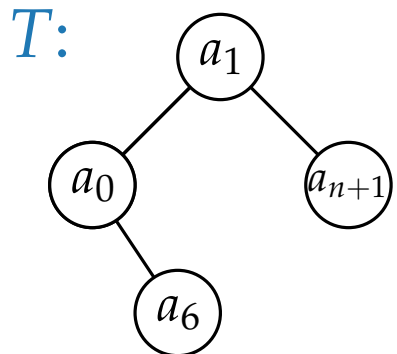
for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

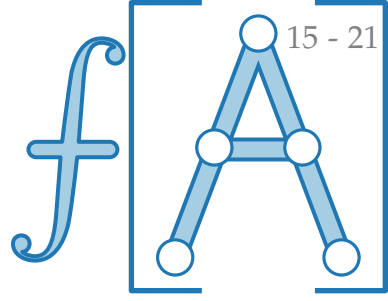
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

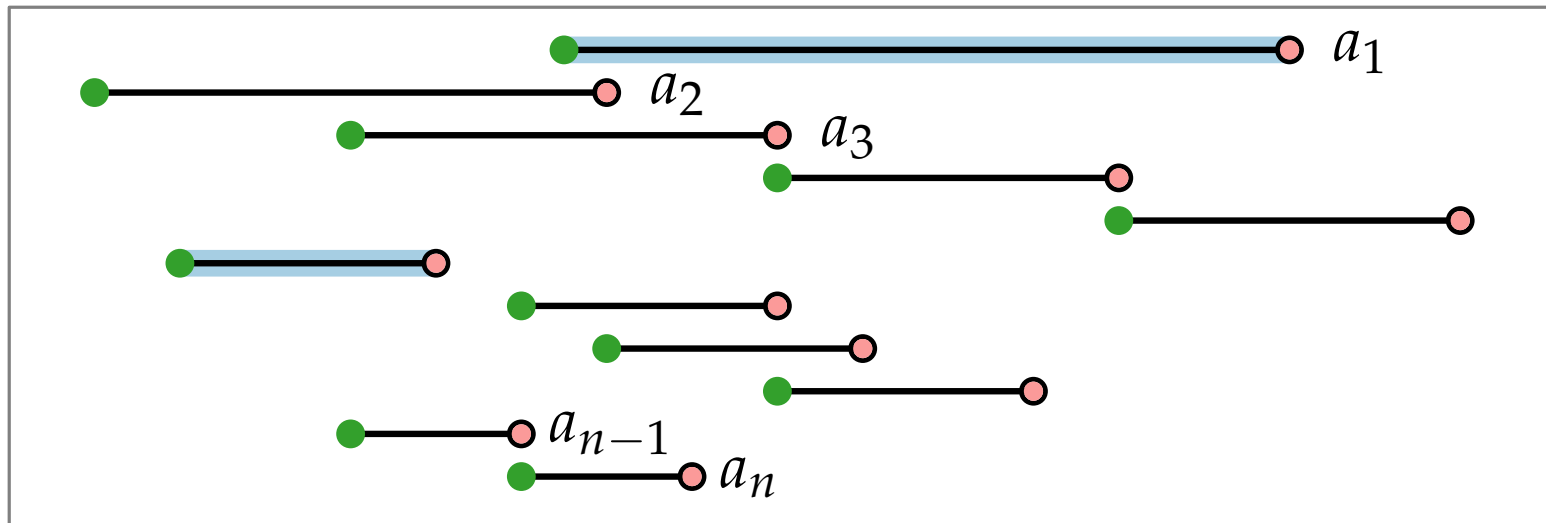
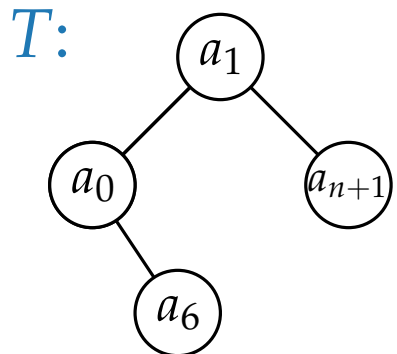
 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

if $e_j < s_i$ **and** $s_k > e_i$ **then**

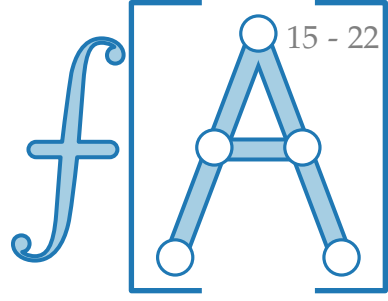
 Füge a_i in T ein

return T ;

Laufzeit.



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

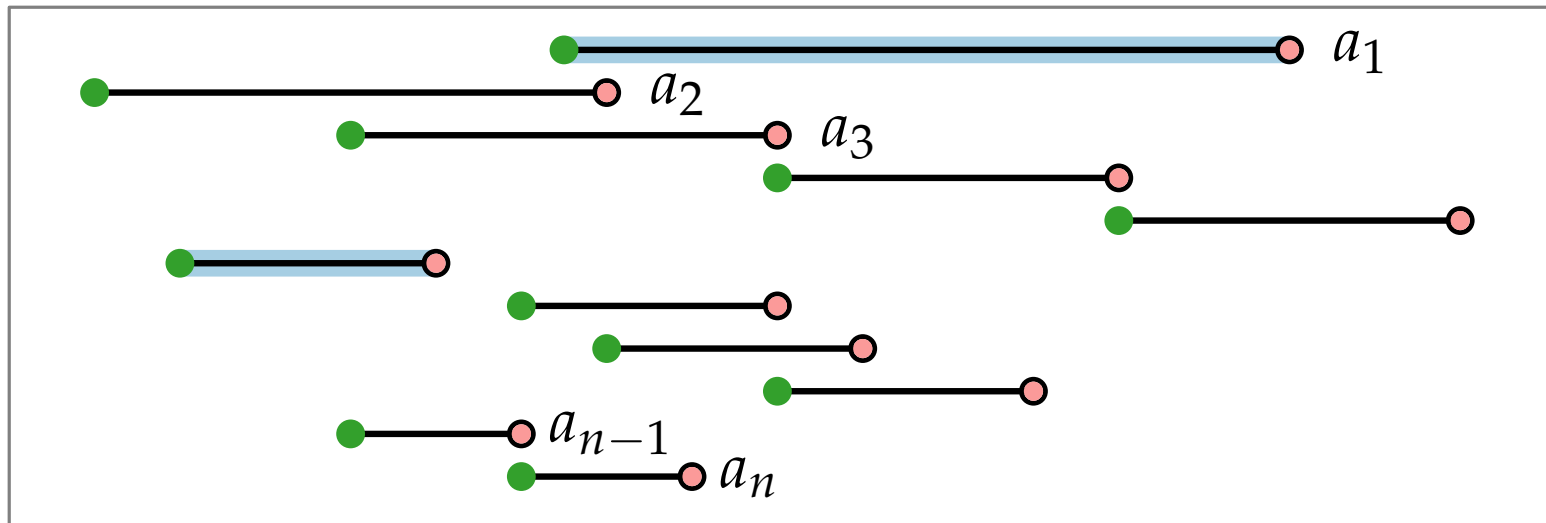
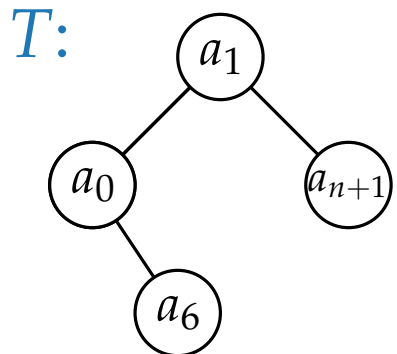
if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

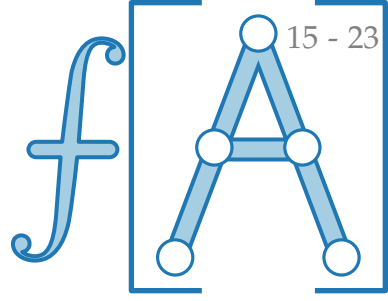
return T ;

Laufzeit.

$\mathcal{O}(n \log n)$



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

if $e_j < s_i$ **and** $s_k > e_i$ **then**

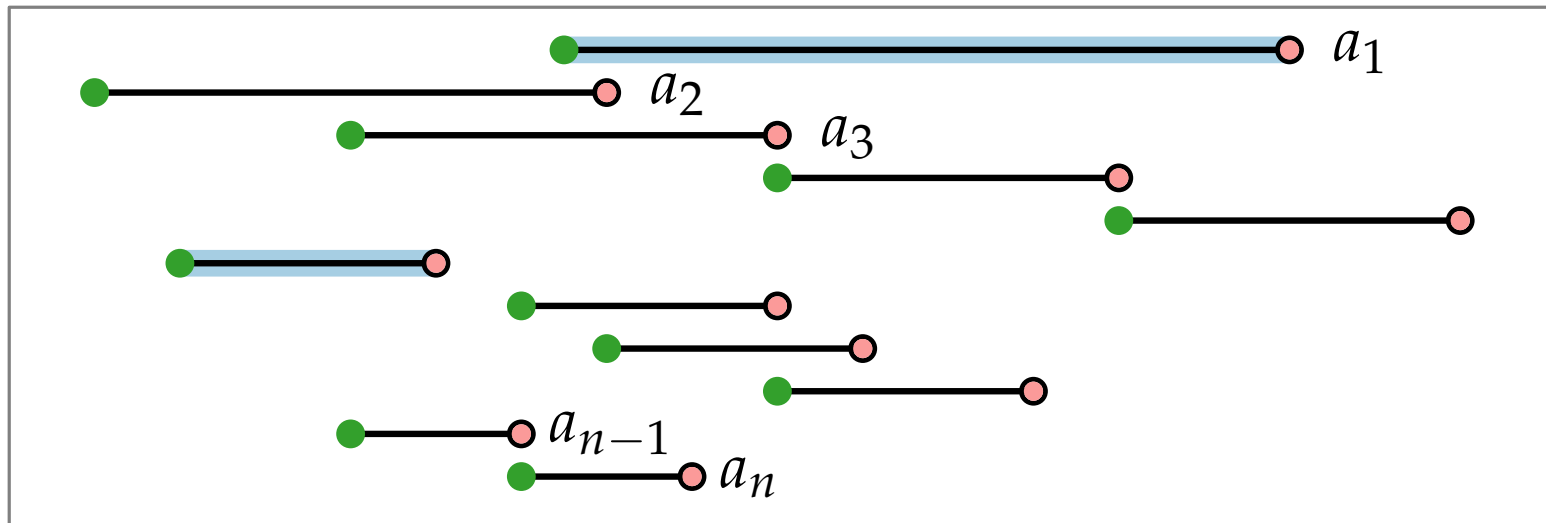
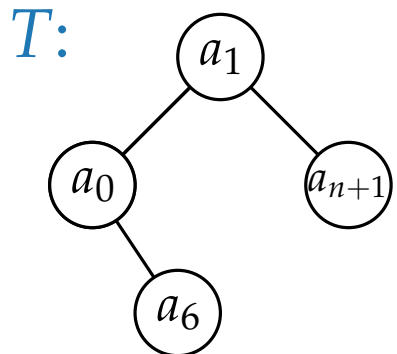
 Füge a_i in T ein

return T ;

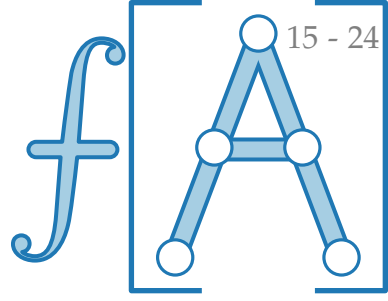
Laufzeit.

$\mathcal{O}(n \log n)$

$\mathcal{O}(1)$



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

if $e_j < s_i$ **and** $s_k > e_i$ **then**

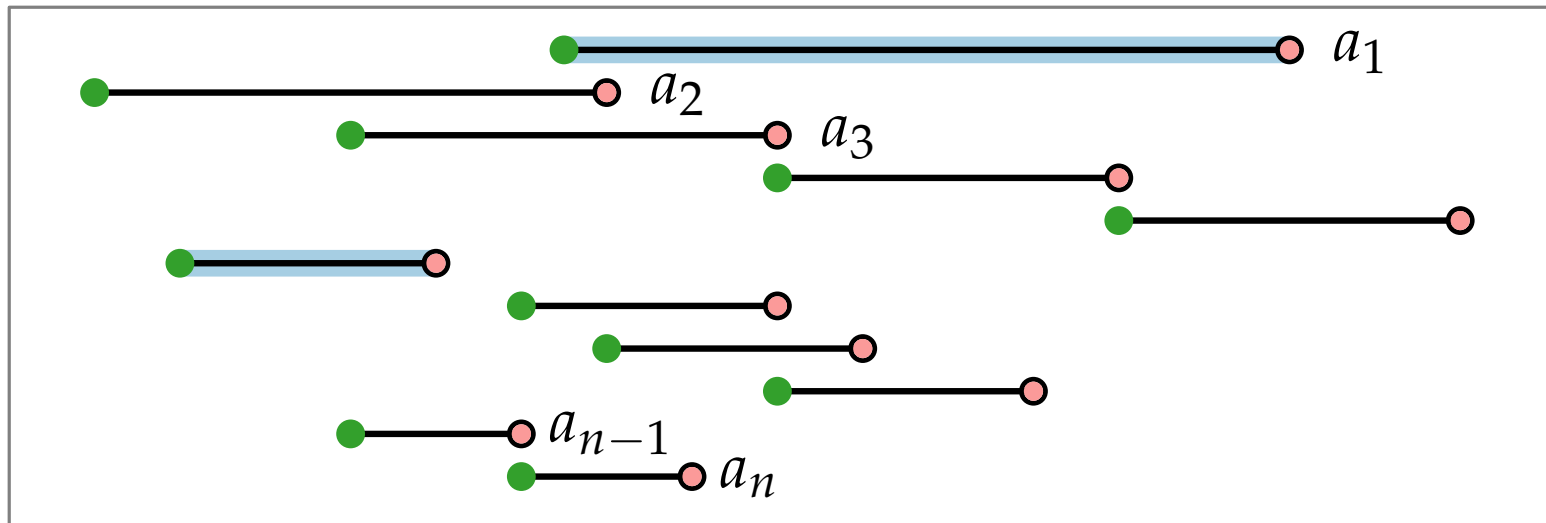
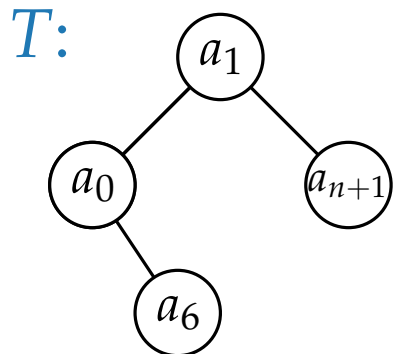
 Füge a_i in T ein

return T ;

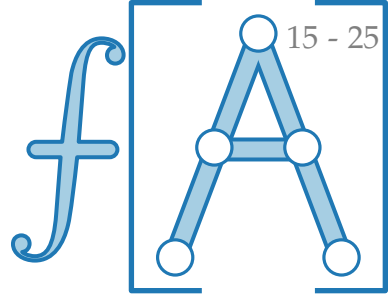
Laufzeit.

$\mathcal{O}(n \log n)$

$\mathcal{O}(1)$



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

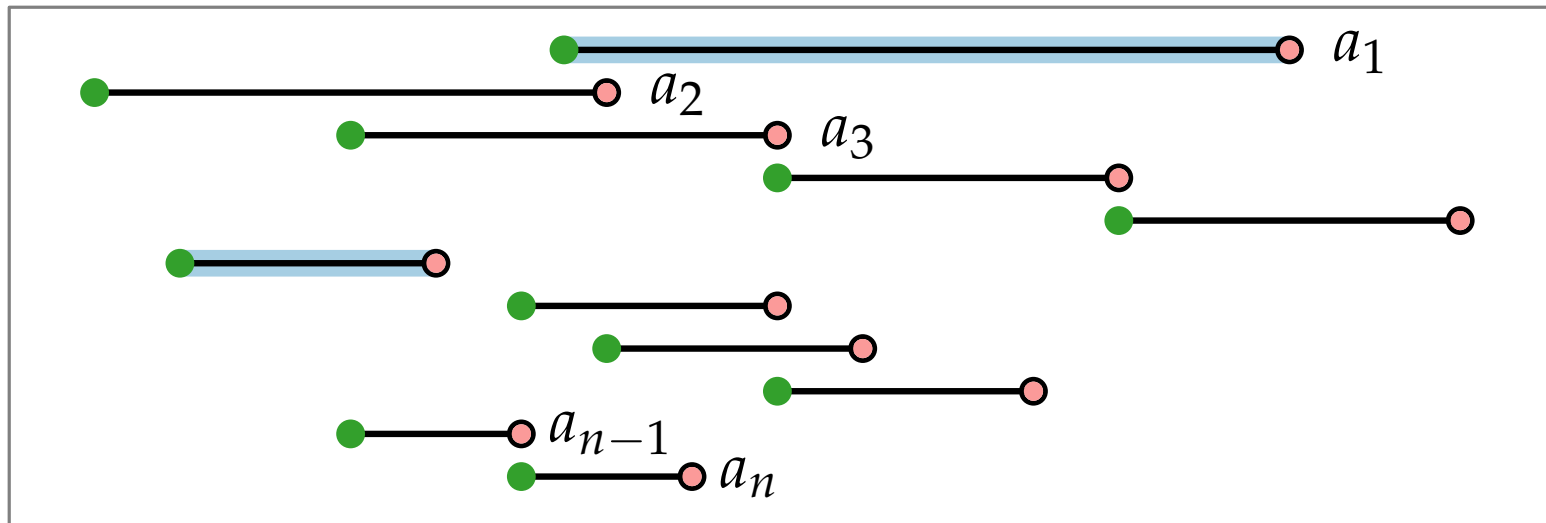
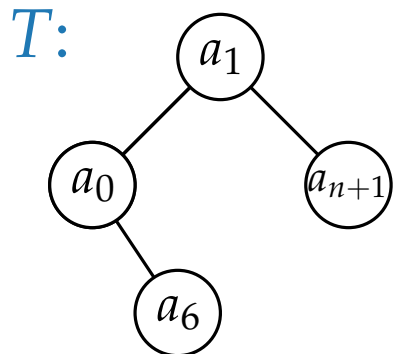
return T ;

Laufzeit.

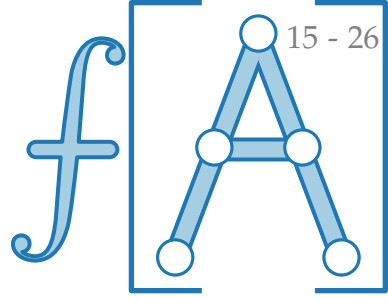
$\mathcal{O}(n \log n)$

$\mathcal{O}(1)$

$\mathcal{O}(n)$.



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;

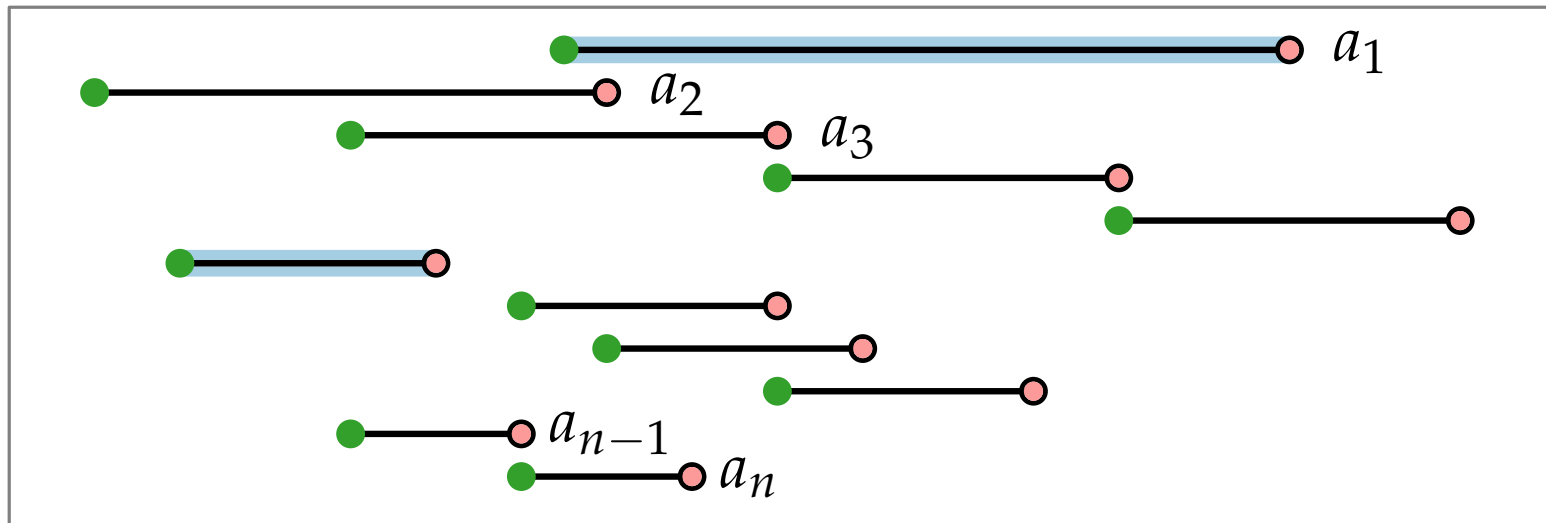
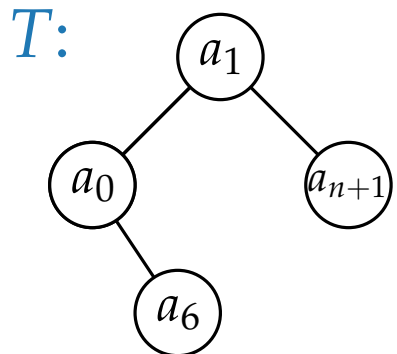
Laufzeit.

$\mathcal{O}(n \log n)$

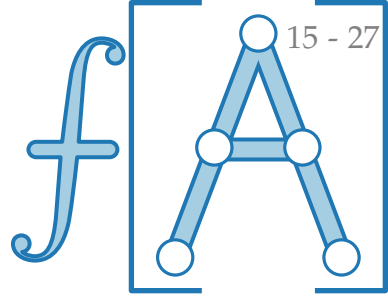
$\mathcal{O}(1)$

$\mathcal{O}(n)$.

$\mathcal{O}(\log n)$



2. GA in $\mathcal{O}(n \log n)$ Zeit



ZWEITERGA(int[] a)

Sortiere a nach Länge

// $(e_1 - s_1) \geq \dots \geq (e_n - s_n)$

$T \leftarrow$ binärer Suchbaum mit $a_0 = (-\infty, -\infty)$ und $a_{n+1} = (\infty, \infty)$.

for $i = 1$ **to** n **do**

 Suche Vorgänger a_j und Nachfolger a_k von a_i in T

if $e_j < s_i$ **and** $s_k > e_i$ **then**

 Füge a_i in T ein

return T ;

Laufzeit.

$\mathcal{O}(n \log n)$

$\mathcal{O}(1)$

$\mathcal{O}(n)$.

$\mathcal{O}(\log n)$

$\mathcal{O}(\log n)$

